



Bíblia do Power System

Laser&Co Power System — Documento oficial de homologação
Mapa completo, módulo por módulo, ancorado no código real

5 módulos · 7 submódulos · ~69 funcionalidades · 124 telas · 57 backends

Gerado em 06/07/2026 por leitura automatizada de todo o repositório + validação no banco de produção

Para confrontar com outra IA: cada funcionalidade traz rota, RBAC, tabelas/ações, integrações, estado real com evidência (arquivo:linha) e esforço estimado.

Bíblia do Laser&Co Power System

Documento oficial de homologação — mapa completo do sistema, módulo por módulo, ancorado no **código real** (não em achismo). Gerado em 06/07/2026 por leitura automatizada e paralela de todo o repositório (`src/` + `scripts/` + `docs/`) com validação cruzada no banco de produção (Supabase `lkiihnxznpqkrgsg`).

Como usar: este arquivo é a camada executiva. Os detalhes tela-a-tela estão nos 5 apêndices (`parte1` ... `parte5`). Para a auditoria confrontada por outra IA (GPT), envie este arquivo + os 5 apêndices; cada funcionalidade traz evidência de código (arquivo:linha), estado real e esforço estimado — dá pra checar item por item.

0. Metodologia (por que confiar nestes números)

- Fonte de verdade da hierarquia: `src/lib/menu.ts` (o menu É a estrutura oficial de módulos/submódulos/funcionalidades, portado 1:1 do legado).
- Estado "funcional" não é opinião: cada tela foi classificada por **evidência** — a server action grava de verdade (`insert/update/delete` reais)? a fonte tem dado no banco? é mock/preview? O código carrega um `Set` chamado `ROTAS_FUNCIONAIS` que declara o que está funcional; conferimos linha a linha se é honesto.
- Convenções: **FUNCIONAL** = grava/lê de verdade, no ar. **PARCIAL** = a tela e as ações existem, mas falta dado, integração de 3º ou uma sub-feature. **FALTA/STUB** = é código a construir (a tela é casca/simulação).
- Separação essencial de prazo: **dev nosso** (o que depende só de programar) × **bloqueio de terceiro** (banco, prefeitura, time do site, credencial do cliente). Misturar os dois é o erro clássico que faz um cronograma mentir.

1. O mapa: 5 módulos, 7 submódulos, ~69 funcionalidades, 124 telas

#	MÓDULO (SEÇÃO DO MENU)	SUBMÓDULOS (GRUPOS)	FUNCIONALIDADES	TELAS (ROTAS)
1	Acompanhamento	—	Dashboard, Agenda, Ordens de Serviço, PDV	4
2	Cadastros	Cadastros básicos (13)	+ Clientes, Colaboradores, Contas da Unidade, Pacotes, Produtos, Serviços	~21
3	Gestão	Relatórios (24), Dashboards (7), RH (9)	+ Automações, Disparos, CRM, Leads do Site, Canais, Indiques, Marketing, Comunicados, Chamados, Checklist, Universidade, Disco, Notas	~57
4	Administração	Expansão (7), SAC (11), Financeiro Franqueadora (9)	+ Implantação, Jurídico, Auditoria	~30
5	Rede & Conta	—	Minha Unidade, Todas Unidades, Minha Conta, App do Cliente, Exportações	5

Total real medido no filesystem: **124 rotas** `page.tsx` + **57 arquivos de server actions** (backend). A percepção do Julio (5 módulos / 36 submódulos / 69 funcionalidades) estava correta — apenas contando "submódulo" como cada grupo + cada tela de 2º nível.

2. Sumário executivo do estado

O **núcleo operacional e financeiro do dia a dia já roda em produção** com dados reais. O que falta divide-se em (a) sub-features acessórias, (b) integrações que dependem de terceiro, e (c) um módulo inteiro ainda não construído: a **visão do Franqueado**.

Verde (no ar, funcional e com dado real): - Financeiro da Franqueadora: razão único `fin_lancamento` como fonte da verdade; DRE (mensal + anual + escopo combinável), Fluxo, Contas a Receber/Pagar, Conciliação (por planilha), Automação de Royalties (cálculo real a partir do faturamento BEMP). *Núcleo do sistema, sólido.* - SAC em produção real: WhatsApp (Uazapi) recebendo/ enviando, IA de 1º atendimento abrindo chamado sozinha, triagem, kanban, atendentes, ranking. - Agenda (~136–156k agendamentos importados do BEMP), Ordens de Serviço, Clientes (~347k com CPF), catálogo (serviços/pacotes/produtos) — todos CRUD real. - Cadastros básicos (13 telas): plano de contas, comissões, contratos, motivos, planos, perfis de acesso (RBAC persiste de verdade), origens, anamnese — todos gravando. - CRM, Leads do Site, Expansão (funil de franquia), Implantação, Jurídico (lógica), Auditoria (leitura) — funcionais.

Amarelo (pronto, falta dado/uso/sub-feature): RH (folha/ponto/férias — motor pronto, sem dado operacional + falta login/salário), Metas da unidade (simulador não persiste), Produtos (sem movimentação de estoque), Parcerias (só cobre "desconto"), vários relatórios (estrutura pronta, fonte vazia).

Vermelho (é código a construir): 1. **Módulo Franqueado** — a visão do dono de franquia não existe (ver §4). *O maior item novo.* 2. **Motor de automações** — as 22 automações de `/automacoes` são catálogo liga/desliga **sem executor**; nenhum worker as dispara. 3. **Boleto bancário real** — hoje é linha digitável simulada (sem CNAB/Open Finance). 4. **NFS-e** — stub, sem provedor da prefeitura. 5. **E-mail** — não há envio real (só `mailto:` e placeholders). 6. **Google Drive** — stub (pastas hardcoded, sem API); o Disco Virtual próprio (Supabase Storage) é real. 7. **Régua de cobrança automática** — a config salva, mas o disparo por dias de atraso não roda sozinho. 8. **App do Cliente** — vitrine/mockup. 9. **Robô de documentos do BEMP** — bloqueado no login do app BEMP (infra pronta).

3. Camada de integrações transversais (auditada no código)

Existem **exatamente 2 route handlers** no sistema (`src/app/api`): `webhooks/uazapi` e `cron/ingest-sac` . Todo o resto é Server Action. Estado real de cada integração:

#	INTEGRAÇÃO	ESTADO	EVIDÊNCIA-CHAVE
1	Site → SAC/ Leads	REAL (falta o site emitir <code>tipo='sac'</code>)	<code>src/lib/sac-ingest.ts</code> insere em <code>sac_tickets</code> ; lê a tabela <code>lasercompany_leads</code> do Supabase do site (<code>riutcbwillvqjrpaeafb</code>) por pull direto (não webhook). Idempotente. Depende do env <code>SITE_SUPABASE_SERVICE_KEY</code> em prod.
2	WhatsApp (Uazapi)	REAL	<code>src/lib/uazapi.ts</code> — envio (<code>/send/text</code> , <code>/sender/simple</code>) e webhook de recebimento gravando <code>sac_whatsapp_chats/mensagens</code> . Evolution NÃO existe (zero no código). Erro 463 (restrição de número novo) tratado honestamente.
3	API / Webhooks	REAL	Só 2 rotas: <code>webhooks/uazapi</code> (entrada + IA + abre chamado) e <code>cron/ingest-sac</code> (protegido por <code>CRON_SECRET</code>).
4	Cron / disparos programados	PARCIAL	Não há pg_cron/pg_net no banco (zero no SQL). Único agendador real = Vercel Cron (<code>vercel.json</code> , 06:00 diário → ingest-sac). Campanhas agendadas: a fila é da Uazapi (externa), não nossa.
5	E-mail	STUB	Sem resend/nodemailer/smtp. <code>notificarCobranca</code> é "placeholder honesto"; UI diz "enviado por e-mail" mas nada sai do servidor.
6	Banco / Conciliação / Boleto	PARCIAL	Conciliação por planilha = real (cruzamento linha a linha, tolerância R\$0,05). Boleto e baixa = MOCK (linha digitável fake, sem CNAB). O próprio sistema avisa na tela: "este módulo simula o ciclo".
7	BEMP	PARCIAL	Imports de clientes/OS/agenda/faturamento/colaboradores = REAIS (Postgres direto → Supabase). Robô de documentos = STUB (2 funções lançam erro; bloqueado no login do app BEMP).
8	Storage	PARCIAL	Disco Virtual (Supabase Storage, bucket <code>disco-virtual</code>) = REAL (upload/download/signed URL). Google Drive = STUB (pastas hardcoded, sem OAuth/API).

Royalties (o coração): o **cálculo é real** (faturamento BEMP por unidade → `fin_lancamento` + `fin_recebeveis` , com royalty 10%, desconto <R\$80k, loja própria não paga). O **gatilho é manual** (botão "Apurar", operador escolhe a competência). A **emissão do boleto e a notificação ao franqueado são stub**.

4. RBAC e a questão Franqueado × Franqueador (validado ao vivo)

Modelo: empresas → unidades; `perfis_usuario` (papel enum: `admin_geral`, `gestor`, `financeiro`, `crm`, `sac`, `operacoes`, `rh`, `tecnico`, `colaborador`) ↔ `usuario_cargos` → `cargos` → `cargo_permissoes` → `permissoes` (recurso × ação × escopo). O menu filtra por `perm` de cada folha. `admin_geral` = bypass total (vê tudo). Escopo multitenant por `activeUnitId` (a unidade do perfil; `null` = vê todas).

Achado crítico (testei criando um usuário franqueado real, cargo `admin_franqueado` amarrado a 1 franquia, e renderizei ao vivo):

O "franqueado" hoje **enxerga praticamente tudo que o franqueador enxerga** — 105 itens de menu, incluindo *Automação de Royalties*, *Contas a Receber da Franqueadora*, *Todas as unidades*, *Financeiro Franqueadora* inteiro, SAC, Expansão. O cargo existe, mas suas permissões liberam quase tudo, e **todas as telas foram desenhadas para o franqueador**.

Conclusão: a experiência do franqueado **não existe de fato** — é um módulo a construir, não um ajuste. O que falta: - **RBAC escopado de verdade:** o cargo `admin_franqueado` precisa perder acesso a tudo que é da rede (royalties, financeiro da franqueadora, todas-as-unidades, expansão, SAC central) e ficar só com a operação da SUA loja. - **Telas escopadas:** agenda, clientes, colaboradores, contas, relatórios — todas filtradas à unidade dele (a fundação `activeUnitId` já existe; falta forçá-la e esconder o resto). - **DRE do franqueado:** hoje só existe o DRE do franqueador (onde a receita da franquia não aparece, só o royalty — decisão do Rafael). O franqueado quer o **faturamento cheio da loja dele**. São **dois DREs diferentes** tratados como um

só — daí a sensação de "ora tira, ora põe franquia". - **(decisão do Rafael)**: o franqueado terá **visual próprio** (outro layout) ou é o mesmo sistema com telas escopadas? Isso muda o tamanho do módulo.

5. Prazo consolidado (dev nosso x bloqueio de terceiro)

Estimativas em **dias de dev** somadas dos apêndices. "Dev nosso" = só programar. "3º/cliente" = destrava rápido quando o insumo chega, mas o relógio é de fora.

5.1 Dev nosso (o que dá pra cravar cronograma)

BLOCO	DIAS DE DEV	OBSERVAÇÃO
Módulo Franqueado (RBAC escopado + telas da loja + DRE da unidade)	10–15	mesmo sistema, telas escopadas. +5–8 se for visual próprio
Motor de automações (executor real das 22 regras)	4–6	hoje é catálogo inerte
Régua de cobrança automática	3–4	disparo por dias de atraso
RH (folha/ponto/férias — fiação + login + salário)	5–8	motor pronto, falta dado/Auth
Sub-features de cadastro (metas-unidade, estoque, parcerias, produtos/ serviços paridade)	6–9	acessórias
Expansão (webhook do site, status de disparo, criar-projeto implantação)	~10	grande parte já funcional
Jurídico (e-mail/storage/assinatura reais)	5–8	lógica pronta
App do Cliente (app operacional real)	15–25	o maior item isolado
Ajustes/relatórios/migrations pendentes	5–8	aplicar migrations no prod, reconciliar RBAC

Subtotal dev nosso: ~65–90 dias de dev. Sem o App do Cliente (que é praticamente um produto à parte): ~50–65 dias.

5.2 Bloqueios de terceiro (prazo não é nosso)

ITEM	DEPENDE DE	DEV QUANDO DESTRAVAR
Boleto real (CNAB/Open Finance)	convênio + credenciais do banco	5–8 dias
NFS-e	provedor fiscal da prefeitura	5–8 dias
E-mail	contratar Resend/SMTP (decisão do cliente)	2–3 dias
Google Drive real	OAuth Google (decisão)	3–4 dias
Robô de docs BEMP	Lucas definir senha do app BEMP	2 dias
Leads/SAC do site 100%	time do site publicar <code>tipo='sac'</code> + env em prod	~1 dia

5.3 Leitura de calendário

O operacional + financeiro do dia a dia **já está no ar**. Para "100% de verdade" com as integrações reais, a faixa honesta é ~6 a 10 **semanas de dev** (sem o App do Cliente; com ele, some ~3–5 semanas). Boleto e NFS-e não entram nesse relógio — são de banco/prefeitura, e programamos rápido assim que o convênio existir.

6. Índice dos apêndices detalhados (tela a tela)

- parte1-acompanhamento-cadastros.md** — Dashboard, Agenda, OS + os 13 Cadastros básicos + Clientes/Colaboradores/ Contas/Pacotes/Produtos/Serviços.
- parte2-relatorios-dashboards.md** — os 24 Relatórios + os 7 Dashboards.
- parte3-gestao-operacao-rh.md** — Automações, Disparos, CRM, Leads, Canais, Indiques, RH (9), Marketing, Comunicados, Chamados, Checklist, Universidade, Disco, Notas.
- parte4-administracao.md** — Expansão (7), Implantação, SAC (11), Financeiro Franqueadora (9), Jurídico, Auditoria.
- parte5-rede-integracoes-rbac.md** — Rede & Conta (5) + a camada de integrações completa + o modelo RBAC e o gap Franqueado x Franqueador.

Cada apêndice traz, por funcionalidade: rota, RBAC (quem vê), o que faz, telas/modais, tabelas e server actions, integrações, estado real com evidência (arquivo:linha) e esforço para 100%.

Módulo 1-2 — Acompanhamento & Cadastros

Documento oficial de homologação. Gerado por leitura direta do código-fonte (`src/app/(app)/**`, `src/components/**`, `src/lib/**`, `scripts/migrations/*.sql`) e da hierarquia de menu (`src/lib/menu.ts`). Nada aqui é presumido: cada afirmação de estado vem de evidência de código (tabela `.from('...')`, `insert/update/delete` na server action, componente/modal importado, ou seed SQL).

0. Como ler este documento — RBAC e convenção de "funcional"

Modelo de visibilidade (fonte: `src/components/layout/Sidebar.tsx` + `src/lib/session.ts`). O menu é gateado por **recurso** (não por papel diretamente). Regra `canSee` :

- `admin_geral` (Super Admin / Administrador da franqueadora) **vê tudo**.
- Caso contrário, o item exige o `perm` (recurso). Sufixo `.` = qualquer recurso do módulo (`hasPerm: perm.endsWith('.') ? recursos.some(startsWith): recursos.includes`).
- Item sem `perm` = visível a todo usuário logado.
- Os `recursos` do usuário são resolvidos em `getSessionContext()` via `usuario_cargos` → `cargo_permissoes` → `permissoes.recurso_id` (service-role, memoizado por request).
- "Módulo único" (`ehSoModulo`): usuário cujos recursos são só `sac*` ou só `financeiro*` enxerga apenas aquele módulo + "Minha conta" (perfis SAC-only e Financeiro-only são redirecionados no Dashboard / — ver bloco Dashboard).

Papel (enum `papel_usuario`) — `colaborador`, `rh`, `gestor`, `admin_geral`, `financeiro`, `crm`, `tecnico`, `operacoes`, `sac` (+ `recepcao`, `gerente` adicionados por migration). O papel resolve o `isAdmin`; a granularidade fina vem dos **cargos** (17 perfis seedados em `scripts/migrations/perfis-acesso.sql`: Super Admin → Administrador → Diretor → ... → Profissional Técnico).

Mapeamento perm→quem-vê (deste escopo):

PERM (RECURSO)	MÓDULO	PERFIS/CARGOS QUE NATURALMENTE ENXERGAM
(nenhum)	—	todos os logados (Dashboard)
<code>operacoes.</code> / <code>operacoes.os</code>	Operações	<code>admin_geral</code> , Gerente/Operações, Recepção (cargos com recurso <code>operacoes*</code>)
<code>comercial.</code>	Comercial	<code>admin_geral</code> , Gerente, Comercial/Consultora, Recepção (cargos com recurso <code>comercial*</code>)
<code>financeiro.</code>	Financeiro	<code>admin_geral</code> , Diretor, Financeiro (cargos com recurso <code>financeiro*</code>)
<code>rh.colaborador</code>	RH	<code>admin_geral</code> , RH, Gerente (cargos com recurso <code>rh.colaborador</code>)
<code>sistema.cargo</code>	Sistema	apenas <code>admin_geral</code> / Super Admin / Administrador (recurso de sistema)

Convenção "funcional" (`ROTAS_FUNCIONAIS` em `menu.ts`): rota listada = tela acesa (query real / empty-state honesto). **Todas as 22 folhas deste escopo estão em `ROTAS_FUNCIONAIS`** — este documento confirma abaixo, caso a caso, se essa marcação é honesta.

Nota de arquitetura — "pontes de rota": 5 folhas do menu apontam para uma rota-ponte que só re-exporta o Server Component do módulo real (para não editar o `menu.ts`):

- `/cadastros/categorias-pagar` → `/catpag`
- `/cadastros/categorias-receber` → `/catrec`
- `/cadastros/parcerias` → `/descontos`
- `/cadastros/planos` → `/planos`
- `/cadastros/perfis` → `/perfis`

1. Acompanhamento

1.1 Dashboard

- **Rota / perm:** `/` · sem `perm` (visível a todo logado). Título "Dashboard".
- **Arquivos:** `src/app/(app)/page.tsx` → `src/components/agenda/DashboardUnidade.tsx`.
- **O que faz:** Painel de abertura da **unidade ativa** — KPIs operacionais/comerciais do período (faturamento, agendamentos, clientes, ticket médio, conversão, metas, ranking). Substitui o clone estático do legado (`view-dashboard`).

- **Telas/abas/modais:** Server Component único `DashboardUnidade`, com filtro de período (`searchParams: per/di/df`).
- **Roteamento por perfil no `page.tsx`:** usuário SAC-only (`papel==='sac'` ou recursos só `sac*`) é `redirect('/sac')`; usuário Financeiro-only (`papel==='financeiro'` ou recursos só `financeiro*`) é `redirect('/financeiro')`.
- **Backend:** consultas reais (sem writes) em `agendamentos`, `clientes`, `metas`, `os`, `servicos`. Escopo pela `activeUnitId`. Sem server actions (só leitura).
- **Integrações:** nenhuma externa direta (agrega dados do lkii). Depende do BEMP para os dados históricos de `agendamentos` já sincronizados.
- **Estado real: FUNCIONAL.** Evidência: `DashboardUnidade` faz 5 queries reais ao lkii; há dado massivo (agendamentos na casa das dezenas/centenas de milhar). Em `ROTAS_FUNCIONAIS`. Marcação honesta.
- **Requisitos p/ 100%:** cobrir funil/no-show consolidados e comparativo mês-a-mês (hoje o foco é KPI de período); confirmar paridade 1:1 com todos os cards do legado.
- **Esforço p/ 100%:** ~1 dia (refino de cards, não é bloqueio).

1.2 Agenda

- **Rota / perm:** `/agenda` · `perm 'operacoes.'` → admin_geral + cargos de Operações/Gerência/Recepção.
- **Arquivos:** `src/app/(app)/agenda/page.tsx`, `src/app/(app)/agenda/actions.ts`, `src/components/agenda/AgendaGrade.tsx`, `src/lib/agenda.ts`.
- **O que faz:** Grade de agendamentos por profissional/horário da unidade — criar/confirmar/ cancelar agendamentos, bloquear horários, cadastro rápido de cliente, e publicar/excluir **eventos da rede** (agenda compartilhada da franqueadora). Inclui sincronização com o BEMP.
- **Telas/abas/modais:** `AgendaGrade` (grade por profissional × horário, com confirmar/ cancelar/bloqueio inline). Fluxos: novo agendamento, bloqueio de agenda, cadastro rápido de cliente, evento de rede.
- **Backend (server actions em `agenda/actions.ts`):**
 - `buscarClientes(termo, unidadeId)` — autocomplete (`clientes`).
 - `criarAgendamento(input)` — **insert** em `agendamentos`.
 - `confirmarAgendamento(id, viaCliente)` — **update** status.
 - `cancelarAgendamento(id, motivo)` — **update** status + motivo.
 - `cadastrarClienteRapido(input)` — **insert** em `clientes`.
 - `criarBloqueio(input)` — **insert** em `bloqueios_agenda`.
 - `publicarEventoRede(input)` / `excluirEventoRede(id)` — **insert/delete** em `rede_eventos`.
 - `sincronizarAgendaBemp()` — **rpc** + leitura de `bemp_agendamentos` → grava novos em `agendamentos` (retorna `{novos, dadosAte}`).
- Tabelas tocadas: `agendamentos`, `bloqueios_agenda`, `clientes`, `rede_eventos`, `bemp_agendamentos`, `perfis_usuario`, `unidades`, `servicos`, `colaboradores`.
- **Integrações: BEMP** (import de agendamentos via `bemp_agendamentos` + rpc de sync); agenda de rede (franqueadora). Sem WhatsApp direto aqui.
- **Estado real: FUNCIONAL.** Evidência: 4 inserts + 2 updates + 1 delete + 1 rpc reais; grade lê `agendamentos` / `colaboradores` / `servicos` / `rede_eventos` reais; sync BEMP implementado. Em `ROTAS_FUNCIONAIS`. Honesto.
- **Requisitos p/ 100%:** confirmação de agendamento *via app/WhatsApp do cliente* (`viaCliente` existe no contrato mas depende da integração de mensageria/app-cliente); validar régua de lembrete automático (no-show) conectada.
- **Esforço p/ 100%:** ~2 dias (lembrete/confirmação automatizada + polimento de grade).

1.3 Ordens de serviço

- **Rota / perm:** `/os` · `perm 'operacoes.os'` → admin_geral + cargos com `operacoes.os`.
- **Arquivos:** `src/app/(app)/os/page.tsx`, `src/app/(app)/os/actions.ts`, `src/components/os/{OsList,OsFiltros,NovaOSModal,OsDetalheModal}.tsx`, `src/lib/os-numero.ts`, `src/lib/pdv.ts`.
- **O que faz:** Abertura, itemização (serviços/produtos/pacotes), pagamento, finalização e cancelamento de Ordens de Serviço da unidade — o núcleo transacional de venda/atendimento.
- **Telas/abas/modais:** lista (`OsList`) + filtros (`OsFiltros`) + **modal Nova OS** (`NovaOSModal`) + **modal de detalhe** (`OsDetalheModal`, carrega itens/pagamentos e gate por permissão). Status de OS: aberta → (itens) → finalizada / cancelada / paga.
- **Backend (server actions em `os/actions.ts`):**
 - `abrirOS(input)` — **insert** em `os` (numeração via `os-numero.ts`).
 - `adicionarItem(input, activeUnitId)` — **insert** em `os_servicos` / `os_produtos` / `os_pacotes`.
 - `removerItem(...)` — **delete** de item.

- `finalizarOS(osId, activeUnitId)` — **update** status.
- `cancelarOS(osId, activeUnitId)` — **update** status.
- `registrarPagamento(input, activeUnitId)` — **insert** em `os_pagamentos`.
- `carregarDetalheOS(osId, activeUnitId)` — leitura consolidada do detalhe.
- Tabelas: `os`, `os_servicos`, `os_produtos`, `os_pacotes`, `os_pagamentos` (+ `clientes`, `servicos`, `perfis_usuario` na listagem).
- **Integrações:** emissão de NFS-e prevista (módulo `/notas`, "EM BREVE") **não** amarrada aqui ainda; pagamentos são registro contábil interno (sem gateway).
- **Estado real: FUNCIONAL** (CRUD transacional real: 2 inserts + 3 updates + 1 delete + inserts de itens/pagamentos). Em `ROTAS_FUNCIONAIS`. Honesto.
- **Requisitos p/ 100%:** amarrar emissão fiscal (NFS-e) na finalização; alçada de desconto por cargo + cortesias (BACKLOG 6.1/1.8); PDV completo (`/pdv` está comentado no menu).
- **Esforço p/ 100%:** ~3 dias (fiscal + alçadas/cortesias + polimento).

2. Cadastros

2.A — Grupo "Cadastros básicos" (13 folhas)

2.1 Anamnese / Ficha Técnica

- **Rota / perm:** `/cadastros/anamnese` · `perm 'comercial.'`.
- **Arquivos:** `cadastros/anamnese/{page.tsx,actions.ts}`, `src/components/anamnese/AnamneseManager.tsx`, `src/lib/anamnese.ts`; seed `scripts/migrations/anamnese.sql`.
- **O que faz:** Cadastro dos **modelos de documento** clínico/ficha (anamnese, termos) que serão preenchidos pelos clientes — com seções, obrigatoriedade e acumulatividade.
- **Telas/abas/modais:** `AnamneseManager` (lista + editor de documento com seções).
- **Backend (`anamnese/actions.ts`):** `criarDocumento` (**insert** documentos), `salvarDocumento(id)` (**update**), `toggleDocumentoStatus(id, ativar)` (**update**). Tabelas: `documentos` (cols: `nome`, `tipo`, `descricao`, `preenchimento`, `obrigatorio`, `status`, `acumulativo`, `secoes(jsonb)`, `empresa_id`), `audit_log`, `empresas`, `perfis_usuario`, `unidades`.
- **Integrações:** grava `audit_log` (LGPD). Ficha ligada ao cliente (aba Anamnese em ClienteFicha).
- **Estado real: FUNCIONAL.** Insert/update reais + seed com documentos-modelo. Honesto.
- **Requisitos p/ 100%:** builder visual de seções mais rico; assinatura/coleta pelo app do cliente.
- **Esforço p/ 100%:** ~1 dia.

2.2 Categorias de Contas a pagar

- **Rota / perm:** `/cadastros/categorias-pagar` → **ponte** para `/catpag` · `perm 'financeiro.'`.
- **Arquivos:** `cadastros/categorias-pagar/page.tsx` (re-export), `catpag/{page.tsx,actions.ts}`, `src/components/catpag/CategoriasManager.tsx`; seed `categorias.sql`.
- **O que faz:** Manutenção do **plano de contas — natureza pagar** (categorias/subcategorias de despesa) usado nos lançamentos financeiros da unidade.
- **Telas/abas/modais:** `CategoriasManager` (árvore de categorias, empty-state "Nenhuma...").
- **Backend (`catpag/actions.ts`):** `criarCategoria` (**insert** plano_contas), `editarCategoria` (**update**), `alternarAtivoCategoria(id, ativo, tipo)` (**update**). Tabela `plano_contas` (cols: `codigo`, `nome`, `tipo`, `natureza`, `parent_id`, `aceita_lancamentos`, `is_sistema`, `ativo`, `empresa_id`). Seed em `categorias.sql` (árvore pagar/receber).
- **Integrações:** consumido por `/contas` (lançamentos) e Financeiro Franqueadora.
- **Estado real: FUNCIONAL.** CRUD real + seed. Honesto.
- **Requisitos p/ 100%:** nenhum estrutural (só dados reais da franquia).
- **Esforço p/ 100%:** 0.

2.3 Categorias de Contas a receber

- **Rota / perm:** `/cadastros/categorias-receber` → **ponte** para `/catrec` · `perm 'financeiro.'`.
- **Arquivos:** `cadastros/categorias-receber/page.tsx` (re-export), `catrec/page.tsx`, mesmo `CategoriasManager`.
- **O que faz:** Idem 2.2 mas para o **plano de contas — natureza receber** (categorias de receita).
- **Telas/abas/modais:** `CategoriasManager` (mesmo componente, filtrado por `tipo=receber`).

- **Backend:** `catrec/actions.ts` está vazio — a tela **reusa as actions do /catpag** (o parâmetro `tipo` da `CategoriasManager` direciona pagar/receber sobre a mesma `plano_contas`). Lê `plano_contas`.
- **Integrações:** idem 2.2.
- **Estado real:** **FUNCIONAL** (compartilha CRUD real do catpag sobre `plano_contas`; empty-state honesto). Honesto — *ressalva:* não tem actions próprias, depende do manager compartilhado.
- **Requisitos p/ 100%:** nenhum estrutural.
- **Esforço p/ 100%:** 0.

2.4 Parcerias

- **Rota / perm:** `/cadastros/parcerias` → **ponte** para `/descontos` · `perm 'comercial.'`.
- **Arquivos:** `cadastros/parcerias/page.tsx` (re-export), `descontos/{page.tsx,actions.ts}`, `src/components/descontos/DescontosManager.tsx`.
- **O que faz:** Cadastro de **parcerias/descontos** (convênios, cupons, políticas de desconto).
- **Telas/abas/modais:** `DescontosManager` (lista + form de desconto/parceria).
- **Backend (`descontos/actions.ts`):** `criarDesconto (insert descontos)`, `editarDesconto (update)`, `alternarAtivoDesconto (update)`. Tabela única `descontos`.
- **Integrações:** aplicável em OS/PDV (alçada de desconto — pendente, BACKLOG 1.8/6.1).
- **Estado real:** **FUNCIONAL** (CRUD real sobre `descontos`). Honesto.
- **Requisitos p/ 100%:** amarrar aplicação do desconto no fluxo de venda com alçada por cargo.
- **Esforço p/ 100%:** ~1 dia (integração com OS/PDV; o cadastro em si está pronto).

2.5 Formas de pagamento

- **Rota / perm:** `/cadastros/formas-pagamento` · `perm 'financeiro.'`.
- **Arquivos:** `cadastros/formas-pagamento/{page.tsx,actions.ts}`; seed `catalogo.sql`.
- **O que faz:** Cadastro das **formas de pagamento** (dinheiro/cartão/pix/recorrência) com taxas, taxa de comissão, mínimo de parcela e base de royalties.
- **Telas/abas/modais:** lista/CRUD inline na própria page.
- **Backend (`formas-pagamento/actions.ts`):** `criarForma (insert)`, `salvarForma (update)`, `toggleFormaAtiva (update)`. Tabela `formas_pagamento` (cols: `nome`, `tipo`, `taxa`, `taxa_comissao`, `rec_min_parcela`, `rec_base_royalties` ('`recorrencia`'|'`venda`'), `ativo`, `ordem`, `empresa_id`). Seed em `catalogo.sql`.
- **Integrações:** taxa de comissão liga à matriz de comissões; base de royalties liga ao Financeiro Franqueadora (automação de royalties).
- **Estado real:** **FUNCIONAL** (CRUD real + seed com colunas de taxa/royalty). Honesto.
- **Requisitos p/ 100%:** nenhum estrutural.
- **Esforço p/ 100%:** 0.

2.6 Grupo de serviços

- **Rota / perm:** `/cadastros/grupo-servicos` · `perm 'comercial.'`.
- **Arquivos:** `cadastros/grupo-servicos/{page.tsx,actions.ts}`, `src/components/servicos/GruposManager.tsx`; seed `catalogo.sql`.
- **O que faz:** Categorização dos serviços em **grupos** (ex.: depilação, estética facial).
- **Telas/abas/modais:** lista de grupos + criar/renomear/ativar.
- **Backend (`grupo-servicos/actions.ts`):** `criarGrupo(nome) (insert)`, `renomearGrupo(id, antigo, novo) (update` — propaga renome aos `servicos`), `toggleGrupoAtivo (update)`. Tabelas `grupo_servicos` (`nome`, `ativo`, `ordem`, `empresa_id`) e `servicos` (para propagação/contagem).
- **Integrações:** consumido por Serviços, Pacotes e relatórios.
- **Estado real:** **FUNCIONAL** (CRUD real + seed). Honesto.
- **Requisitos p/ 100%:** nenhum estrutural.
- **Esforço p/ 100%:** 0.

2.7 Matriz de comissões

- **Rota / perm:** `/cadastros/comissoes` · `perm 'financeiro.'`.
- **Arquivos:** `cadastros/comissoes/{page.tsx,actions.ts}`, `src/components/comissoes/ComissoesBoard.tsx`, `src/lib/comissoes.ts`; seed `comissoes.sql`.
- **O que faz:** Define a **matriz de percentuais de comissão** por categoria/base (individual/ equipe) aplicada aos colaboradores.
- **Telas/abas/modais:** `ComissoesBoard` (grid editável de categorias × percentuais; empty-state).

- **Backend (`comissoes/actions.ts`):** `salvarMatriz(cats)` — **delete-all + insert** (regrava a matriz inteira). Tabela `matriz_comissoes` (`base_individual_pct` numeric(6,2) , categorias, `empresa_id`) + `colaborador_servicos` (vínculo). Seed em `comissoes.sql`. Page lê `colaboradores` + `matriz_comissoes`.
- **Integrações:** alimenta cálculo de comissão (folha/pagamentos) e ranking.
- **Estado real: FUNCIONAL** (persiste matriz real via delete+insert; seed presente). Honesto. *Ressalva:* FRONTEND-STATUS previa "simulador" (EPIC 4.1) — a matriz grava, mas o simulador de cenários é o refino pendente.
- **Requisitos p/ 100%:** simulador de comissão + aplicação automática no fechamento de OS/folha.
- **Esforço p/ 100%:** ~1,5 dia.

2.8 Metas

- **Rota / perm:** `/cadastros/metasp/` · perm `'comercial.'`.
- **Arquivos:** `cadastros/metasp/{page.tsx,actions.ts}`, `src/components/metasp/{MetasColaboradorCrud,MetasUnidadeSimulador}.tsx`.
- **O que faz:** Define **metas por colaborador** e simula **metas da unidade**, comparando realizado (a partir de agendamentos) vs meta.
- **Telas/abas/modais:** `MetasColaboradorCrud` (CRUD de meta por pessoa) + `MetasUnidadeSimulador` (simulador da unidade sobre realizado).
- **Backend (`metas/actions.ts`):** `criarMeta` (**insert**), `salvarMeta` (**update**), `atualizarRealizado(id, valor)` (**update**), `excluirMeta` (**delete**). Tabela `metas_colaborador`. Page lê `metas_colaborador` + `colaboradores` + `agendamentos` (realizado).
- **Integrações:** consome `agendamentos` (realizado); aparece no Dashboard e relatório de metas.
- **Estado real: FUNCIONAL** (CRUD completo real + simulador sobre dados reais). Honesto.
- **Requisitos p/ 100%:** automação do "realizado" (hoje pode ser atualizado manual/derivado); metas por unidade persistidas (o componente da unidade é simulador).
- **Esforço p/ 100%:** ~1 dia.

2.9 Modelos de contrato

- **Rota / perm:** `/cadastros/contratos` · perm `'comercial.'`.
- **Arquivos:** `cadastros/contratos/{page.tsx,actions.ts}`, `src/components/contratos/ContratosManager.tsx`, `src/lib/contratos.ts`; seed `categorias.sql` (bucket storage + `contratos_modelo`).
- **O que faz:** Biblioteca de **modelos de contrato** (arquivo + regra de "quando emitido") usados na venda de planos/pacotes.
- **Telas/abas/modais:** `ContratosManager` (lista + upload/gerência de modelo).
- **Backend (`contratos/actions.ts`):** `criarContrato` (**insert**), `salvarContrato` (**update**), `alternarAtivoContrato` (**update**), `urlArquivoContrato(id)` (gera URL assinada do **Storage**). Tabela `contratos_modelo` (`nome`, `quando_emitido`, `titulo`, `ordem`, `empresa_id`) + bucket de storage.
- **Integrações:** **Supabase Storage** (arquivos de contrato).
- **Estado real: FUNCIONAL** (CRUD real + storage). Honesto.
- **Requisitos p/ 100%:** geração/assinatura do contrato preenchido na venda (merge de variáveis + e-sign).
- **Esforço p/ 100%:** ~2 dias (se exigir preenchimento/assinatura automatizados).

2.10 Motivos de cancelamento

- **Rota / perm:** `/cadastros/motivos` · perm `'comercial.'`.
- **Arquivos:** `cadastros/motivos/{page.tsx,actions.ts}`, `src/components/motivos/MotivosManager.tsx`; seed `anamnese.sql`.
- **O que faz:** Cadastro dos **motivos de cancelamento/no-show** + configuração da **automação de no-show** (régua).
- **Telas/abas/modais:** `MotivosManager` (lista de motivos + painel de config de no-show).
- **Backend (`motivos/actions.ts`):** `criarMotivo` (**insert**), `salvarMotivo` (**update**), `toggleMotivoAtivo` (**update**), `excluirMotivo` (**delete**), `salvarNoshowConfig(cfg)` (**upsert** em `noshow_automacao`). Tabelas `motivos_cancelamento` (`nome`, `sistema`, `ativo`, `ordem`), `noshow_automacao`, `audit_log`.
- **Integrações:** `noshow_automacao` prepara régua automática (WhatsApp/lembrete — depende do canal conectado).
- **Estado real: FUNCIONAL** (CRUD + upsert de config real, com `audit_log`). Honesto.
- **Requisitos p/ 100%:** executor da régua de no-show (cron/pg_net + envio WhatsApp) ligado à config.
- **Esforço p/ 100%:** ~1,5 dia (a config grava; falta o disparo automático).

2.11 Planos de Assinatura

- **Rota / perm:** `/cadastros/planos` → **ponte** para `/planos` · perm `'comercial.'`.

- **Arquivos:** `cadastros/planos/page.tsx` (re-export), `planos/{page.tsx,actions.ts}`, `src/components/planos/PlanosManager.tsx`.
- **O que faz:** Cadastro dos **planos de assinatura** (recorrência) e os **serviços inclusos** em cada plano.
- **Telas/abas/modais:** `PlanosManager` (lista + editor com seleção de serviços do plano).
- **Backend (`planos/actions.ts`):** `criarPlano` (`insert planos_assinatura` + `insert` itens em `plano_assinatura_servicos`), `editarPlano` (`update` + `delete/insert` de itens), `togglePlanoAtivo` (`update`). Tabelas `planos_assinatura`, `plano_assinatura_servicos` (junção plano↔serviço), `servicos`.
- **Integrações:** liga a assinaturas/crédito recorrente (relatórios) e cobrança/royalties.
- **Estado real:** **FUNCIONAL** (CRUD real com junção de serviços). Honesto.
- **Requisitos p/ 100%:** cobrança recorrente automática do plano (gateway/pix recorrente) — decisão do cliente.
- **Esforço p/ 100%:** ~2 dias (se incluir motor de cobrança recorrente).

2.12 Perfis de acesso

- **Rota / perm:** `/cadastros/perfis` → **ponte** para `/perfis` · `perm 'sistema.cargo'` (apenas `admin_geral` / **Super Admin** / **Administrador**).
- **Arquivos:** `cadastros/perfis/page.tsx` (re-export), `perfis/{page.tsx,actions.ts}`, `perfis/matriz/page.tsx`, `perfis/[cargoId]/page.tsx`, `src/components/perfis/PerfisLista.tsx`; seed `perfis-acesso.sql` + `rbac.sql`.
- **O que faz:** **Editor de RBAC real** — cria/edita cargos, edita a matriz de permissões (`cargo_permissoes`), aplica presets, atribui cargo a usuário, marca "bate ponto".
- **Telas/abas/modais:** `PerfisLista` (rota `/perfis`), **matriz de permissões** (`/perfis/matriz`), **editor de cargo** (`/perfis/[cargoId]`).
- **Backend (`perfis/actions.ts`):** `salvarPermissoesCargo` (`upsert/delete cargo_permissoes`), `aplicarPreset`, `criarCargo` (`insert cargos`), `atualizarCargo` (`update`), `alternarAtivoCargo`, `alternarBatePonto`, `excluirCargo` (`delete`), `atribuirCargoUsuario` (`insert usuario_cargos`), `removerCargoUsuario` (`delete`). Tabelas `cargos`, `cargo_permissoes`, `permissoes`, `usuario_cargos`, `perfis_usuario`, `empresas`, `audit_log`.
- **Integrações:** é a **fonte do gate de menu/botões** (lido por `getSessionContext`); grava `audit_log`. 17 perfis seedados (`perfis-acesso.sql`).
- **Estado real:** **FUNCIONAL**. Evidência: grava `cargo_permissoes` / `usuario_cargos` de verdade (6 delete + 4 insert + 4 update + 2 upsert) — resolve o gap histórico do protótipo ("só dava toast"). Honesto.
- **Requisitos p/ 100%:** validar escopo (global/empresa/unidade/próprio) por permissão e testes de isolamento entre franquias (BACKLOG 1.2/1.3).
- **Esforço p/ 100%:** ~1 dia (testes/escopo; editor pronto).

2.13 Origens de Cliente

- **Rota / perm:** `/cadastros/origens` · `perm 'comercial.'`.
- **Arquivos:** `cadastros/origens/{page.tsx,actions.ts}`, `src/components/origens/OrigensManager.tsx`; seed `anamnese.sql`.
- **O que faz:** Cadastro das **origens de captação** do cliente (Instagram, indicação, site...), com flag `auto` (origem automática) e `campo`.
- **Telas/abas/modais:** `OrigensManager` (lista + CRUD).
- **Backend (`origens/actions.ts`):** `criarOrigem` (`insert`), `salvarOrigem` (`update`), `toggleOrigemAtiva` (`update`), `excluirOrigem` (`delete`). Tabela `origens_cliente` (`nome`, `ativo`, `auto`, `campo`, `ordem`, `empresa_id`), `audit_log`.
- **Integrações:** usada no cadastro de cliente e nos leads do site (origem automática).
- **Estado real:** **FUNCIONAL** (CRUD real + audit + seed). Honesto.
- **Requisitos p/ 100%:** nenhum estrutural.
- **Esforço p/ 100%:** 0.

2.B — Cadastros de topo (6 folhas)

2.14 Clientes

- **Rota / perm:** `/clientes` · `perm 'comercial.'`.
- **Arquivos:** `clientes/{page.tsx,actions.ts}`, `clientes/[id]/page.tsx`, `clientes/export/`, `src/components/clientes/{ClientesList,ClientesFiltros,NovoClienteModal,ImportarClientesModal,ClienteFicha}.tsx`, `src/lib/clientes.ts`.
- **O que faz:** Base de clientes da unidade — lista com filtros, cadastro/edição, **checagem de duplicados + unificação**, **importação em massa (Excel/CSV)**, ficha 360° do cliente e exportação.

- **Telas/abas/modais:** lista (`CientesList`) + filtros; **modal Novo Cliente** (`NovoClienteModal`) com checagem de duplicado; **modal Importar** (`ImportarClientesModal`); **ficha do cliente** `/clientes/[id]` (`ClienteFicha`) com abas **Dados/Carteira**, **Agendamentos**, **Anamnese**, **Contratos**, **Acompanhamento**; **export** `/clientes/export`.
- **Backend** (`clientes/actions.ts`): `checarDuplicado`, `criarCliente(input, forcar)` (insert), `salvarCliente(id)` (update), `inativarCliente/reativarCliente` (update), `importarClientes(...)` (insert em lote), `listarDuplicados(id)`, `unificarClientes(manterId, removerIds)` (update/merge). Tabelas `clientes`, `agendamentos`, `unidades`.
- **Integrações:** **leads do site** entram como clientes/origem; importação Excel; base compartilhada com Agenda/OS/CRM.
- **Estado real:** **FUNCIONAL** com **dado real massivo** (base de clientes importada do BEMP). Evidência: 2 inserts + 6 updates, dedupe/merge e importação implementados; ficha lê dados reais. Honesto.
- **Requisitos p/ 100%:** ficha 360° completa (crédito/financeiro do cliente); régua de reengajamento por IA (P2); confirmar mapeamento de todas as colunas do Excel do cliente.
- **Esforço p/ 100%:** ~2 dias.

2.15 Colaboradores

- **Rota / perm:** `/colaboradores` · perm `'rh.colaborador'`.
- **Arquivos:** `colaboradores/{page.tsx,actions.ts}`, `colaboradores/[id]/page.tsx`, `src/components/colaboradores/{ColaboradoresList,ColaboradoresFiltros,NovoColaboradorModal,ColaboradorFicha}.tsx`, `src/lib/pessoas.ts`.
- **O que faz:** Cadastro dos **colaboradores** da unidade (recepção/técnica/gestão), com checagem de CPF duplicado e **vínculo de serviços que o colaborador executa**. Modelo de pessoas: `colaboradores ↔ perfis_usuario` (via `perfil_id`) — colaborador pode virar usuário/atendente.
- **Telas/abas/modais:** lista (`ColaboradoresList`) + filtros; **modal Novo Colaborador**; **ficha** `/colaboradores/[id]` (`ColaboradorFicha`) com abas **Dados**, **Serviços**, **Comissão**.
- **Backend** (`colaboradores/actions.ts`): `checarCpfDuplicado`, `criarColaborador(input, forcar)` (insert), `salvarColaborador(id)` (update), `inativar/reativar` (update), `carregarServicosColaborador`, `salvarServicosColaborador(id, servicoIds)` (delete+insert em `colaborador_servicos`). Tabelas `colaboradores`, `colaborador_servicos`, `servicos`.
- **Integrações:** liga a RH (`/rh/colaboradores`), Ponto, Comissões, Agenda (profissional), e ao RBAC (quando vira usuário). Fonte única `src/lib/pessoas.ts`.
- **Estado real:** **FUNCIONAL** (CRUD real + vínculo de serviços via junção). Honesto.
- **Requisitos p/ 100%:** filtros avançados (EPIC 20.4); amarração ponto↔folha↔comissão; foto/documentos.
- **Esforço p/ 100%:** ~1,5 dia.

2.16 Contas da Unidade (pagar/receber)

- **Rota / perm:** `/contas` · perm `'financeiro.'`. Título "Contas a pagar/receber · Unidade".
- **Arquivos:** `contas/{page.tsx,actions.ts}`, `src/components/contas/ContasManager.tsx`, `src/lib/financeiro.ts`; seed `financeiro.sql`.
- **O que faz:** Financeiro **da unidade** (distinto do Financeiro Franqueadora) — lançamentos a pagar/receber, com baixa/pagamento, classificados no plano de contas.
- **Telas/abas/modais:** `ContasManager` (abas pagar/receber, lista + novo lançamento + baixa).
- **Backend** (`contas/actions.ts`): `novoLancamento(input)` (insert `lancamentos_financeiros`), `registrarPagamento(lancamentoId)` (update → status pago), `editarLancamento(input)` (update). Tabelas `lancamentos_financeiros`, `plano_contas`, `unidades`.
- **Integrações:** usa `plano_contas` (categorias 2.2/2.3); consolida no Financeiro Franqueadora (DRE/fluxo de caixa) por escopo de unidade.
- **Estado real:** **FUNCIONAL** (insert + updates reais sobre `lancamentos_financeiros`). Honesto.
- **Requisitos p/ 100%:** recorrência de lançamento, anexo de comprovante (storage), conciliação com extrato bancário da unidade.
- **Esforço p/ 100%:** ~2 dias.

2.17 Pacotes

- **Rota / perm:** `/pacotes` · perm `'comercial.'`.
- **Arquivos:** `pacotes/{page.tsx,actions.ts}`, `src/components/pacotes/PacotesManager.tsx`, `src/lib/catalogo.ts`; seed `catalogo.sql`.
- **O que faz:** Cadastro de **pacotes** (combos de serviços com preço fechado / nº de sessões).
- **Telas/abas/modais:** `PacotesManager` (lista + editor com **seleção de serviços/itens do pacote**).
- **Backend** (`pacotes/actions.ts`): `criarPacote` (insert `pacotes` + insert itens `pacote_itens`), `editarPacote` (update + delete/insert itens), `togglePacoteAtivo` (update). Tabelas `pacotes`, `pacote_itens` (junção pacote↔serviço), `servicos`.
- **Integrações:** vendável em OS (`os_pacotes`); aparece em relatório de pacotes.

- **Estado real: FUNCIONAL** (CRUD real com junção de itens; seed presente). Honesto.
- **Requisitos p/ 100%:** controle de saldo de sessões por cliente (consumo do pacote na OS).
- **Esforço p/ 100%:** ~1,5 dia (saldo/consumo; cadastro pronto).

2.18 Produtos

- **Rota / perm:** `/produtos` · `perm 'comercial.'`.
- **Arquivos:** `produtos/{page.tsx,actions.ts}`, `src/components/produtos/{ProdutosList,ProdutosFiltros,ProdutoModal}.tsx`.
- **O que faz:** Cadastro de **produtos** revendidos (cosméticos etc.) — preço e status.
- **Telas/abas/modais:** lista (`ProdutosList`) + filtros + **modal Produto** (`ProdutoModal` , com gate por permissão). Empty-state honesto ("Nenhum produto cadastrado ainda").
- **Backend (`produtos/actions.ts`):** `criarProduto` (insert), `salvarProduto` (update), `toggleProdutoAtivo` (update). Tabela única `produtos`.
- **Integrações:** vendável em OS (`os_produtos`); relatórios.
- **Estado real: FUNCIONAL** (CRUD real; empty-state honesto). Honesto.
- **Requisitos p/ 100%:** **controle de estoque** (não há tabela/coluna de estoque nas actions); custo/margem; código de barras — decisão do cliente se entra no escopo.
- **Esforço p/ 100%:** ~2 dias (se incluir estoque); 0 para o cadastro puro.

2.19 Serviços

- **Rota / perm:** `/servicos` · `perm 'comercial.'`.
- **Arquivos:** `servicos/{page.tsx,actions.ts}`, `src/components/servicos/{ServicosList,ServicosFiltros,ServicoModal,GruposManager}.tsx` ; seed `catalogo.sql`.
- **O que faz:** Cadastro dos **serviços** prestados (preço, duração, grupo) — catálogo central consumido por Agenda, OS, Pacotes, Planos, Comissões.
- **Telas/abas/modais:** lista (`ServicosList`) + filtros + **modal Serviço** (`ServicoModal`) + gerência de grupos embutida (`GruposManager`).
- **Backend (`servicos/actions.ts`):** `criarServico` (insert), `salvarServico` (update), `toggleServicoAtivo` (update), `renomearGrupo(de, para)` (update em massa). Tabelas `servicos` , `grupo_servicos`.
- **Integrações:** é o **catálogo-mãe** referenciado por `os_servicos` , `pacote_itens` , `plano_assinatura_servicos` , `colaborador_servicos` , `matriz_comissoes`.
- **Estado real: FUNCIONAL** (CRUD real + seed). Honesto.
- **Requisitos p/ 100%:** nenhum estrutural.
- **Esforço p/ 100%:** 0.

3. Tabela-resumo

#	FUNCIONALIDADE (FOLHA DO MENU)	ROTA (→ PONTE)	ESTADO	DIAS P/ 100%
1.1	Dashboard	<code>/</code>	FUNCIONAL	1
1.2	Agenda	<code>/agenda</code>	FUNCIONAL	2
1.3	Ordens de serviço	<code>/os</code>	FUNCIONAL	3
2.1	Anamnese / Ficha Técnica	<code>/cadastros/anamnese</code>	FUNCIONAL	1
2.2	Categorias Contas a pagar	<code>/cadastros/categorias-pagar</code> → <code>/catpag</code>	FUNCIONAL	0
2.3	Categorias Contas a receber	<code>/cadastros/categorias-receber</code> → <code>/catrec</code>	FUNCIONAL	0
2.4	Parcerias	<code>/cadastros/parcerias</code> → <code>/descontos</code>	FUNCIONAL	1
2.5	Formas de pagamento	<code>/cadastros/formas-pagamento</code>	FUNCIONAL	0
2.6	Grupo de serviços	<code>/cadastros/grupo-servicos</code>	FUNCIONAL	0
2.7	Matriz de comissões	<code>/cadastros/comissoes</code>	FUNCIONAL (simulador pendente)	1,5
2.8	Metas	<code>/cadastros/metas</code>	FUNCIONAL	1
2.9	Modelos de contrato	<code>/cadastros/contratos</code>	FUNCIONAL	2

#	FUNCIONALIDADE (FOLHA DO MENU)	ROTA (→ PONTE)	ESTADO	DIAS P/ 100%
2.10	Motivos de cancelamento	<code>/cadastros/motivos</code>	FUNCIONAL (disparo no-show pendente)	1,5
2.11	Planos de Assinatura	<code>/cadastros/planos</code> → <code>/planos</code>	FUNCIONAL	2
2.12	Perfis de acesso	<code>/cadastros/perfis</code> → <code>/perfis</code>	FUNCIONAL	1
2.13	Origens de Cliente	<code>/cadastros/origens</code>	FUNCIONAL	0
2.14	Clientes	<code>/clientes</code>	FUNCIONAL (dado real)	2
2.15	Colaboradores	<code>/colaboradores</code>	FUNCIONAL	1,5
2.16	Contas da Unidade	<code>/contas</code>	FUNCIONAL	2
2.17	Pacotes	<code>/pacotes</code>	FUNCIONAL	1,5
2.18	Produtos	<code>/produtos</code>	FUNCIONAL (sem estoque)	2
2.19	Serviços	<code>/servicos</code>	FUNCIONAL	0

Totais do escopo: - 22 folhas de menu documentadas (3 Acompanhamento + 13 Cadastros básicos + 6 Cadastros de topo). - **31 arquivos `page.tsx`** materializam essas folhas: 22 telas principais + 5 pontes de rota (categorias-pagar, categorias-receber, parcerias, planos, perfis) + sub-telas (`clientes/[id]` , `colaboradores/[id]` , `perfis/matriz` , `perfis/[cargoId]`) — além da rota de export `clientes/export` . - **Estado geral: 22/22 FUNCIONAIS** (todas em `ROTAS_FUNCIONAIS` , marcação confirmada como honesta por evidência de insert/update/delete real e/ou empty-state honesto). Nenhuma tela em FALTA; 6 telas com refino PARCIAL sinalizado (comissões-simulador, no-show-disparo, produtos-estoque, contratos-esign, planos-cobrança, clientes/contas-360º). - **Esforço somado p/ 100% (refinos): ~28,5 dias-dev** (a maior parte é integração/decisão do cliente, não reconstrução — o núcleo CRUD já grava de verdade no Ikii).

Ressalvas de fidelidade: - `/catrec` não tem `actions.ts` próprio — reusa as actions de `/catpag` sobre a mesma tabela `plano_contas` (parâmetro `tipo`). Funcional, mas é dependência a registrar. - Produtos **não têm coluna/tabela de estoque** nas actions atuais — se estoque for requisito de homologação, é desenvolvimento novo (~2 dias), não ajuste. - "Realizado" de Metas e "consumo" de Pacotes ainda não têm automação de baixa a partir da OS.

BÍBLIA DE HOMOLOGAÇÃO — Gestão › Relatórios & Dashboards

Sistema: Laser&Co Power System (Next.js 15 App Router + Supabase `lkii`) **Escopo deste documento:** os 2 grupos grandes da seção **Gestão** do `src/lib/menu.ts`: grupo **"Relatórios"** (`key: 'rel'`, 24 folhas de menu) e grupo **"Dashboards"** (`key: 'dash'`, 7 folhas). Rotas em `src/app/(app)/relatorios/*` e `src/app/(app)/dashboards/*`. **Contagem de telas verificada no filesystem:** 25 `page.tsx` em `/relatorios` (24 no menu + `notas-fiscais` órfã do menu) e 7 `page.tsx` em `/dashboards` (as 4 `vendas-*` são wrappers de 1 componente compartilhado). **Total: 32 rotas de página.**

Fundação compartilhada (ler antes de cada bloco)

Todas as páginas são **Server Components** (`export const dynamic = 'force-dynamic'`), read-only, resolvem escopo multitenant por `getSessionContext().activeUnitId` (unidade fixada na topbar vence; senão `?unidade=`), e caem em empty-states honestos em erro.

- `src/lib/relatorios.ts` — backbone de leitura de OS/pagamentos. `pullOS()` e `pullPagamentos()` **paginam com** `.range(from, from+999)` em lotes `PAGE=1000` até teto `PULL_CAP=20000` (padrão correto que evita o corte silencioso de 1000 linhas do PostgREST — documentado em comentário). `nomesPerfis()` / `nomesClientes()` resolvem nomes via `.in(ids.slice(0,1000))`. Tabelas reais: `os`, `os_pagamentos`, `clientes`, `perfis_usuario`.
- `src/components/dashboards/agg.ts` — "regra de ouro": nunca puxar linhas cruas em massa (`agendamentos` ≈136k, `clientes` ≈347k). `contar()` usa `count:'exact'`, `head:true` (zero linhas transferidas); `pullLancamentos()` pagina `lancamentos_financeiros` (≈12.9k) só com colunas baratas até `SUM_CAP=20000`; lança `DashAggError` para não exibir zeros silenciosos.
- `src/lib/dashboards.ts` — regras de negócio do legado: royalties (10% do faturamento realizado do mês anterior, venc. dia 10), própria×franqueada por prefixo de CNPJ `44.442.908`, `pctInt()`. **Os ratios/tickets HARDCODED do legado ("Dashboard de Revenda" ilustrativo) foram REMOVIDOS** — o funil usa dado real.
- `resolveRelRange()` / `resolveDashRange()` — mapeiam `?periodo=` (mes/mes_passado/90d/ano/custom/tudo) para janela `{ini, fim}` meia-aberta + período anterior p/ comparativo.
- **RBAC** (`perm` no menu, sufixo `.` = prefixo): relatórios `comercial.` são vistos por qualquer perfil com permissão `comercial.*`; `financeiro.` por perfis `financeiro.*`; dashboards `vendas-*` são `admin:true` e ainda gated por `ehAdmin(ctx.papel)` dentro do `VendasReal` (dupla trava franqueadora).
- **Alerta de coerência do menu:** `ROTAS_FUNCIONAIS` marca **todas** as 24 rotas de relatório + 7 dashboards como "funcional" — mas o critério (nota de 2026-06-29) é *"query real OU empty-state honesto"*, **não** *"tem dado"*. `avaliacoes` e `ocorrencias` acendem no menu porém não têm tabela-fonte (ver blocos). Menu-funcional ≠ com-dados.

PARTE 1 — RELATÓRIOS (grupo `rel`)

1. Assinaturas — `/relatorios/assinaturas` · `perm comercial.`

1. **O que faz:** Relatório do CATÁLOGO de planos de assinatura e do MRR *potencial* — não de assinaturas reais por cliente (não existe tabela de vínculo).
2. **Telas:** Filtro Ativos(default)/Todos (`?ativo=todos`). KPIs: *Planos ativos*, *MRR potencial*, *Ticket médio*, *Adesão média*. Charts: "Top planos por mensalidade" (top 10), "Planos por faixa de mensalidade" (≤100/100-200/200-400/>400). Tabela: Plano/Status/Mensalidade/Adesão/Duração/Serviços-mês/Criado em + tfoot MRR.
3. **Backend:**
`planos_assinatura` `.select('id,nome,descricao,valor_mensal,valor_adesao,duracao_meses,beneficios,ativo,criado_em').orderBy(△ .limit(1000) — teto rígido, catálogo); .eq('ativo',true) no modo ativos.
plano_assinatura_servicos .select('plano_id,quantidade_mensal').in('plano_id',ids). Erro → banner semFonte.`
4. **Integrações:** nenhuma.
5. **Estado real: PARCIAL.** Lê tabelas reais de catálogo (fonte `/planos`). Documentado em código: **não existe** tabela de assinatura por cliente (`cliente_assinaturas` / `assinaturas` / `clientes_planos`), logo assinaturas ativas/churn/MRR realizado NÃO são calculáveis; MRR é "potencial". Sem mock/iframe.
6. **P/ 100%:** criar tabela cliente↔plano (status, datas início/cancelamento, valor); ligar fluxos assinar/cancelar; KPIs de ativas/novas/canceladas/churn/MRR realizado.
7. **Esforço: 4 dias.**

2. Ocorrências e Intercorrências — `/relatorios/ocorrencias` · perm `comercial`.

1. **O que faz:** Deveria reportar ocorrências/intercorrências clínico-operacionais por atendimento; hoje é **placeholder de empty-state** porque não há tabela-fonte.
2. **Telas:** KPIs com `value: ''` (Total de registros, Ocorrências, Intercorrências); legenda conceitual Ocorrência×Intercorrência; tabela (Data/Cliente/Profissional/Serviço/Descrição/Tipo) com única linha "Nenhum registro". Sem filtro/charts.
3. **Backend: NENHUM** — a página não chama `createClient()` nem `from()`. Comentário documenta: `ocorrencias/ocorrencias_frequencia` não existem; `acoes` =RBAC, `atestados` =RH. Não consulta nada para evitar erro 42P01.
4. **Integrações:** nenhuma.
5. **Estado real: FALTA (placeholder honesto).** No legado era 100% mock (array `OCOR`). Sem mock/query quebrada/iframe. TODO em código especifica o que construir.
6. **P/ 100%:** criar tabela de ocorrências (cliente/profissional/serviço/descrição/tipo/data/unidade) + UI de captura + relatório (KPIs/charts/tabela por período+unidade).
7. **Esforço: 4 dias.**

3. Agendamentos — `/relatorios/agendamentos` · perm `comercial`.

1. **O que faz:** Relatório só-contagem de agendamentos por status e (em janelas curtas) por dia, escopado à unidade ativa.
2. **Telas:** `RelFiltros` (default `mes`). KPIs: *Total, Concluídos (+% conclusão), Cancelados (+% cancelamento, tom down se >25%), Confirmados*. Charts: "Por status"; "Por dia" (só se janela fechada ≤31 dias). Tabela "Resumo por status" (concluido/confirmado/aberto/em_atendimento/cancelado/no_show).
3. **Backend:** `agendamentos` — todas as queries `.select('id',{count:'exact',head:true})` (zero linhas). Filtros `.eq('status')`, `.eq('unidade_id')`, `.gte/.lt('inicio')`. Total+6 status em `Promise.all`; por-dia = até 31 head-counts. Sem `.range/.limit/embed`. Seguro contra a tabela de 136k.
4. **Integrações:** nenhuma.
5. **Estado real: FUNCIONAL (dimensões parciais).** Tabela e status reais. TODO honesto: sem quebra por profissional (`profissional_id` sempre null no import BEMP; sem tabela `profissionais`).
6. **P/ 100%:** popular `profissional_id` + tabela `profissionais`; quebra por serviço.
7. **Esforço: 1.5 dias.**

4. Anamnese / Ficha Técnica — `/relatorios/anamnese` · perm `comercial`.

1. **O que faz:** Reporta o CATÁLOGO de fichas digitais (`documentos`), não preenchimentos por cliente (não há tabela de respostas).
2. **Telas:** Filtro "Tipo" (`?tipo=`) + `ExportCsvButton`. KPIs: *Documentos, Ativos (+%), Obrigatórios, Perguntas (total)*. Metric-boxes (perguntas que inviabilizam, obrigatórios, média perguntas/doc). Charts "Por tipo"/"Por status". Tabela "Fichas digitais".
3. **Backend:** `documentos` `.select('id,nome,tipo,descricao,preenchimento,obrigatorio,status,acumulativo,unidades_ids,secoes,atualizado')` [`△ .limit(1000)`]. Escopo por `unidades_ids` (array, em memória). Perguntas computadas de `secoes` (JSONB). Erro → banner "migration não aplicada".
4. **Integrações:** nenhuma.
5. **Estado real: PARCIAL (só catálogo).** KPIs mock do legado ("488 preenchidos" etc.) removidos. Sem tabela de preenchimento/assinatura → per-cliente não calculável. Depende de migration `anamnese.sql`.
6. **P/ 100%:** esquema de respostas/assinaturas (`documento_respostas / assinaturas_documento` com `cliente_id/documento_id/status/assinado_em`) + persistência do submit + KPIs per-cliente.
7. **Esforço: 4 dias.**

5. atendimentos — `/relatorios/atendimentos` · perm `comercial`.

1. **O que faz:** Analisa agendamentos `status='concluido'` como "atendimentos" — volume por mês/unidade + duração real (fim–inicio).
2. **Telas:** `RelFiltros` (default `90d`). KPIs: *Concluídos (+ se capped), Unidades atendendo, Duração média (min), Tempo total (h)*. Charts "por mês"/"por unidade top10". Tabela por unidade. Banners capped/erro.
3. **Backend:** `agendamentos` **paginado** `.range()` (PAGE 1000) até `SUM_CAP=20000` (flag `capped`). Select `id,inicio,fim,unidade_id,unidade:unidades(nome)` — **embed** `unidades` `.eq('status','concluido')`, `.eq('unidade_id')`, `.gte/.lt('inicio')`. Agregação em JS.
4. **Integrações:** nenhuma.
5. **Estado real: FUNCIONAL (dimensões parciais).** Keyed por `status` porque `concluido_em` veio vazio do import. `servico_id/profissional_id/cliente_id` null → sem breakdown por serviço/profissional. Cap 20k pode subcontar janelas amplas (sinalizado com `+`).
6. **P/ 100%:** ingestão de `servico_id/profissional_id` daqui p/ frente; agregação server-side (RPC) p/ remover cap.

7. **Esforço: 2 dias.**

6. Avaliações — `/relatorios/avaliacoes` · perm `comercial`.

1. **O que faz:** Scaffold NPS/CSAT que consulta defensivamente tabela `avaliacoes` inexistente; hoje renderiza empty-state honesto.
2. **Telas:** `RelFiltros` (default `mes`). KPIs: *Avaliações*, *Nota média*, *Promotores %*, *Detratores %*. Metric-boxes NPS/Neutros/Escala. Charts distribuição/classificação. Tabela Data/Cliente/Serviço/Profissional/Nota/Comentário.
3. **Backend:** `avaliacoes` (try/catch)
`.select('id,nota,comentario,cliente_nome,servico_nome,profissional_nome,criado_em,unidade_id').order('criado_em').limit(500)` [seguro]. Tabela não existe → `semFonte=true`. Auto-detecta escala 1-5/0-10.
4. **Integrações:** nenhuma.
5. **Estado real:** **FALTA (empty-state honesto)**. KPIs mock do legado removidos. Sem tabela `avaliacoes`; `avaliacoes_desempenho` (RH) deliberadamente não usada. Acende sozinho quando a tabela existir.
6. **P/ 100%:** criar tabela `avaliacoes` + coleta pós-atendimento (survey/WhatsApp).
7. **Esforço: 4 dias** (código do relatório pronto).

7. Clientes — `/relatorios/clientes` · perm `comercial`.

1. **O que faz:** Relatório da base: totais, ativos/inativos/verificados, novos por mês.
2. **Telas:** `RelFiltros` (default `mes`). KPIs: *Base total*, *Ativos (+%)*, *Novos (período)*, *Verificados*. Charts "Novos (6 meses)"/"Composição". Tabela "Resumo da base".
3. **Backend:** `clientes` `.select('id',{count:'estimated',head:true}) [count:'estimated' — estatística do Postgres, escolhido porque exact sobre ~347k×10 levava 13s]`. Filtros `.eq('ativo',true)`, `.eq('verificado',true)`, `.gte/.lt('criado_em')`. 4 counts globais + 6 mensais em paralelo.
4. **Integrações:** nenhuma.
5. **Estado real:** **FUNCIONAL (2 ressalvas honestas)**. KPIs APROXIMADOS (estimated). `unidade_origem_id` sempre null → **sem segmentação por unidade** (nota + TODO).
6. **P/ 100%:** popular `unidade_origem_id`; breakdowns cidade/estado/canal_origem + cohort; opcional exact counts.
7. **Esforço: 2 dias.**

8. Contratos — `/relatorios/contratos` · perm `comercial`.

1. **O que faz:** Ativos/assinados/inadimplentes + MRR contratado, da tabela `contratos`.
2. **Telas:** `RelFiltros` (default `90d`) + ExportCsv. KPIs: *Ativos*, *Assinados*, *Inadimplentes (tom down se >0)*, *Valor contratado (MRR)*. Charts "ativos por plano"/"por status". Tabela Cliente/Plano/Status/Criação/Assinatura/Valor-mês. Banner migration se ausente/zero.
3. **Backend:** `contratos` — (a) lista
`.select('id,cliente_nome,plano,status,valor_mensal,criado_em,assinado_em').order('criado_em').limit(1000)` [△ `.limit(1000)`], só display; header mostra "exibindo N"; (b) KPIs por `.range()` paginado (PAGE 1000, até offset 50000) `select status,plano,assinado_em,valor_mensal` — contagem EXATA. `cliente_nome/plano` são strings denormalizadas (sem FK).
4. **Integrações:** nenhuma.
5. **Estado real:** **PARCIAL/condicional**. Código funcional lendo tabela real, MAS `contratos` é seed de `scripts/migrations/relatorios.sql` (contratos-exemplo derivados de clientes reais), não fluxo produtivo. Migration ausente → banner honesto.
6. **P/ 100%:** aplicar migration; substituir seed por ciclo real (criação/assinatura/inadimplência); normalizar `cliente_id/plano_id`.
7. **Esforço: 3 dias.**

9. Crédito em dinheiro — `/relatorios/credito-dinheiro` · perm `comercial`.

1. **O que faz:** Apura recebimentos com `os_pagamentos.metodo='dinheiro'` por período, totalizando por cliente + movimentação. Proxy: não existe carteira/saldo.
2. **Telas:** KPIs *Recebido em dinheiro*, *Recebimentos (+ se capped)*, *Clientes*, *Ticket médio*, *% do recebido*. `RelFiltros` (default `90d`) + ExportCsv. Charts "por mês"/"top clientes". Tabelas "Situação por cliente" + "Movimentação" (≤300). Banner cap.
3. **Backend:** `pullOS` (só IDs da unidade) → `pullPagamentos(ini,fim,osIds)` filtrando `metodo='dinheiro'` em memória (KPIs só `status='aprovado'`). `os` → `clientes`. [△ `osIds.slice(0,1000)` e `osComPag.slice(0,1000)` — unidade com >1000 OS subconta silenciosamente]. `.range()` até 20000.
4. **Integrações:** nenhuma.
5. **Estado real:** **PARCIAL (funcional sobre proxy)**. Lê tabela real `os_pagamentos`; sem mock/iframe. Conceito de saldo/crédito não existe (nota honesta).

6. **P/ 100%:** tabela de carteira/saldo; recalculer "Situação" como saldo; corrigir cap 1000 os_ids; `unidade_id` em `os_pagamentos`.
7. **Esforço: 4 dias.**

10. CRM — `/relatorios/crm` · perm `comercial`.

1. **O que faz:** Funil de leads de CLIENTE (`crm_leads pipeline='cliente'` , separado da Expansão) por origem/etapa/conversão.
2. **Telas:** KPIs *Leads no período (+ se capped)*, *Em negociação (Δ R\$ pipeline)*, *Convertidos (Δ R\$ ganho)*, *Taxa de conversão*. `RelFiltros` (default `mes`) + ExportCsv. Charts "por etapa"/"por origem top10". Tabelas Funil/Origem/Leads(≤ 300). Estado `semFonte`.
3. **Backend:** `crm_etapas ( .eq('ativo',true) , .eq('pipeline','cliente') );`
`crm_leads .select('id,nome,origem,servico_interesse,valor_estimado,etapa_id,status,temperatura,criado_em').eq('pipeline','cliente').eq('unidade_id') + .gte/.lt('criado_em') , .range()` **paginado** até `PULL_CAP=8000`.
4. **Integrações:** nenhuma (WhatsApp/Instagram só como rótulo de origem).
5. **Estado real: FUNCIONAL (dependente de dados).** Tabela real correta, escopo/paginação corretos, estados honestos. TODO: cohort de conversão e tempo médio no funil pendem de histórico de movimentação de etapa.
6. **P/ 100%:** persistir histórico de movimentações de etapa; garantir população de leads `pipeline='cliente'` por unidade.
7. **Esforço: 2 dias.**

11. Crédito Recorrente — `/relatorios/credito-recorrente` · perm `comercial`.

1. **O que faz:** Apura cobranças `os_pagamentos.metodo='credito_recorrente'` (forma PagoLivre) — MRR, falhas, cancelamentos.
2. **Telas:** KPIs *Assinaturas recorrentes*, *MRR recorrente*, *Falhas (recusado/estornado)*, *Cancelamentos*, *Ticket médio*. `RelFiltros` (default `90d`) + ExportCsv. Charts "por mês"/"top clientes". Tabelas por cliente + movimentação (≤ 300). Banner cap.
3. **Backend:** idêntico ao credito-dinheiro (mesmo `slice(0,1000)`), filtrando `metodo='credito_recorrente'`.
4. **Integrações:** conceitualmente PagoLivre, mas a página **não chama** API — só lê `os_pagamentos`.
5. **Estado real: PARCIAL.** Funcional se houver cobranças; sem tabela de assinatura cliente $\leftrightarrow$ plano $\rightarrow$ status Ativo/Pausado/Cancelado, modo e próxima cobrança ausentes.
6. **P/ 100%:** tabela `cliente_assinaturas`; integração real PagoLivre; corrigir cap 1000.
7. **Esforço: 5 dias.**

12. Descontos — `/relatorios/descontos` · perm `comercial`.

1. **O que faz:** Cada OS não-cancelada com `desconto_total>0` = aplicação de desconto; total, nº, % médio ponderado, vendedor de maior impacto.
2. **Telas:** KPIs *Descontos concedidos*, *Nº aplicações (+ se capped)*, *% médio*, *Maior impacto*. `RelFiltros` (default `90d`) + ExportCsv. Chart "por colaborador top10". Tabela aplicações (≤ 300). Banner cap.
3. **Backend:** `pullOS(status:['aberta','fechada'])` $\rightarrow$ filtra `desconto_total>0` em memória; % via `total_bruto`. `nomesPerfis` (via `criado_por`), `nomesClientes`. `.range()` até 20000.
4. **Integrações:** nenhuma.
5. **Estado real: FUNCIONAL.** Tabela real `os`, sem mock/TODO/iframe. Empty-state honesto.
6. **P/ 100%:** nenhum gap estrutural (opcional: cap 1000 em nomesPerfis, irrelevante).
7. **Esforço: 0.5 dia.**

13. Estatísticas — `/relatorios/estatisticas` · perm `comercial`.

1. **O que faz:** Visão consolidada (faturamento/agendamentos/ticket/clientes) com comparativo vs período anterior.
2. **Telas:** KPIs *Faturamento (Δ %)*, *Atendimentos (Δ % conclusão)*, *Ticket médio*, *Novos clientes*. `RelFiltros` (default `90d`). Charts "faturamento vs anterior"/"agendamentos". Tabela "Indicadores consolidados". Banner cap.
3. **Backend** (`Promise.all`): `lancamentos_financeiros ( tipo='receita' , .range()` até `SUM_CAP=20000` , `2x`); `agendamentos count:'exact',head:true` (`4x`); `clientes head:true` (`3x`, `unidade_origem_id` NÃO usado — base global).
4. **Integrações:** nenhuma.
5. **Estado real: FUNCIONAL (ressalva de escopo).** Fontes reais corretas, head:true eficiente, comparativo real. Base de clientes global até `unidade_origem_id` popular. TODO aba "Colaborador".
6. **P/ 100%:** popular `unidade_origem_id`; aba Colaborador (tabela `profissionais`).
7. **Esforço: 2 dias.**

14. Exportações — `/relatorios/exportacoes` · perm `comercial`.

1. **O que faz:** Substitui o log de exportações mock do legado por **hub read-only**: lista 6 datasets exportáveis com contagem real + link p/ `/exportacoes`.
2. **Telas:** Sem filtro/charts. KPIs (metric-box) *Registros exportáveis*, *Fontes com dados*, *Fontes vazias*, *Contagem indisponível*. Card CTA "Central de Exportações". Tabela "Fontes disponíveis" (Fonte/Descrição/Escopo/Registros/Status). 6 datasets: `clientes`, `lancamentos_financeiros` (×2), `agendamentos`, `colaboradores`, `sac_tickets`, `site_leads`.
3. **Backend:** por dataset `.select('id',{count:'exact',head:true}) + .eq(unidadeCol,unidadeId)`. Sem pull de linhas. `siteConfigurado()` decide texto de integração dos leads.
4. **Integrações:** referencia leads do site (`site_leads`) mas só faz count local; geração de CSV real está em `/exportacoes`.
5. **Estado real:** **FUNCIONAL (como hub)**; **histórico do legado FALTA**. Counts reais; honesto que log "quem/quando/qual arquivo" não existe (sem `export_log`).
6. **P/ 100%:** criar tabela de log de exportações + registrar cada download.
7. **Esforço: 2 dias.**

15. Faturamento — `/relatorios/faturamento` · perm `financeiro`.

1. **O que faz:** Soma receitas (`lancamentos_financeiros tipo='receita'`) por período, com comparativo, quebra por unidade e mês de competência.
2. **Telas:** KPIs *Faturamento*, *Lançamentos (+ se capped)*, *Ticket médio*, *[KPI período anterior + Δ%]*. `RelFiltros` (default `90d`) — sem `ExportCsv`. Charts "por mês" (label cru `YYYY-MM`)/"top unidades". Tabela por unidade + tfoot 100%. Banner cap.
3. **Backend:** `lancamentos_financeiros .select('valor,unidade_id,data_competencia').eq('tipo','receita') + .eq('unidade_id') + .gte/.lt('data_competencia')`, `.range()` até `SUM_CAP=20000` (2× atual+prev). Nomes de unidade vêm do contexto (sem query extra).
4. **Integrações:** nenhuma.
5. **Estado real:** **FUNCIONAL**. Um dos mais maduros do conjunto. Empty-state honesto. Cosmético: label de mês cru.
6. **P/ 100%:** formatar label do mês; opcional `ExportCsv`.
7. **Esforço: 0.5 dia.**

16. Ranking de Vendas — `/relatorios/ranking-vendas` · perm `comercial`.

1. **O que faz:** Ranqueia vendedores (`os.criado_por`) por valor de OS **fechadas** no período (réplica de `RANKS.vendas`).
2. **Telas:** KPIs *Total vendido*, *Vendedores*, *Líder (nome+valor)*, *Ticket médio*. Chart "Top 10 (R\$)". Tabela Posição(top3 destaque)/ Colaborador/Vendas/Valor/Ticket. `RankLimitSel` (Top 10/50/100/250/500). `ExportCsv`.
3. **Backend:** `pullOS(status:'fechada') .range() até 20000`; agrega por `criado_por`; `nomesPerfis (.in(ids.slice(0,1000)))`. Null → "Sem vendedor vinculado".
4. **Integrações:** nenhuma.
5. **Estado real:** **FUNCIONAL**. Empty-state e banner cap honestos, sem mock.
6. **P/ 100%:** essencialmente completo; opcional coluna por unidade; elevar cap 20k.
7. **Esforço: 0.5 dia.**

17. Fidelidade — `/relatorios/fidelidade` · perm `comercial`.

1. **O que faz:** SNAPSHOT de saldos de fidelidade (pontos + créditos) de `clientes`. Sem filtro de período (estado atual).
2. **Telas:** Toggle Pontos/Créditos (`?ordem=`). KPIs *Base total*, *Com pontos (+%)*, *Com créditos (+%)*, *Pontos/Créditos (top 100)*. Chart "Top 10". Tabela Cliente/Telefone/Pontos/Créditos + tfoot "Total (janela)".
3. **Backend:** `clientes` — counts `count:'estimated',head:true (.gt('saldo_pontos',0) / .gt('saldo_creditos',0))`; janela `.select('id,nome,telefone,saldo_pontos,saldo_creditos').order(ordemCol desc).gt(ordemCol,0).limit(TOP_N=100) [.limit(100) seguro]`. Totais só sobre os 100.
4. **Integrações:** nenhuma.
5. **Estado real:** **FUNCIONAL (ressalva de escopo)**. Colunas reais `saldo_pontos / saldo_creditos`. Fallback `indisponivel`. `unidade_origem_id` null → sem segmentação. TODO ledger de pontos p/ extrato temporal/expiração.
6. **P/ 100%:** popular `unidade_origem_id`; tabela de movimentação de pontos.
7. **Esforço: 2 dias.**

18. Financeiro / Contábil (DRE simples) — `/relatorios/financeiro` · perm `financeiro`.

1. **O que faz:** DRE simples sobre `lancamentos_financeiros`: receita×despesa por categoria, resultado e margem.
2. **Telas:** `RelFiltros` (default `90d`). KPIs *Receita*, *Despesa*, *Resultado (Δ margem%)*, *Lançamentos (+ se capped)*. Charts "Receita×Despesa×Resultado"/"Receita por categoria top8". Tabela "Demonstrativo" (linhas +Receitas, −Despesas, =Resultado). **Nota honesta quando `totalDespesa===0`**: "base atual só tem receitas".

3. **Backend:** `lançamentos_financeiros` `.select('valor,categoria_id,data_competencia,status').eq('tipo','receita','despesa')` (2 pulls) + `.eq('unidade_id') + .gte/.lt('data_competencia')`, `.range()` até `SUM_CAP=20000`. `plano_contas` `.in('id',catIds)` p/ nomes.
4. **Integrações:** nenhuma.
5. **Estado real:** **PARCIAL** — "só receitas, sem despesas" **CONFIRMADO**. O caminho de despesa está codado (a DRE já consome), mas não há linhas `tipo='despesa'` no backend → `totalDespesa===0` + nota honesta. Receita FUNCIONAL. TODO: integrar contas a pagar reais.
6. **P/ 100%:** ingerir lançamentos `tipo='despesa'` (contas a pagar); opcional comparativo período.
7. **Esforço:** 3 dias (dominado por ingestão de dados, não UI).

19. Mensagens WhatsApp API — `/relatorios/whatsapp` · perm **comercial**.

1. **O que faz:** Métricas de disparo de CAMPANHAS (enviadas/entregues/lidas/respondidas/falhas) de `campanhas_whatsapp`. Lê contadores agregados, não logs por mensagem.
2. **Telas:** KPIs (metric-box 2x4) Campanhas/Enviadas/Taxa entrega%/Taxa leitura%; Destinatários/Entregues/Respostas%/Falhas. Charts "Funil de mensagens"/"por status". Tabela "Campanhas" (Campanha/Status/Segmentação/Enviadas/Entregues/Lidas/Respostas/Falhas/Data) + tfoot.
3. **Backend:** `campanhas_whatsapp` `.select('id,nome,template_nome,segmentacao_tipo,status,concluido_em,criado_em,total_destinatarios,t_desc').limit(LIMITE=300)` [`.limit(300)` seguro, mas sem paginação → subconta se >300 campanhas] + `.eq('unidade_id') + .gte/.lt('criado_em')`. Erro → `semFonte`.
4. **Integrações:** WhatsApp — só LÊ a tabela agregada `campanhas_whatsapp` (populada por marketing/disparo). **Não** chama Uazapi/Evolution ao vivo; não há tabela de log por mensagem.
5. **Estado real:** **FUNCIONAL se houver campanhas**. Tabela real correta, sem iframe/mock. Empty-state honesto. **Coerência do menu:** o index `/relatorios` ainda lista esta rota como "Em desenvolvimento".
6. **P/ 100%:** confirmar população em prod; paginar >300; drill-down por destinatário se surgir log.
7. **Esforço:** 1 dia.

20. Metas — `/relatorios/metas` · perm **comercial**.

1. **O que faz:** Meta×realizado por colaborador de `metas_colaborador`, escopado à unidade via `colaboradores`; premiação liberada a ≥80% de atingimento agregado de venda (regra legado).
2. **Telas:** Toggle %Atingido/Valor (`?visualizar=`). ExportCsv. KPIs *Meta (venda)*, *Realizado*, *%Atingido* (Δ 80%), *Premiação* (*Liberada/Bloqueada*). Chart "Atingimento por colaborador top10". Tabela colorida (verde≥100/âmbar≥80/vermelho<80). Empty-state "Cadastre metas em Cadastros-Metas".
3. **Backend:** `colaboradores` `.select('id,nome').eq('status','ativo')` `.range()` paginado até 50000 (escopo por unidade, pois `metas_colaborador` não tem `unidade_id`). `metas_colaborador` `.select('id,colaborador_id,indicador,unidade_medida,valor_alvo,valor_realizado,status')` — com unidade: `.in('colaborador_id',grupo)` em lotes de 200; `.range()`.
4. **Integrações:** nenhuma.
5. **Estado real:** **FUNCIONAL (dependente de dados)**. Tabelas reais, multitenant paginado correto (corrige bug de corte >500 colaboradores). Empty-state honesto. Depende de metas cadastradas.
6. **P/ 100%:** garantir metas populadas; opcional filtro de período; auto-preencher `valor_realizado` de vendas reais.
7. **Esforço:** 1.5 dias.

21. Ordens de serviço — `/relatorios/ordens-servico` · perm **comercial**.

1. **O que faz:** Relatório sobre `os`: contagem por status/origem, valor total, lista detalhada, escopo unidade+criação.
2. **Telas:** `RelFiltros` (default `90d`) + ExportCsv. KPIs *OS no período*, *Finalizadas* (Δ % amostra), *Em aberto*, *Valor total* (todos com + se capped). Charts "por status"/"por origem". Tabela ($\leq$ 300) Cliente/Origem/Status/Abertura/Valor. Banner cap.
3. **Backend:** `pullOS` → `os` (colunas do helper) + `.eq('unidade_id') + .gte/.lt('criado_em')` + status opcional, `.range()` até `PULL_CAP=20000`. `nomesClientes` (`.in(ids.slice(0,1000))`). Sem embed.
4. **Integrações:** nenhuma.
5. **Estado real:** **FUNCIONAL**. O mais production-ready dos relatórios. Sem mock/iframe/TODO.
6. **P/ 100%:** essencialmente completo; opcional paginação server-side >300; comparativo.
7. **Esforço:** 0.5 dia.

22. Pacotes — `/relatorios/pacotes` · perm **comercial**.

1. **O que faz:** Vendas de pacotes = OS com `origem='pacote'`, agregadas por mês.

2. **Telas:** `RelFiltros` (default `90d`). KPIs *Receita de pacotes*, *Pacotes vendidos (+ se capped)*, *Ticket médio (pagos)*, *Cortesia (100% desc.)*. Charts "Receita por mês"/"Vendidos por mês". Tabela "Detalhamento por mês" + tfoot.
3. **Backend:** `pullPacotes()` local →
`os .select('status,preco_total,desconto_total,total,criado_em').eq('origem','pacote') + .eq('unidade_id') + .gte/.lt('criado_em'), .range() até PULL_CAP=20000`. Cancelada excluída; `total>0` = pago senão cortesia. Agregação mensal em JS.
4. **Integrações:** nenhuma.
5. **Estado real: FUNCIONAL (limitação honesta).** `os_pacotes` veio VAZIO do import e OS sem `pacote_id` → **sem ranking por nome de pacote histórico** (só aparece com novas vendas do PDV). Divulgado no rodapé. Sem mock/iframe.
6. **P/ 100%:** popular `os_pacotes`/backfill `pacote_id` (migração/PDV) p/ breakdown por pacote.
7. **Esforço: 1 dia** (página; detalhe por pacote depende de dado externo).

23. Pagamentos — `/relatorios/pagamentos` · perm `financeiro`.

1. **O que faz:** Apuração de pagamentos de OS por `data_pagamento`: Previsto/Recebido/Pendente/Erro, taxa de sucesso, quebra por método.
2. **Telas:** KPIs *Previsto*, *Recebido (Δ %sucesso)*, *Pendente*, *Com erro (tom down se >0)*, *Taxa de sucesso*. Charts "Recebido por método top10"/"Previsto×Recebido×Pendente×Erro". Tabela (≤ 300) Data/Cliente/Método/Valor/Status. `RelFiltros` (default `90d`) + ExportCsv.
3. **Backend:** `pullOS` (IDs unidade) → `pullPagamentos(ini,fim,osIds)`
`os_pagamentos.select('os_id,data_pagamento,metodo,tipo,valor,status') .range() até 20000; .in('os_id',osIds.slice(0,1000))` [△ **subconta unidade >1000 OS**]. Nomes via `os` → `nomesClientes`. `METODO_LABEL`.
4. **Integrações:** nenhuma (sem gateway; lê `os_pagamentos`).
5. **Estado real: FUNCIONAL.** Dado real. TODO(legado `relPremiacoesHTML`): roster de premiação/Matriz de Metas NÃO implementado aqui (deferido a Cadastros-Comissões/Relatórios-Metas). Sem mock/iframe.
6. **P/ 100%:** tratar cap `slice(0,1000)` p/ unidades grandes (chunk `.in` ou `unidade_id` em `os_pagamentos`); implementar premiação.
7. **Esforço: 1.5 dias.**

24. Perfis de acesso — `/relatorios/perfis-acesso` · perm `comercial`.

1. **O que faz:** Relatório RBAC read-only — quem tem qual cargo, contagem de permissões por perfil, escopo por unidade.
2. **Telas:** KPIs *Perfis cadastrados*, *Usuários com perfil*, *Perfis de sistema*, *Perfis inativos (Δ desativados)*. Charts "Usuários por perfil top10"/"Permissões por perfil top10". Tabelas "Perfis cadastrados" e "Usuários e seus perfis". Link "Gerenciar perfis" → `/cadastros/perfis`.
3. **Backend:** usa `adminClient()` **service-role**. `Promise.all: cargos.select('id,nome,slug,descricao,is_sistema,ativo'); usuario_cargos.select('perfil_id,cargo_id,unidade_id,ativo')` [△ **sem .limit / .range** → **corte silencioso de 1000**]; `cargo_permissoes.select('cargo_id')` [△ **mesmo risco**]; `perfis_usuario.select('id,nome_completo,email').limit(2000)` [△ **.limit(2000) > max_rows default 1000** → **trunca a 1000**]. Joins em memória. Escopo por `usuario_cargos.unidade_id`.
4. **Integrações:** nenhuma.
5. **Estado real: FUNCIONAL (bug latente de escala).** Dados RBAC reais; `semFonte` honesto. Subconta se `usuario_cargos / cargo_permissoes / perfis_usuario` passarem de 1000 linhas.
6. **P/ 100%:** paginar (`.range()`) as três tabelas; corrigir `.limit(2000)`.
7. **Esforço: 1 dia.**

[Órfã] Notas Fiscais (relatório) — `/relatorios/notas-fiscais` · fora do menu (removida 03/07, comentário linha 99)

1. **O que faz:** Relatório NFS-e sobre `nfse`, escopo unidade + data de emissão, agregando por status e tipo. (*page.tsx existe e está em `ROTAS_FUNCIONAIS`, mas o link foi retirado do menu — só a emissão em `/notas` fica.*)
2. **Telas:** `RelFiltros` (default `90d`, eixo `criado_em`). KPIs *Notas emitidas (Δ %)*, *Valor autorizado*, *Canceladas (Δ processando)*, *Com erro*. Charts "Por status"/"Valor por tipo". Tabelas "Resumo por status" + "Notas no período" (≤ 200).
3. **Backend:**
`nfse .select('id,numero,competencia,tipo,fato_gerador,cliente_nome,valor,status,criado_em,unidade_id,cliente:clientes)` — **embed `clientes`**. `.eq('unidade_id') + .gte/.lt('criado_em'), .range() até ROW_CAP=5000`. Erro `relation|does not exist` → **semTabela** "em preparação".
4. **Integrações:** nenhuma direta (emissão fiscal fica em `/notas`).

5. **Estado real: PARCIAL/condicional.** Código completo lendo tabela real; exibe dado se migration `nfse` aplicada + linhas existirem, senão empty honesto. `competencia` irregular → período usa `criado_em`.
6. **P/ 100%:** aplicar migration `nfse` + integração de emissão real; (decidir se volta ao menu).
7. **Esforço: 2 dias.**

PARTE 2 — DASHBOARDS (grupo `dash`)

D1. Financeiro / Contábil — `/dashboards/financeiro` · perm `financeiro`.

1. **O que faz:** Previsto (todos) × realizado (`status='pago'`) de contas a pagar/receber, royalties auto para franqueadas, receita por categoria.
2. **Telas:** `DashFiltros` (8 presets + custom, select Unidade, ExportCsv; default `90d`). 6 KPIs: *Contas a pagar previstas/realizadas*, *Royalties a pagar (auto)*, *Contas a receber previstas/realizadas*, *Total a receber*. Banner royalties (franqueada 10%/venc.10 vs "Loja própria"). 3 Charts (Movimentação; Categorias pagar; Categorias receber top8). Tabela "Receita por categoria" + tfoot. Banner cap.
3. **Backend:** `pullLancamentos('receita'|'despesa')` `.range()` até `SUM_CAP=20000` ; `unidades.select('cnpj').eq('id').maybeSingle()` ; `faturamentoMesAnterior ( status='pago' )` ; `plano_contas.in('id',catIds)` .
4. **Integrações:** nenhuma.
5. **Estado real: FUNCIONAL.** Tudo de tabelas reais; royalties de receita realizada do mês anterior (`calcRoyalties` , CNPJ `44.442.908`). Sem iframe/mock. Ressalva: KPI "Total a receber" duplica "previstas" (intencional). Depende de `status='pago'` correto no ERP.
6. **P/ 100%:** validar semântica `data_competencia × data_vencimento` e `status='pago'` ; distinguir "Total a receber" de "previstas".
7. **Esforço: 1 dia.**

D2. Gerencial — `/dashboards/gerencial` · perm `comercial`.

1. **O que faz:** Faturamento, ticket, atendimentos, sessões e ranking top-10 serviços de OS fechadas (`os_servicos`) + financeiro real.
2. **Telas:** `DashFiltros` (default `90d`). 5 KPIs: *Faturamento*, *Ticket médio*, *Atendimentos*, *Sessões realizadas*, *Taxa de retorno (Δ agend.)*. `GerServBusca` (busca serviço fora do top-10). 4 Charts (Top10 faturamento; Top10 sessões; Faturamento por forma de pagamento; Receita por mês). Tabela "Top 10 detalhamento" + CSV.
3. **Backend:** `contar('agendamentos')` `count: 'exact', head: true` ×3 (total/concluído/cancelado, `dateCol='inicio'`); `pullLancamentos('receita')` `.range()` ; `pullOS(status: 'fechada')` `.range()` até `20000` ; `pullServicosPorOS` → `os_servicos.select('servico_id,quantidade,preco_total,total,servicos(nome)')` `embed servicos` , `.in('os_id', chunk de 120)` , `amostra RANK_MAX_OS=3000`, degrada em erro.
4. **Integrações:** nenhuma.
5. **Estado real: FUNCIONAL.** Ratios/tickets **REAIS, não hardcoded** (`ticket=receita/atend` , `taxaRetorno=pct(concluido,total)` , serviços de `os_servicos`). Ressalvas: "Taxa de retorno" é proxy de comparecimento (não retorno real); ranking é amostra 3000 OS (top-10 aproximado em janelas amplas).
6. **P/ 100%:** métrica real de recompra; remover cap 3000 (RPC/materialized view); validar `os_servicos.total × preco_total` .
7. **Esforço: 2 dias.**

D3. Funil de Vendas — `/dashboards/funil` · perm `comercial`.

1. **O que faz:** Funil real do ERP: Agendamentos → Comparecimento → Vendas (OS fechadas) → Receita, com conversão por unidade e leads por origem.
2. **Telas:** Filtros Período (default `tudo`), Unidade, **Tipo de unidade** (Ambas/Próprias/Franquias). Funnel SVG 4 estágios. 6 KPIs (*Agendamentos*, *Comparecimento (n+%)*, *Vendas (n+%)*, *Ticket médio*, *Conversão total*, *Receita*). 2 Charts (Leads por origem; Agendamentos por status). Breakdown por unidade (chart+tabela). Nota "Novos×Revenda não registrado". Banner cap.
3. **Backend:** `unidades.select('id,cnpj')` (filtro própria×franqueada via `uniEhPropria`); `contar('agendamentos')` `head: true` por status; `pullOS(status: 'fechada')` por unidade; `crm_leads.select('origem').eq('pipeline','cliente')` [△ `sem .range / .limit / count` → **leitura potencialmente ilimitada de origem, contra a própria "regra de ouro"**].
4. **Integrações:** nenhuma (crm_leads interno).
5. **Estado real: FUNCIONAL.** Ratios hardcoded do legado **REMOVIDOS** (confirmado em `dashboards.ts`). Números reais. Nota honesta Novos×Revenda. Risco de eficiência: query `crm_leads.origem` sem limite.
6. **P/ 100%:** limitar/agregar `crm_leads.origem` (RPC group-by); expor Novos×Revenda quando o ERP registrar por venda.

7. Esforço: 1.5 dias.

D4–D7. Vendas (vendas-geral / vendas-mes / vendas-comparativo / vendas-historico) · admin:true (+ ehAdmin(papel))

Os 4 são wrappers de 3 linhas que renderizam <VendasReal slug sp podeVer={ehAdmin(papel)}> , diferindo só pelo slug → VENDAS_CFG :

SLUG	TÍTULO	DEFPERIODO	COMPARATIVO
vendas-geral	Vendas · Visão Geral	ano	não
vendas-mes	Vendas · Mês Atual	mes	não
vendas-comparativo	Vendas · Comparativo	mes	sim
vendas-historico	Vendas · Histórico	ano	não

- 1. **O que faz:** Dashboards de vendas admin-only (franqueadora) sobre OS fechadas reais; vendas-comparativo puxa janela anterior e mostra deltas.
- 2. **Telas:** Gate RBAC "Acesso restrito" p/ não-admin (nenhuma query roda). Badge ADMIN. Filtros Período/Unidade/ExportCsv. 4 KPIs (Receita (OS fechadas), Vendas, Ticket médio, Descontos concedidos; Receita/Vendas com Δ% no comparativo). Card comparativo. Charts "Receita por mês (12m)" + "Receita por unidade" (só sem unidade fixa). Tabela "Vendas por unidade".
- 3. **Backend:** pullOS(status:'fechada') .range() até 20000 (2× no comparativo; por unidade nas 20 primeiras). vendas=rows.length , receita=Σtotal , desconto=Σdesconto_total , série mensal por criado_em.slice(0,7) .
- 4. **Integrações:** nenhuma.
- 5. **Estado real: FUNCIONAL (migrado do iframe legado).** Comentários confirmam substituição do iframe estático que apontava p/ outro projeto Supabase (login próprio, tabelas inexistentes no ERP). 100% real agora. Empty-states honestos. Ressalva: os 4 são quase-duplicatas (mesma view, presets diferentes); vendas-historico mostra os mesmos 12 meses de vendas-geral .
- 6. **P/ 100%:** diferenciar as telas (histórico multi-ano — hoje ultimosMeses(fim,12) ; comparativo por unidade/serviço; ranking de vendedor via criado_por + nomesPerfis , disponível mas não usado).
- 7. **Esforço: 2.5 dias** (para os 4 combinados virarem views genuinamente distintas).

Helpers de dashboard: error.tsx (boundary — "Não foi possível carregar os indicadores" + retry; garante que query falha nunca vira zero silencioso). SegToggle.tsx é código morto — fornecido mas NÃO usado por nenhuma página (o funil usa <select> do DashFiltros).

TABELA-RESUMO

Relatórios (25 telas — 24 no menu + notas-fiscais órfã)

#	RELATÓRIO	PERM	ESTADO	DIAS
1	Assinaturas	comercial.	PARCIAL (só catálogo)	4
2	Ocorrências/Intercorrências	comercial.	FALTA (placeholder, sem query)	4
3	Agendamentos	comercial.	FUNCIONAL (dims parciais)	1.5
4	Anamnese/Ficha Técnica	comercial.	PARCIAL (só catálogo)	4
5	Atendimentos	comercial.	FUNCIONAL (dims parciais)	2
6	Avaliações	comercial.	FALTA (empty honesto, sem tabela)	4
7	Clientes	comercial.	FUNCIONAL (estimated, sem unidade)	2
8	Contratos	comercial.	PARCIAL (seed data)	3
9	Crédito em dinheiro	comercial.	PARCIAL (proxy, Δcap1000)	4
10	CRM	comercial.	FUNCIONAL (dep. dados)	2
11	Crédito Recorrente	comercial.	PARCIAL (Δcap1000)	5
12	Descontos	comercial.	FUNCIONAL	0.5
13	Estatísticas	comercial.	FUNCIONAL (base global)	2
14	Exportações	comercial.	FUNCIONAL (hub; log falta)	2
15	Faturamento	financeiro.	FUNCIONAL	0.5
16	Ranking de Vendas	comercial.	FUNCIONAL	0.5

#	RELATÓRIO	PERM	ESTADO	DIAS
17	Fidelidade	comercial.	FUNCIONAL (sem unidade)	2
18	Financeiro/Contábil (DRE)	financeiro.	PARCIAL (só receitas)	3
19	Mensagens WhatsApp API	comercial.	FUNCIONAL (se há campanhas)	1
20	Metas	comercial.	FUNCIONAL (dep. dados)	1.5
21	Ordens de serviço	comercial.	FUNCIONAL	0.5
22	Pacotes	comercial.	FUNCIONAL (sem breakdown pacote)	1
23	Pagamentos	financeiro.	FUNCIONAL (Δcap1000; sem premiação)	1.5
24	Perfis de acesso	comercial.	FUNCIONAL (Δbug escala >1000)	1
—	Notas Fiscais (órfã do menu)	(s/perm)	PARCIAL/condicional (migration)	2
	Subtotal Relatórios			≈54.5

Dashboards (7 telas)

#	DASHBOARD	PERM	ESTADO	DIAS
D1	Financeiro/Contábil	financeiro.	FUNCIONAL	1
D2	Gerencial	comercial.	FUNCIONAL (retorno=proxy; ranking amostra 3k)	2
D3	Funil de Vendas	comercial.	FUNCIONAL (Δcrm_leads sem limite)	1.5
D4	Vendas · Visão Geral	admin	FUNCIONAL (migrado do iframe)	—
D5	Vendas · Mês Atual	admin	FUNCIONAL	—
D6	Vendas · Comparativo	admin	FUNCIONAL	—
D7	Vendas · Histórico	admin	FUNCIONAL (=geral, 12m)	—
	D4–D7 (VendasReal compartilhado)			2.5
	Subtotal Dashboards			7

TOTAL GERAL p/ 100%: ≈ 61.5 dias-dev (dominado por fontes de dados upstream — despesas, nfse, tabelas de avaliações/ ocorrências/assinatura-cliente/carteira — não por UI de relatório).

Achados críticos para a homologação (confrontar)

- 1. **Menu otimista:** ROTAS_FUNCIONAIS marca as 31 rotas como "funcional", mas avaliacoes e ocorrencias não têm tabela-fonte (FALTA real, empty-state honesto). Critério do flag = "empty-state honesto OU query real", não "com dados".
- 2. **Bugs de corte silencioso (slice(0,1000)):** credito-dinheiro, credito-recorrente, pagamentos — unidades com >1000 OS subcontam pagamentos silenciosamente. perfis-acesso tem 3 leituras sem paginação (usuario_cargos, cargo_permissoes, perfis_usuario.limit(2000)) que truncam em 1000. funil lê crm_leads.origem sem limite (eficiência).
- 3. **relatorios/financeiro (DRE):** confirmado "só receitas, sem despesas" — código pronto p/ despesa, mas backend sem linhas tipo='despesa'.
- 4. **Vendas migrados:** os 4 dashboards vendas-* saíram do iframe estático (que apontava p/ outro Supabase) e hoje leem OS reais — porém são quase-duplicatas (mesma view, presets diferentes).
- 5. **os_pacotes e várias FKs vieram VAZIAS do import BEMP** (profissional_id, servico_id, cliente_id em agendamentos; pacote_id em OS; unidade_origem_id em clientes) — bloqueia breakdowns por profissional/serviço/pacote/unidade em vários relatórios; não é bug de código, é lacuna de dado.
- 6. **Nenhuma página toca integração ao vivo** (Uazapi/Evolution/PagoLivre/storage): relatorios/whatsapp só lê a tabela agregada campanhas_whatsapp; credito-recorrente só menciona PagoLivre conceitualmente.
- 7. **Código morto:** src/components/dashboards/SegToggle.tsx não é usado por nenhuma página.
- 8. **Index /relatorios desatualizado:** lista pacotes e whatsapp em "Em desenvolvimento" embora ambos estejam construídos e leiam tabelas reais (3 arrays hardcoded como fonte de verdade paralela ao menu).

Arquivos-fonte de fundação relevantes: src/lib/relatorios.ts, src/lib/dashboards.ts, src/components/dashboards/agg.ts, src/components/dashboards/VendasReal.tsx, src/components/relatorios/relPeriodo.ts, src/lib/menu.ts.

Módulo 3b — Gestão: Operação, Marketing & RH

Documento oficial de homologação. Escopo: seção **Gestão** do menu (`src/lib/menu.ts`), **exceto** Relatórios e Dashboards. 14 funcionalidades / 24 rotas: Mensagens e Automações, Disparos WhatsApp API, CRM, Leads do Site, Canais, Gestão de Indiques, Recursos Humanos (9 folhas), Marketing, Comunicados, Chamados, Checklist de Indicadores, Universidade Corporativa, Disco Virtual, Notas Fiscais. Base: `/home/jvneto/ProjetosLMK/Laser/laserco-power-system` · Next.js 15 (App Router) + Supabase (backend `lki`). Cada bloco é fiel ao código-fonte lido integralmente (page.tsx + actions.ts + libs + migrations). Nada foi inventado.

Fundação comum — como o RBAC realmente decide (vale para TODAS as telas abaixo)

Há dois eixos independentes de controle. Confundi-los é o erro clássico nesta base:

(A) Visibilidade no menu → por **RECURSO** (cargo→permissões), **NÃO** por **papel**. `src/components/layout/Sidebar.tsx:23-25` `hasPerm(perm, recursos)` : se o `perm` termina em `.` (ex.: `'marketing.'`, `'operacoes.'`, `'rh.'`) basta ter QUALQUER recurso do módulo (`recursos.some(r => r.startsWith(perm))`); sem sufixo (ex.: `'crm.lead'`, `'rh.ponto'`, `'treinamento.curso'`) exige o recurso exato. `recursos` vêm de `getSessionContext` → `resolveRecursos` (`src/lib/session.ts:63-82`), que lê `usuario_cargos` → `cargo_permissoes` → `permissoes.recurso_id` via service-role. `admin_geral` faz bypass total (`isAdmin`, `recursos=[]`). Seed dos perfis em `scripts/migrations/perfis-acesso.sql:20-77` (nível **CARGO**): - `crm.*` → super_admin, administrador, **expansão** (`crm *`), marketing (`crm ler`), diretor/auditor (`* ler/exportar`). - `marketing.*` → super_admin, administrador, **marketing**, diretor, auditor, expansão (`marketing ler`). - `operacoes.*` → super_admin, administrador, **operacoes**, franqueado, gerente_unidade, diretor, auditor, supervisor/técnico (`ler`). - `rh.*` → super_admin, administrador, **rh**, diretor, auditor, franqueado (`ler`), gerente_unidade (`ler`). - `treinamento.*` → super_admin, administrador, **rh**, diretor, auditor, franqueado (`ler`). - `financeiro.*` → super_admin, administrador, **financeiro**, diretor, jurídico (`ler/exportar`), auditor, franqueado/gerente (`ler`).

(B) Autorização de AÇÃO (server actions) → por **papel** (`perfis_usuario.papel`). `src/lib/rbac.ts: ehAdmin/temPapel/exigirPapel`. `admin_geral` sempre passa; senão o `papel` (`colaborador`, `rh`, `gestor`, `admin_geral`, `financeiro`, `crm`, `tecnico`, `operacoes`, `sac`) precisa estar na lista da action.

Descasamento estrutural (atravessa todas as telas): quem **VÊ** (eixo recurso/cargo) pode **não poder ESCREVER** (eixo papel). Ex.: um cargo com `marketing.*` mas `papel='colaborador'` vê a tela mas toda escrita retorna "sem permissão".

Integração WhatsApp (Uazapi) é REAL (`src/lib/uazapi.ts`, `UAZAPI_BASE_URL` / `UAZAPI_ADMIN_TOKEN` presentes em `.env.local`), mas **poucas telas a disparam de fato** — o mapa de quem envia de verdade está por bloco. **Não há pg_cron/pg_net/Edge Function** para nenhuma automação programada desta seção (as únicas rotas `/api` são `webhooks/uazapi` e `cron/ingest-sac`).

Todas as 24 rotas constam em `ROTAS_FUNCIONAIS` (`src/lib/menu.ts:215-256`) → "acesas" no menu. Isso **NÃO** significa 100% completas — o estado real (funcional/parcial/stub) está detalhado por bloco.

1) Mensagens e Automações — `/automacoes`

1. Rota/perm. `/automacoes` · `perm: 'marketing.campanha'` (recurso exato). **Vê**: `admin_geral` + cargos com `marketing.campanha` (`super_admin`, `administrador`, `marketing`). Diretor/auditor têm `marketing:ler/exportar` → veem se o recurso `marketing.campanha` estiver na grade deles. **Escreve**: `PAPEIS_ESCRITA=['gestor','operacoes']` + `admin` (`actions.ts:9`). Arquivos: `src/app/(app)/automacoes/page.tsx`, `.../actions.ts`, `src/lib/automacoes.ts`, `src/components/automacoes/AutomacoesView.tsx`.

2. O que faz. Catálogo das 22 automações padrão da rede (revenda 8m, boas-vindas, confirmação/lembretes de agenda, no-show, pós-sessão, NPS, aniversário, reativação, nutrição de leads, metas etc.) com liga/desliga por unidade, automações personalizadas e a configuração do fluxo de não-comparecimento.

3. Telas/abas/modais. 1 tela. Grid de cards do catálogo (`AUTOS_PADRAO`, 22 itens em `src/lib/automacoes.ts:44-80`) filtráveis por categoria (`AUTO_CATEGORIAS`), cada card com switch ativa/inativa e detalhe de passos (`AutoDet`). Card do WhatsApp da unidade (status do canal). Modais: **Nova automação** (personalizada), **Editar automação**, **Config Não-comparecimento** (`NoShowForm`: 1ª msg após, máx/dia, intervalo, mensagem, reagenda/exclui/oculta). KPIs: ativas X/Y (real) + enviadasMes/taxaResposta/recuperados.

4. Backend. Tabelas: `automacoes_estado` (override on/off por `unidade_id,chave`), `automacoes_custom` (personalizadas: escopo `rede / unidade`, nome/gatilho/acao/categoria), `automacao_noshow` (config por `unidade_id`), `canais_whatsapp` (para o status do

canal). Server actions (`actions.ts`): `alternarAutomacao` (upsert estado), `criarAutomacao` (admin→escopo `rede`; gestor/operacoes→ `unidade`), `editarAutomacao`, `excluirAutomacao` (padrão da rede só admin), `salvarNoShow`. O catálogo `AUTOS_PADRAO` é **estático em código** (não é tabela).

5. Integrações. STUB de execução — confirmado. Nenhum scheduler/cron/worker executa as automações: `grep` por `automacoes_estado / automacao_noshow / AUTOS_PADRAO` só bate dentro da própria feature — nada nas rotas `/api`, nada em `webhooks/uazapi`. As automações **não disparam WhatsApp nem push**; a tela apenas **persiste a configuração**. O próprio código confessa a ausência de telemetria: KPIs `enviadasMes/taxaResposta/recuperados` são `null` propositalmente — *"sem telemetria de envio/no-show no backend ainda → null = estado honesto (antes eram 4820/38/64 inventados)"* (`page.tsx:82-90`). O catálogo declara que o estado do legado *"vive em memória"* (`src/lib/automacoes.ts:4-9`).

6. Estado real: PARCIAL (config real, execução FALTA). Evidência de persistência real:

`sb.from('automacoes_estado').upsert({... ativa ...}, { onConflict: 'unidade_id,chave' })` (`actions.ts:32-35`); `custom/no-show` idem. Evidência de que NÃO executa: inexistência de qualquer consumidor/agendador dos gatilhos. É a definição de "toggle de vitrine": grava a intenção, não realiza a ação.

7. Requisitos p/ 100%. Motor de automação: agendador (pg_cron/Edge Function/worker externo) que avalie cada gatilho (8 meses desde a sessão, 24h/2h antes, no-show +2h, aniversário, inatividade 60d, meta a cada 3 dias etc.) contra dados reais (agendamentos, clientes, metas), monte a mensagem com placeholders e chame `uazapi.sendText/criarCampanhaSimples`; telemetria de envio (enviadas/respostas/recuperados) por automação; execução real do fluxo no-show (2 tentativas, exclusão, cômputo). É o item mais pesado da seção.

8. Esforço p/ 100%: ~10 dev-days.

2) Disparos WhatsApp API — `/disparos`

1. Rota/perm. `/disparos` · perm: `'marketing.campanha'`. **Vê:** = Automações (marketing.campanha + admin). **Escreve:** `PAPEIS_ESCRITA=['gestor','operacoes']` + admin (`actions.ts:10`). Arquivos: `src/app/(app)/disparos/page.tsx`, .../ `actions.ts`, componente `src/components/disparos/DisparosTabs.tsx` + `DisparoComposer.tsx`; **reusa o disparo real de** `src/app/(app)/expansao/disparos/actions.ts`.

2. O que faz. Envio em massa por WhatsApp: monta campanhas por canal conectado, segmenta/importa bases de contatos, agenda envios (delay anti-ban gerido pela Uazapi) e organiza Grupos VIP; histórico de campanhas por unidade.

3. Telas/abas/modais. Abas (`DisparosTabs`): **Campanhas** (histórico + KPIs), **Bases** (segmentador + importar externa), **VIP** (agendamento de grupos), **API** (cards de status dos canais Uazapi). **Composer** (`DisparoComposer`) com seleção de canal, texto/template, público (listas reais), delay min/max, agendamento. Modais de segmento (`SEG_CAMPOS`) e importação.

4. Backend. Tabelas: `disparo_campanhas` (nome, base_nome, canal_nome, status[draft/sched/run/done], `enviadas/entregues/lidas/respostas`, `agendada_para`, `uazapi_id`, `unidade_id`), `disparo_bases` (nome, tipo[sistema/externa], contatos, criterios jsonb, numeros[]), `vip_grupos` (datas convite/aquecimento/oferta, membros, status, link_publico), `disparo_templates`, `clientes/servicos/unidades` (segmentador). Actions (`disparos/actions.ts`): `criarBaseSegmento` (COUNT real), `importarBaseExterna`, `excluirBase`, `registrarCampanha`, `excluirCampanha`, `respondentesParaCRM`, `agendarGrupoVip`, `excluirGrupoVip`. Envio real (`expansao/disparos/actions.ts`): `dispararCampanha`, `dadosDisparos`, `listarTemplates/salvarTemplate/excluirTemplate`.

5. Integrações. Envio = REAL via Uazapi `/sender/simple`. `DisparoComposer` chama `dispararCampanha` (`expansao/disparos/actions.ts:51`) → valida canal `connected` → `criarCampanhaSimples`(canal.token, { `numbers`, `text`, `delayMin/Max`, `scheduledFor` }) (`src/lib/uazapi.ts:232-245`, endpoint `/sender/simple`, agendamento por epoch-ms) → `registrarCampanha` grava o histórico com `enviadas = nº submetidos`. Segmentador = **COUNT real** de `clientes` (verificado/cidade/estado; critérios de histórico de compra são honestamente ignorados em vez de fabricar estimativa — `disparos/actions.ts:22-35`).

6. Estado real: PARCIAL (envio funcional; métricas e ações derivadas mortas). - FUNCIONAL: disparo/agendamento real, bases (segmento COUNT real + importação de números), VIP e templates persistem. - **ACHADO CRÍTICO — métricas nunca gravadas:** as colunas `disparo_campanhas.entregues / lidas / respostas` e `vip_grupos.membros` **jamais são escritas por código algum**. `grep from('disparo_campanhas')` retorna só `insert` (`actions.ts:113`), `delete` (`:127`), `select` (`:141` e `page.tsx:51`) — **nenhum update**. O webhook `api/webhooks/uazapi/route.ts` alimenta apenas `sac_whatsapp_chats` (SAC/IA), não os `messages_update` das campanhas. Consequência: `entregues/lidas/respostas/membros` ficam **permanentemente 0** (estado honesto, mas cego). - **Ação derivada morta:** `respondentesParaCRM` (`actions.ts:137-165`) lê `disparo_campanhas.respostas` (sempre 0) → sempre retorna *"Esta campanha ainda não tem respondentes"*. Fica inoperante até existir a integração de callbacks (ou preenchimento manual do campo no banco). - VIP: `agendarGrupoVip` grava `link_publico = laserco.app/vip/<slug>` (URL cosmética, sem página pública real que colete membros).

7. Requisitos p/ 100%. Ligar `messages / messages_update` do webhook Uazapi às campanhas (casar `uazapi_id / messageid` → atualizar enviadas/entregues/lidas e criar respostas → destravar `respondentesParaCRM`); página/coleta real de membros VIP; opcional: envio de mídia em massa e relatório por campanha.

8. Esforço p/ 100%: ~4 dev-days.

3) CRM — `/crm`

1. Rota/perm. `/crm` · perm: `'crm.lead'` (recurso exato). **Vê:** `admin_geral` + cargos com `crm.lead` (`super_admin`, `administrador`, `expansão`, e quem tenha `crm:ler` como marketing/diretor/auditor). **Escreve leads:** qualquer sessão autenticada (RLS decide); **personalizar funil (etapas):** só `admin_geral`. Arquivos: `src/app/(app)/crm/page.tsx`, `.../actions.ts`, `src/components/crm/CrmBoard.tsx`.

2. O que faz. Funil de vendas de CLIENTES em Kanban (drag-and-drop entre etapas), com origem/temperatura/score do lead; separado do funil de Expansão (franquias) por `pipeline='cliente'`.

3. Telas/abas/modais. 1 tela — board Kanban (`CrmBoard`) com colunas = `crm_etapas` (pipeline cliente) e cards = `crm_leads`; contagem real por etapa (query separada, sem cair no teto de 500 do board — `page.tsx:31-45`). Modal **Novo lead** (nome, telefone, origem, serviço, valor estimado, unidade, etapa, responsável, temperatura). Ações de funil (admin): criar/renomear/excluir etapa.

4. Backend. Tabelas: `crm_etapas` (nome, ordem, cor, is_sistema, ativo, `pipeline`), `crm_leads` (nome, telefone, origem, `servico_interesse`, `valor_estimado`, `etapa_id`, status, `ia_score`, temperatura, `responsavel_id`, `empresa_id`, `unidade_id`, `pipeline`), `perfis_usuario` (responsáveis), `unidades`. Constraints reais (migrations 015/050): `origem` CHECK (`manual/formulario/instagram/whatsapp/indicacao/google/outros/geolocalizado/site`) e `temperatura` CHECK (`gelado/frio/morno/quente/ardente`) — o código mapeia rótulos do legado para esses valores (`mapOrigem`, `actions.ts:31-43`). Actions: `criarLead`, `moverLead`, `criarEtapa / renomearEtapa / excluirEtapa` (todas admin; `excluirEtapa` protege etapas do sistema e etapas com leads).

5. Integrações. Sem WhatsApp/e-mail direto na tela. É o **destino** de vários fluxos: `disparos.respondentesParaCRM`, `leads-site.rotearSiteLead`, `indiques.enviarNovosAoCrm / criarIndicacao` — todos inserem em `crm_leads` na 1ª etapa do `pipeline='cliente'`. `ia_score` é campo persistido (entrada manual/externa, não há IA de scoring rodando aqui).

6. Estado real: FUNCIONAL. Evidência: leads e etapas persistem de verdade (`crm_leads.insert` com resolução de `empresa_id` pela unidade, `actions.ts:64-76`; `moverLead` faz `update etapa_id`); contagem por etapa é COUNT real; `excluirEtapa` tem guarda de integridade (`actions.ts:135-139`). Board respeita `pipeline='cliente'` para não misturar com Expansão.

7. Requisitos p/ 100%. Detalhe/edição do lead (histórico, atividades, conversão em cliente/venda); filtros/busca no board; disparo de WhatsApp a partir do card; scoring IA real (hoje `ia_score` é campo passivo).

8. Esforço p/ 100%: ~1,5 dev-days.

4) Leads do Site — `/leads-site`

1. Rota/perm. `/leads-site` · perm: `'crm.lead'`. **Vê:** = CRM (`crm.lead` + admin). Roteamento exige sessão autenticada. Arquivos: `src/app/(app)/leads-site/page.tsx`, `.../actions.ts`, `src/lib/supabase/site.ts` (`siteClient`), `src/lib/sac-ingest.ts`.

2. O que faz. Caixa de entrada dos formulários do site institucional (Laser Company): lista os leads captados e roteia cada um para o destino correto — CRM (comercial), SAC (atendimento, franqueadora) ou RH (currículo → banco de talentos).

3. Telas/abas/modais. 1 tela — lista de leads do site com tipo/nome/contato/mensagem; ação **Rotear** (escolhe unidade de destino → CRM/SAC/RH). Origem exibida a partir de `lasercompany_leads` (site) ou `site_leads` (fallback).

4. Backend. Ponte de dois bancos: fonte primária `siteClient()` → `lasercompany_leads` (Supabase do SITE, `riut...`), com **fallback** para `site_leads` (lkii) quando o cliente do site não está configurado. Destinos: `crm_leads` (pipeline cliente, 1ª etapa), `sac_tickets` (empresa franqueadora, `unidade_id=null`), `candidatos` (+ garante `vagas` "Banco de Talentos (Site)"). Action: `rotearSiteLead(siteLeadId, unidadeId)` — parseia o payload, decide destino por `tipo` (`currículo` → RH, `sac` → SAC, demais → CRM), insere no destino e marca o lead como roteado (`_roteado / status='roteado'`).

5. Integrações. Ponte com o SITE = REAL. `siteClient()` conecta ao Supabase externo do site e lê/atualiza `lasercompany_leads` (`actions.ts:21-58`); se ausente, degrada para `site_leads` no lkii. **Importante:** o fluxo SAC normal é ingerido **automaticamente** por `src/lib/sac-ingest.ts` (rota `api/cron/ingest-sac`); esta tela é o caminho **manual** de triagem. Não há WhatsApp/e-mail no roteamento.

6. Estado real: FUNCIONAL. Evidência: roteamento persiste no destino certo com escopo correto — currículo cria `candidatos` sob a vaga guarda-chuva (`actions.ts:63-83`); SAC vai para a franqueadora (`FRANQUEADORA_EMPRESA_ID`, `unidade_id=null`,

`actions.ts:89-101`); CRM insere na etapa inicial do `pipeline='cliente'` (`actions.ts:116-127`); marca `_roteado` para evitar duplicidade (`jaRoteado`). Depende de a env do `siteClient` estar configurada em prod (senão opera só o fallback `site_leads`).

7. Requisitos p/ 100%. Confirmar credenciais do `siteClient` em produção; filtros/busca e status na lista; roteamento em lote; deduplicação por telefone/e-mail; auditoria do roteamento.

8. Esforço p/ 100%: ~1,5 dev-days.

5) Canais — `/canais`

1. Rota/perm. `/canais` · `perm: 'marketing.'` (prefixo). **Vê:** `admin_geral` + cargos com qualquer recurso `marketing.*` (`super_admin`, `administrador`, `marketing`, `diretor`, `auditor`, `expansão-leitura`). **Opera canal:**

`PAPEIS_CANAL=['gestor','operacoes','sac']` + `admin` (`canais/actions.ts:13`), com guarda de escopo (gestor só canal geral ou da própria unidade ativa; `admin` qualquer; `SAC` só canais `geral`). Arquivos: `src/app/(app)/canais/page.tsx`, `.../actions.ts`, `src/lib/uazapi.ts`, `src/components/canais/CanaisManager.tsx`.

2. O que faz. Central de números WhatsApp da rede: cria/vincula instâncias Uazapi por unidade ou "geral" (franqueadora), conecta via QR, monitora status e saúde de envio, e configura o webhook que faz as mensagens caírem na Triagem/IA.

3. Telas/abas/modais. 1 tela (`CanaisManager`) — lista de canais (`/laser/i`) com status (`connected/...`), owner (número), escopo/unidade/atendente, rótulo, delay min/max e **selo de restrição** (número novo sob timelock 463). Ações: **Criar canal**, **Vincular/editar** (escopo unidade/geral, delay, atendente), **Conectar** (modal QR com polling), **Sincronizar** (reaplica webhook), **Desconectar**, **Excluir**.

4. Backend. Tabela `canais_whatsapp` (`instancia_nome`, `escopo[unidade/geral]`, `unidade_id`, `atendente_id`, `rotulo`, `delay_min`, `delay_max`, `criado_por`). As instâncias em si vivem na Uazapi (não no Supabase). Escrita em `canais_whatsapp` usa **service-role** (`adminClient`) porque a RLS não libera `SAC`, mas a autorização já foi feita por papel+escopo. Actions: `criarCanal`, `salvarVinculo`, `conectarCanal`, `statusCanal`, `sincronizarCanal`, `desconectarCanal`, `excluirCanal`.

5. Integrações. 100% REAL Uazapi. `createInstance` (`/instance/create`, `admintoken`), `connectInstance` (`/instance/connect`, gera QR data-URL), `getStatus` (`/instance/status`), `disconnectInstance`, `deleteInstance`, `configurarWebhook` (`/webhook`, eventos `messages/messages_update/connection`, `excludeMessages:['wasSentByApi']` p/ evitar loop com a IA), `limitesEnvio` (`/instance/wa_messages_limits` → detecta restrição 463 de número novo). `urlWebhook()` força domínio público (nunca localhost) com `?secret=`. Verificado: `uazapiConfigurado()` true (envs presentes).

6. Estado real: FUNCIONAL (a mais madura do cluster WhatsApp). Evidência: todo o ciclo de vida do número é operado ao vivo contra a Uazapi (`criar`→`QR`→`conectar`→`status`→`sincronizar`→`desconectar`→`excluir`); saúde de envio real (`limitesEnvio` pinta o selo de restrição). Guarda de escopo por unidade/papel implementada e testável.

7. Requisitos p/ 100%. Nada estrutural — depende de operação (aparelhos/números conectados) e da liberação do timelock 463 dos números novos. Melhorias: histórico de conexões, alerta proativo de queda.

8. Esforço p/ 100%: ~0,5 dev-day.

6) Gestão de Indiques — `/indiques`

1. Rota/perm. `/indiques` · `perm: 'crm.lead'`. **Vê:** = CRM (`crm.lead` + `admin`). **Escreve indicação/status/sorteio:** qualquer sessão; **prêmio/meta do mês:** só `admin_geral` (`actions.ts:170`). Arquivos: `src/app/(app)/indiques/page.tsx`, `.../actions.ts`, `src/lib/indiques.ts`.

2. O que faz. Programa de indicações "Indique e Ganhe": registra indicador + 3 a 5 indicados, empurra cada indicado como lead no CRM, acompanha o Kanban de status, define prêmio/meta mensal e sorteia o ganhador do mês.

3. Telas/abas/modais. 1 tela — KPIs do mês (indiques no mês, % da meta, projeção), Kanban de indicados (`IND_STATUS`: `Novo`/`Em contato/...`), card **Prêmio & Meta do mês**, card **Sorteio** (registrar ganhador / notificar). Modais: **Nova indicação** (indicador + lista 3–5), **Definir prêmio**, **Registrar sorteio**. Janela sempre do MÊS ATUAL por `criado_em` (`indicacoes` não tem `mes_ref` — `page.tsx:15-18`).

4. Backend. Tabelas: `indicacoes` (`indicador_nome/telefone/email/cpf`, `origem[balcao/site/link]`, `premio_descricao`, `qtd_indicados`, `status`), `indicacao_indicados` (`nome`, `telefone`, `email`, `status[pendente...comprou/desistiu]` + timestamps por status), `indique_config` (`premio`, `valor_ref`, `observacao`, `meta_mensal`, `mes_ref`, `upsert` por empresa/unidade/mês), `indique_sorteios` (`ganhador_nome/whats/email`, `premio`, `notificado`), `crm_leads` / `crm_etapas`, `unidades`, `perfis_usuario`. Actions: `criarIndicacao` (valida 3–5, cria indicação+indicados e leads no CRM), `setStatusIndicado`, `enviarNovosAoCrm`, `salvarPremio` (admin), `registrarSorteio`, `notificarGanhador`.

5. Integrações. Geração de leads no CRM = REAL; notificação = STUB. `criarIndicacao` e `enviarNovosAoCrm` inserem de verdade em `crm_leads` (origem `indicacao`, temperatura `morno`, pipeline cliente — helper `criarLeadsCrmDeIndicados`, `actions.ts:141-160`). **notificarGanhador** é flag apenas: apesar do comentário "*notificado (e-mail + WhatsApp)*", o corpo só faz

`update({ notificado: true })` (`actions.ts:239-247`) — não envia WhatsApp nem e-mail. `link_publico` de origem "link"/"site" não tem página pública de captação.

6. Estado real: FUNCIONAL (com notificação stub). Evidência de real: indicação + indicados persistem e viram leads no CRM (`revalidatePath('/crm')`); status com carimbo temporal; prêmio/meta upsert por mês; sorteio grava ganhador. Evidência de stub: `notificarGanhador` só marca booleano. Degrada com instrução clara se `indiques.sql` não aplicada (`actions.ts:62-63,195,231`).

7. Requisitos p/ 100%. Envio real ao ganhador (Uazapi/e-mail); página pública de indicação por link (origem `link/site`); sorteio automático a partir do pool do mês; recompensa/crédito ao indicador quando o indicado "comprou".

8. Esforço p/ 100%: ~2 dev-days.

7) Recursos Humanos — grupo (`/ponto` + 8 folhas `/rh/*`)

Grupo de menu `perm: 'rh.'` (`src/lib/menu.ts:123-135`). **Vê o grupo:** `admin_geral` + cargos com `rh.*` (`super_admin`, `administrador`, `rh`, `diretor`, `franqueado-ler`, `gerente_unidade-ler`, `auditor`). **Detalhe RBAC:** só `Ponto Digital` tem `perm` próprio (`rh.ponto`); as outras 8 folhas **não têm** `perm` → `canSee` cai em `return true` (`Sidebar.tsx:49`): uma vez que o grupo abre, as 8 aparecem para quem vê o grupo (sem recorte fino por sub-tela). **Migration:** `scripts/migrations/rh.sql` (degrade gracioso com banner se ausente). **Modelo pessoas:** `perfis_usuario` (`login+papel`, `id = auth.user.id`) ↔ `colaboradores` (`perfil_id`).

7.1 Ponto Digital — `/ponto`

1. `perm='rh.ponto'` · badge GPS. Vê quem tem `rh.ponto`. Escreve: `registrarPonto` (sessão), `salvarPontoConfig` (só admin), `criarAjustePonto` / `editarPonto` (`PAPEIS_GESTAO=['admin_geral','gestor','gerente','recepcao','rh']`). **2.** Registro de ponto por geolocalização com cerca virtual (Haversine) contra unidade OU casa (home office); gestão vê o espelho e ajusta marcações. **3.** 1 tela: card "Meu ponto" (toggle Presencial↔Home office, 4 botões `PONTO_TIPOS`), 4 KPIs, painel Mapa (Google Maps se `maps_key`, senão OSM), painel Config (admin), filtros GET, tabela paginada; **2 modais** (marcação manual, ajuste). **4.** Tabelas `registros_ponto` (tipo, data_hora, lat, lng, distancia_m, modo, validado_geo, fonte, ajustado_por, motivo_ajuste), `ponto_config` (raio, uni_lat, uni_lng, maps_key, modo_padrao), `colaboradores`. Helpers `haversine` / `dentroDaCerca` (`src/lib/rh.ts:35-52`, raio default 150 m). **5. GPS = REAL** (`navigator.geolocation.getCurrentPosition`; Haversine server-side; GPS da casa em `localStorage`); Google Maps embed real-opcional; sem GPS grava `fonte='manual'`. Sem cron/BEMP. **6. FUNCIONAL** — `registros_ponto.insert({... validado_geo, distancia_m ...})`; não-gestão só vê o próprio ponto. **7.** Aplicar `rh.sql` em prod; export do espelho; RBAC granular das 4 ações. **8. 2 dias.**

7.2 RH · Dashboard — `/rh`

1. Sem `perm` próprio (herda `rh.`); read-only. **2.** Painel inicial: KPIs (ativos, pendências, vagas abertas, departamentos), colaboradores por departamento, atalhos. **3.** 1 tela, sem abas/modais. **4.** COUNT reais em `colaboradores`, `solicitacoes_ferias` (pendentes), `atestados` (pendentes), `vagas` (abertas); escopo por `unidade_id` / `colabIds`; helper `safe()` → 0 se migration ausente. **5.** Nenhuma. **6. FUNCIONAL** — números por agregação real; empty-state honesto. **7.** Tendência/headcount histórico; aniversariantes/admissões do mês. **8. 0,5 dia.**

7.3 RH · Colaboradores — `/rh/colaboradores`

1. Herda `rh.`; usa `criarColaborador` / `checarCpfDuplicado` de `/colaboradores/actions.ts` (`PAPEIS_ESCRITA=['admin_geral','gerente','recepcao','gestor','rh']`). **2.** Lista/busca/filtra colaboradores e abre admissão completa (~30 campos) na MESMA tabela `colaboradores` (sem duplicação). **3.** 1 tela: filtros + tabela paginada (25/pág) + **modal NovoColaborador** (pessoais/vínculo/financeiro/endereço/home office); detalhe `ColaboradorFicha`. **4.** `colaboradores` SELECT `count:'exact'` com filtros e busca `.or()`; escrita compartilhada com CPF-dedup. **5.** Nenhuma externa (liga ao modelo pessoas via `perfil_id`). **6. FUNCIONAL** — query paginada + escrita real. **7.** Edição/inativação inline; upload de documentos; export. **8. 1 dia.**

7.4 RH · Ponto (Jornada) — `/rh/ponto`

1. Herda `rh.`; read-only (marcação vive em `/ponto`). **2.** Espelho de jornada da semana + banco de horas a partir de `registros_ponto` reais. **3.** 1 tela: 3 KPIs (carga prevista, trabalhadas, banco), tabela colaborador×7 dias + Total + Saldo. **4.** `colaboradores` + `registros_ponto` da semana; cálculo `horasNoDia()`. Sem escrita. **5.** Nenhuma. **6. PARCIAL** — lê marcações reais e calcula, MAS `const HORAS_DIA = 8` é **fixo** (`page.tsx:11`), ignorando `colaboradores.jornada_diaria_horas` real; sem feriados/faltas. **7.** Usar jornada real do cadastro; feriados; fechamento mensal; export. **8. 1,5 dia.**

7.5 RH · Recrutamento — /rh/recrutamento

1. Herda `rh.`; escrita só por sessão (sem gate de papel). 2. Banco de talentos + processo seletivo Kanban (dnd) por estágios, com WhatsApp de disponibilidade. 3. **2 abas** (Currículos, Kanban 7 colunas) + **2 modais** (Novo currículo, Notas+score). 4. `candidatos` (embed `vagas!inner(...)`, escopo por `vagas.unidade_id`), `vagas`. Actions: `moverCandidato`, `iniciarProcesso`, `atualizarNotas`, `criarCurrículo`, `avisarDisponibilidade`, `definirScore`. 5. **WhatsApp Uazapi = REAL** em `avisarDisponibilidade` (`sendText(canal.token,...)`, `actions.ts:113`); `iniciarProcesso` só grava nota (envio automático NÃO dispara); `score_triagem_ia` é **manual** (não há IA). 6. **FUNCIONAL** — CRUD + Kanban dnd otimista com rollback; cria vaga guarda-chuva se faltar. 7. Disparo automático ao mover estágio; triagem IA real; anexo de currículo; RBAC por papel. 8. **1,5 dia**.

7.6 RH · Folha de Pagamento — /rh/folha

1. Herda `rh.`; `PAPEIS_FOLHA=['admin_geral','gestor','financeiro','rh']`. 2. Gera/fecha/paga folha por competência calculando INSS/IRRF/FGTS/13º/líquido do salário bruto; holerite. 3. 1 tela: seletor de competência, "Gerar folha"/"Gerar com 13º", 6 KPIs, tabela status (aberta→fechada→paga); **1 modal Holerite**. 4. `folha_pagamento` (competencia, salario_bruto, inss, irrf, fgts, outros_, `decimo_terceiro`, `salario_liquido`, `status`), `colaboradores`. `gerarFolha` *upsert idempotente* `onConflict:'colaborador_id,competencia'` (não sobrescreve fechada/paga); `alterarStatusFolha`. 5. **Cálculo fiscal = REAL** (tabelas progressivas INSS/IRRF 2025 + FGTS 8% em `src/lib/rh.ts:80-147`); sem eSocial/banco. 6. **FUNCIONAL** (amplitude parcial) — *persiste e respeita travas*; MAS `outros_proventos/descontos` fixados em 0 (sem UI de lançamentos avulsos); holerite sem export/PDF; 13º integral. 7. UI de proventos/descontos avulsos; PDF do holerite; 13º proporcional; eSocial; remessa bancária. 8. **2 dias.***

7.7 RH · Férias e Ausências — /rh/ferias

1. Herda `rh.`; `PAPEIS_APROVA=['gestor','gerente','rh']` +admin; colaborador comum só lança para si. 2. Solicitações de férias (período aquisitivo, abono ≤10 dias, aprovação) e atestados (CID, afastamento). 3. **2 abas** (Férias, Atestados) + 4 KPIs + **3 modais** (FeriasForm, AtestadoForm, RecusaForm). 4. `solicitacoes_ferias` (periodo_aquisitivo, datas, dias, vender_dias, status, aprovado_por), `atestados` (data_inicio, dias, cid, status); escopo por `colabIds`. Actions: `solicitarFerias` (valida ≤30d, abono ≤10 — CLT), `decidirFerias`, `registrarAtestado`, `decidirAtestado`. 5. Nenhuma. 6. **FUNCIONAL** — *persiste, valida CLT no servidor, só decide pendentes, auto-scope*. 7. Cálculo automático de período aquisitivo/saldo; alerta de vencimento; anexo do atestado; notificação na decisão. 8. **1 dia**.

7.8 RH · Desempenho — /rh/desempenho

1. Herda `rh.`; `PAPEIS_ESCRITA=['gestor','gerente','rh']` +admin. 2. Avaliações (5 notas 0–5), PDIs (com progresso) e resumo de metas por colaborador. 3. **3 abas** (Avaliações, PDI, Metas) + KPIs + **2 modais** (Avaliação, PDI). Metas é leitura (CRUD em `/cadastros/metas`). 4. `avaliacoes_desempenho` (notas produtividade/qualidade/comportamento/equipe/geral), `pdi` (titulo, prazo, status, progresso), `metas_colaborador` (leitura); escopo `colabIds`. Actions: `criarAvaliacao` / `salvar` / `excluir`, `criarPdi` / `salvar` / `atualizarProgressoPdi` / `excluir`. 5. Nenhuma. 6. **FUNCIONAL** — CRUD completo real; nota geral auto-média; PDI→'concluido' a 100%. 7. Ciclos formais trimestrais; autoavaliação; gráfico de evolução; CRUD de metas na aba. 8. **1 dia**.

7.9 RH · Regras da Rede — /rh/regras

1. Herda `rh.`; sem actions. 2. Exibe as 10 regras/políticas da rede (conduta) em accordion com busca/filtro. 3. 1 tela, sem modais: busca + categoria + accordion (r1..r10, pílula Obrigatório/Importante/Recomendado). 4. **NENHUM backend** — conteúdo estático em `src/lib/rh.ts:161-263` (`REGRAS_REDE`). 5. Nenhuma. 6. **FUNCIONAL (conteúdo estático por design)** — busca/filtro/accordion operantes; não persiste porque não precisa. 7. Se regras editáveis pela franqueadora → tabela+editor (~1,5 dia); aceite "li e concordo" por colaborador se exigido. 8. **0,25 dia**.

8) Marketing — /marketing

1. **Rota/perm.** `/marketing` · `perm: 'marketing.'` **Vê:** `admin_geral` + `super_admin`, administrador, marketing, diretor, auditor, expansão-leitura. **Escreve campanhas:** `PAPEIS_ESCRITA` (`gestor/operacoes/marketing`)+admin; **publicar notícia:** só admin. Arquivos: `src/app/(app)/marketing/page.tsx`, `.../actions.ts`, `src/lib/marketing.ts`.

2. **O que faz.** Une numa tela (a) Central de Materiais da Rede (atualizações/materiais/notícias da franqueadora) e (b) Campanhas de WhatsApp por unidade (CRUD segmentado).

3. **Telas/abas/modais.** `MarketingManager` — **4 abas:** Atualizações (marca lidas no mount), Materiais (árvore breadcrumb, Baixar/Canva), Notícias (**modal Publicar** — admin), Campanhas (`CampanhasWhatsapp`: KPIs, filtros, tabela, **modal criar/editar**, cancelar). Banner `migrationPendente` se `mkt_*` ausentes.

- 4. Backend.** `mkt_atualizacoes`, `mkt_noticias`, `mkt_materiais` (kind/parent_id/link_url/ordem), `campanhas_whatsapp` (mensagem_base, segmentacao_tipo, status, `total_enviados/entregues/lidos/responderam/falhou`, `ia_personalizar/ia_instrucao`), `whatsapp_templates`, `unidades / empresas / perfis_usuario`. Actions: `criarCampanha`, `atualizarCampanha`, `cancelarCampanha`, `publicarNoticia` (admin), `marcarAtualizacoesLidas`.
- 5. Integrações.** **Campanhas WhatsApp = STUB; materiais/notícias = REAL parcial.** `CampanhasWhatsapp` importa só `criarCampanha/atualizar/cancelar` — **nenhum** `sendText` / `criarCampanhaSimples`. `criarCampanha` faz **só INSERT** em `campanhas_whatsapp` (`actions.ts:92`); **não há worker de disparo** → `total_enviados/entregues/lidos` **nunca populam** (KPIs de entrega sempre 0). `campanha_destinatarios` é citada em comentário mas nunca lida/escrita. Materiais/Notícias = leitura real; `publicarNoticia` grava.
- 6. Estado real: PARCIAL.** Materiais/Notícias/Atualizações FUNCIONAL (persiste, marca lido). Campanhas = CRUD-fantasma: cria/edita no banco mas **não dispara nada** e métricas não se movem.
- 7. Requisitos p/ 100%.** Worker de disparo (materializar `campanha_destinatarios` da segmentação — aniversariantes = query na base — chamar `uazapi.criarCampanhaSimples`, avançar status, ligar webhook `messages_update` às métricas); CRUD/upload de materiais; cron para agendadas.
- 8. Esforço p/ 100%: ~5 dev-days.**

9) Comunicados — `/comunicados`

- 1. Rota/perm.** `/comunicados` · `perm: 'operacoes.'` · **Vê:** `admin_geral` + `super_admin`, administrador, operacoes, franqueado, gerente_unidade, supervisor, técnico, diretor, auditor. **Publicar:** só `admin_geral`. Arquivos: `src/app/(app)/comunicados/page.tsx`, `.../actions.ts`.
- 2. O que faz.** Mural de avisos internos da franqueadora para a rede, com leitura obrigatória ("ciente") e relatório de quem leu.
- 3. Telas/abas/modais.** `ComunicadosManager` — KPIs, barra Cientes×Pendentes, **4 abas** (Todos/Publicados/Agendados/Encerrados), tabela; **modal Novo** (admin); **PreviewModal/relatório de leitura** (roster); **CienteModal/ ComunicadosGate** (gate global que força ciente em comunicado obrigatório). Ações admin: Encerrar/Reabrir/Link.
- 4. Backend.** `comunicados` (titulo, mensagem, prioridade, categoria, `audiencia[]`, `leitura_obrigatoria`, `enviar_email`, status, `total_destinatarios`, `publicado_em`, `agendado_para`, `autor_*`), `comunicado_leituras` (`comunicado_id`, `perfil_id`, `ciente`, `lido_em`, `unidade_id`), `perfis_usuario` (pool = ativos via `adminClient`), `unidades`. Actions: `criarComunicado` (só admin), `marcarCiente` (upsert ON CONFLICT DO NOTHING), `relatorioLeitura`, `rosterLeitura`, `definirStatusComunicado`.
- 5. Integrações. STUB.** O campo `enviar_email` é rotulado no UI como *"Enviar também por WhatsApp"* e vira ícone verde, mas é **apenas boolean persistido** — sem Uazapi, sem e-mail. `agendado_para` grava mas **não há cron** que publique no horário (fica `agendado` até alguém mudar).
- 6. Estado real: FUNCIONAL (com ressalvas).** CRUD + ciente + roster/relatório persistem; contagens coerentes (`dest` = ativos, `lidos` filtrados por ativos). Ressalvas: envio WhatsApp/e-mail é flag decorativa; agendamento não auto-publica.
- 7. Requisitos p/ 100%.** Disparo real (Uazapi/e-mail) quando `enviar_email`; cron para publicar agendados; filtro real por `audiencia` (hoje pool = todos ativos).
- 8. Esforço p/ 100%: ~2 dev-days.**

10) Chamados — `/chamados`

- 1. Rota/perm.** `/chamados` · `perm: 'operacoes.'` (= Comunicados). Abrir/responder: qualquer sessão; Assumir: só admin. **Distinto do SAC** (`/sac/chamados`, `perm sac.`, WhatsApp, centralizado): aqui é **suporte interno** franquia⇌franqueadora, sem cliente final nem WhatsApp. Arquivos: `src/app/(app)/chamados/page.tsx`, `.../actions.ts`.
- 2. O que faz.** Tickets de suporte entre unidades e a franqueadora, com thread de mensagens, atribuição de responsável e finalização.
- 3. Telas/abas/modais.** `ChamadosManager` — KPIs, **2 abas** (Recebidos/Enviados), tabela; **modal Abrir** (para não-admin o campo "De" vem travado como `Franqueado · <unidade>`); painel **detalhe/thread** com chat, botões Assumir (admin) e Finalizar/Reabrir. Classificação franqueadora-cêntrica: `de_parte ~ /franquead/i` → box Recebidos.
- 4. Backend.** `chamados` (`numero`, `assunto`, `etiqueta`, `de_parte/para_parte`, `de_unidade_id`, `prioridade`, `responsavel_`, `aberto_por_`, `finalizado`, `finalizado_em`), `chamado_mensagens` (`papel_remetente_solicitante/responsavel`, `mensagem`), `perfis_usuario`, `unidades`. Escopo por `de_unidade_id = activeUnitId`. Actions: `abrirChamado`, `carregarThread`, `responderChamado`, `assumirChamado`, `finalizarChamado`.
- 5. Integrações. Nenhuma (in-app puro).** Sem Uazapi/e-mail/cron.

6. Estado real: **FUNCIONAL (a mais madura das telas de operação)**. Fluxo completo persiste: abrir→thread→assumir→responder→finalizar/reabrir, todos com `revalidatePath('/chamados')`. Sem mocks.
7. Requisitos p/ 100%. Notificação (sino/e-mail/WhatsApp) ao responsável/solicitante; anexos; SLA/prazo; gate de escrita por papel (hoje aberto a todos os logados).
8. Esforço p/ 100%: ~1,5 dev-days.

11) Checklist de Indicadores — `/checklist`

1. Rota/perm. `/checklist` · perm: `'operacoes.'` (= Comunicados/Chamados). **Escreve:** só `gestor` + admin (`PAPEIS_ESCRITA=['gestor']`, `actions.ts:22`). Arquivos: `src/app/(app)/checklist/page.tsx`, `.../actions.ts`, `src/lib/checklist.ts`.
2. O que faz. Avalia os indicadores do funil da unidade (nota 0–10 vs meta), monta um checklist mensal modelo SULTS (6 seções, 340 pts) e gerencia Planos de Ação PDCA com tarefas.
3. Telas/abas/modais. `CheckListView` — 4 KPI cards, **3 abas** (Avaliação, Mensal, Planos); Avaliação = tabela dos 7 indicadores nota/status + CTA "gere um plano"; Mensal = seções SULTS + pontuação; Planos = lista; **modal PlanoModal** (pré-preenchido com gargalos).
4. Backend. `kpis_unidade_snapshot` (agendamentos_total, taxa_comparecimento/conversao, ticket_medio, data_referencia, periodo — lido escopado + via service-role p/ média da rede), `planos_acao` (semana_inicio/fim, status, prioridade, resumo, cumprimento_pct), `plano_acao_tarefas` (titulo, categoria, ordem, prazo_dias, concluida). Lib pura: `avaliarFunil / notaIndicador / montarChecklistMensal / FUNIL_INDS`. Actions: `criarPlano` (insert plano→tarefas com rollback de órfão), `toggleTarefa` (recalcula `cumprimento_pct` server-side), `definirStatusPlano`.
5. Integrações. Nenhuma; sem **BEMP/cron**. Snapshots são **consumidos**, não coletados aqui. TODO confirma que coleta semanal automática por unidade (`pg_cron`) e geração automática de planos **não existem** (`actions.ts:181-184`).
6. Estado real: **PARCIAL**. FUNCIONAL: avaliação em tempo real, planos+tarefas persistem, cálculo de %. **Não-persistente/ heurístico:** a aba Mensal SULTS é calculada on-the-fly e NUNCA gravada — `sults_checklist_avaliacoes` "existe mas está vazia/sem colunas confirmadas" (`actions.ts:185-187`); vários itens são derivações sintéticas de UM snapshot (ex.: `novos = max(5, round(conv*0.38))`, `contratos pendentes = ag % 2`, tendência de 3 meses inferida de 1 ponto — `src/lib/checklist.ts:246-267`). Sem snapshot da unidade → aba mensal em empty-state.
7. Requisitos p/ 100%. Persistir a avaliação mensal (`sults_checklist_avaliacoes`); cron de coleta de KPIs por unidade; geração automática de plano quando indicador < 7; séries temporais reais; chat/comentários no plano.
8. Esforço p/ 100%: ~3 dev-days.

12) Universidade Corporativa — `/universidade`

1. Rota/perm. `/universidade` · perm: `'treinamento.curso'` (exato). **Vê:** `admin_geral` + `super_admin`, administrador, diretor, **rh**, auditor, franqueado. **Não vê:** `operacoes`, financeiro, marketing, sac, comercial, técnico, expansão, TI, jurídico. **Escrita de trilhas/ etapas:** só `admin_geral` (`ehAdmin`); prova: qualquer autenticado. Arquivos: `src/app/(app)/universidade/page.tsx`, `.../actions.ts`, `src/components/universidade/UniversidadeManager.tsx`.
2. O que faz. EAD interno: trilhas de vídeo por cargo (links não-listados do YouTube) com prova por etapa e prova final que "libera o certificado"; provas corrigidas no servidor, notas/progresso persistidos por usuário.
3. Telas/abas/modais. **4 abas** (`UniversidadeManager:29-34`): Trilhas (cards→detalhe com etapas/vídeo/prova + final travada), Alunos & Notas (KPIs+tabela+"Gerar certificado"), Dashboards (KPIs+3 gráficos), Gerenciar (só admin: editor inline de etapas). **Modal QuizModal** (prova). Certificado = HTML gerado no browser.
4. Backend. `uni_trilhas` (slug, nome, role, cor, prazo, ordem), `uni_etapas` (trilha_id, ordem, nome, `yt`, min, `prova` jsonb, `is_final`), `uni_progresso` (trilha_id, perfil_id, etapa_key, concluido, nota; unique). Actions: `submeterProva` (corrige, aprova ≥7,0, valida pré-requisito da final), `criarTrilha / salvar / excluir`, `adicionarEtapa / salvarEtapa / excluirEtapa` (**todas só admin**).
5. Integrações. Vídeos = YouTube externo (não armazenado) — `ytUrl(e.yt)` monta link não-listado; sem storage/streaming. Dados = Supabase REAL. Certificado = **STUB client-side** (`gerarCertificado`, `UniversidadeManager:514-554`): monta HTML, `Blob` + `createObjectURL`, download `.html / print()` — sem lib de PDF, sem persistência, código de validação é hash local não gravado. E-mail: nenhum.
6. Estado real: **FUNCIONAL (com ressalvas)**. Correção e gravação de nota reais (`upsert({... concluido:aprovado, nota ...})`, `actions.ts:80-89`). Ressalvas: `prazo` é texto e status "No prazo" é fixo (`page.tsx:87`) → KPI "Atrasados" nunca dispara; certificado não é PDF real nem registrado; trilha↔cargo é rótulo (`role` texto), sem atribuição automática.

7. Requisitos p/ 100%. Certificado em PDF real + registro auditável (código verificável); lógica real de prazo; vincular **role** ao cargo; opcional: hospedar vídeo próprio em Storage.

8. Esforço p/ 100%: ~3 dev-days.

13) Disco Virtual — **/disco**

1. Rota/perm. **/disco** · **perm:** 'operacoes.'. **Vê:** admin_geral + super_admin, administrador, diretor, operacoes, auditor, franqueado, gerente_unidade, supervisor, técnico. **Escrita (criar/enviar/excluir/vincular):** só **admin_geral**; **urlArquivo** (baixar): todos. Arquivos: **src/app/(app)/disco/page.tsx**, **.../actions.ts**, **src/components/disco/DiscoManager.tsx**.

2. O que faz. "Drive da rede": pastas hierárquicas + arquivos com upload/download/exclusão; vínculo opcional (decorativo) com Google Drive.

3. Telas/abas/modais. Tela única: banner Google Drive (conectado/"Vincular"), toolbar breadcrumb+busca+Nova pasta/Enviar (admin), grade de pastas, tabela de arquivos (Baixar/Excluir), empty-state. Sem modais formais (usa **prompt / confirm / input file**).

4. Backend. **disco_config** (empresa_id, drive_linked, drive_url), **disco_pastas** (parent_id CASCADE, nome, por, **drive**), **disco_arquivos** (pasta_id CASCADE, nome, tipo, bytes, **arquivo_path**, por, **drive**). **Bucket Storage disco-virtual (PRIVADO).** Actions: **novaPasta / excluirPasta / uploadArquivo / excluirArquivo / vincularDrive / desvincularDrive / importarDrive** (todas só admin); **urlArquivo** (todos → signed URL).

5. Integrações. Supabase Storage = REAL. Upload: **data URI → Buffer → sbAdmin.storage.from('disco-virtual').upload(...)**, cap 25 MB (**actions.ts:116-133**); download **createSignedUrl(path, 300)** (5 min); exclusão remove objeto+registro. **Google Drive = STUB (cosmético):** **vincularDrive** só valida **/drive\google\.com/** na string e grava a flag (sem OAuth, sem Drive API); **importarDrive** insere 3 nomes hardcoded (**['Fotos Institucionais', 'Vídeos da Rede', 'Planilhas Financeiras']**), não lê o Drive; o booleano **drive** só pinta ícone "replicado" — nenhuma replicação ocorre. E-mail: nenhum.

6. Estado real: PARCIAL. Núcleo (pastas+arquivos em Storage real) FUNCIONAL; camada Google Drive é **fachada**. Evidência real: **sbAdmin.storage.from(BUCKET).upload(...)**. Evidência stub: **if(!/drive\google\.com/.test(url)) return...** seguido de **upsert({... drive_linked:true ...})** (único efeito).

7. Requisitos p/ 100%. Google Drive real (OAuth + Drive API: listar/importar/replicar) **ou** remover a promessa de replicação e rotular como link externo; opcional: mover/renomear, preview, ACL por pasta. **Atenção:** o bucket **disco-virtual** precisa ser criado manualmente no painel Supabase (não é criado por migration).

8. Esforço p/ 100%: ~4 dev-days (~1 dia se descontinuar a replicação e manter só link).

14) Notas Fiscais — **/notas** (badge "EM BREVE")

1. Rota/perm. **/notas** · **perm:** 'financeiro.' (prefixo), badge **EM BREVE**. **Vê:** admin_geral + super_admin, administrador, diretor, financeiro, jurídico, auditor, franqueado, gerente_unidade. **Escrita:** **ehAdmin(papel) || papel ∈ {'gestor', 'financeiro'}**. Arquivos: **src/app/(app)/notas/page.tsx**, **.../actions.ts**, **src/lib/nfse.ts**, **src/components/notas/NotasView.tsx**.

Reconciliação do badge: **/notas ∈ ROTAS_FUNCIONAIS** mas com badge EM BREVE → o **LeafLink** renderiza selo âmbar "EM BREVE" (não "prévia"): a tela é funcional (política/config/registo reais), mas a **emissão fiscal** é stub — é o que "EM BREVE" comunica.

2. O que faz. Governança de NFS-e da rede: define política de emissão (não emitir / na venda / na execução) + "calcular por sessão"; conecta cada unidade à prefeitura; registra/lista notas com KPIs e filtros; emissão manual e ações de status.

3. Telas/abas/modais. Tela única (**NotasView**): card Política (3 segment-buttons + toggle por sessão), tabela Integração com prefeituras (provedor/alíquota/status/ambiente), 4 KPIs (emitidas/valor/canceladas/processando), Filtros (competência/unidade/tipo/status; **Exportar desabilitado**), tabela Notas. **2 modais:** EmitirModal (NFS-e manual), ConfigUnidadeModal (prefeitura por unidade).

4. Backend. **nfse_politica** (politica, por_sessao), **nfse_config_unidade** (provedor, aliquota_iss, inscricao_municipal, certificado_token, ambiente, status_conexao), **nfse** (numero, competencia, tipo, cliente_nome, valor, status, **xml**). Actions: **definirPolitica**, **definirPorSessao**, **salvarConfigUnidade**, **emitirManual**, **alterarStatusNota** (todas gated por **podeAdministrar**); auditoria best-effort em **audit_log**.

5. Integrações. Emissão fiscal = STUB confirmado — sem API fiscal alguma. Nenhuma chamada a Focus NFe / PlugNotas / eNotas / WebISS / ADN / prefeitura (os nomes de provedor só aparecem como strings de exibição). O código confessa: **"EMIÇÃO FISCAL REAL fica como TODO"** (**actions.ts:14**) e **"aqui registramos a nota como 'processando'"** (**:180-182**). **emitirManual** só faz **insert({... status:'processando' ...})** — **não seta numero nem xml, e nada avança o status** (sem fila/cron/webhook); **alterarStatusNota** muda status **manualmente**. Defaults de provedor/alíquota/conexão são **hashes determinísticos**

falsos (`nfseConectada = hashStr%4!==0` , `nfseAliquota = [2,2.5,3,3.5,4,5][hash%6]` , `src/lib/nfse.ts:55-62`). Coluna `xml` nunca é escrita. Exportar XML/CSV: botões `disabled` . E-mail: nenhum.

6. Estado real: PARCIAL (administração real; emissão FALTA). Política, config por unidade, registro/listagem/filtros/KPIs com counts reais e escopo = FUNCIONAL. Emissão fiscal ausente — `.from('nfse').insert({... valor, status:'processando' ...})` sem `numero/xml` , status só progride por clique humano.

7. Requisitos p/ 100%. Integrar provedor fiscal real (Focus NFe / PlugNotas / ADN): envio do RPS, retorno de `numero+xml+protocolo`, polling/webhook de autorização, cancelamento junto à prefeitura, guarda do XML e exportação; substituir hashes decorativos por status de conexão real; emissão automática conforme política plugada em OS/pagamentos.

8. Esforço p/ 100%: ~12 dev-days.

Tabela-resumo

#	ITEM	ROTA	PERM (MENU)	INTEGRAÇÃO-CHAVE	ESTADO	DIAS P/ 100%
1	Mensagens e Automações	<code>/automacoes</code>	<code>marketing.campanha</code>	Uazapi real, mas nenhum motor executa as automações	PARCIAL (config real, execução FALTA)	10
2	Disparos WhatsApp API	<code>/disparos</code>	<code>marketing.campanha</code>	Envio real via <code>/sender/simple</code> ; métricas entregues/lidas/respostas e vip_grupos.membros NUNCA gravadas	PARCIAL	4
3	CRM	<code>/crm</code>	<code>crm.lead</code>	Destino de leads (disparos/site/indiques); sem envio próprio	FUNCIONAL	1,5
4	Leads do Site	<code>/leads-site</code>	<code>crm.lead</code>	Ponte real com Supabase do site (<code>lasercompany_leads</code>) + fallback	FUNCIONAL	1,5
5	Canais	<code>/canais</code>	<code>marketing.</code>	100% Uazapi real (QR/status/webhook/ limites)	FUNCIONAL	0,5
6	Gestão de Indiques	<code>/indiques</code>	<code>crm.lead</code>	Gera leads reais no CRM; notificarGanhador só flag (sem WhatsApp/e-mail)	FUNCIONAL (notify stub)	2
7.1	RH · Ponto Digital	<code>/ponto</code>	<code>rh.ponto</code>	GPS real (geolocation + Haversine)	FUNCIONAL	2
7.2	RH · Dashboard	<code>/rh</code>	herda <code>rh.</code>	—	FUNCIONAL	0,5
7.3	RH · Colaboradores	<code>/rh/colaboradores</code>	herda <code>rh.</code>	—	FUNCIONAL	1
7.4	RH · Ponto (Jornada)	<code>/rh/ponto</code>	herda <code>rh.</code>	—	PARCIAL (jornada 8h fixa)	1,5
7.5	RH · Recrutamento	<code>/rh/recrutamento</code>	herda <code>rh.</code>	WhatsApp real (avisar disponibilidade); auto-msg/IA stub	FUNCIONAL	1,5
7.6	RH · Folha de Pagamento	<code>/rh/folha</code>	herda <code>rh.</code>	Cálculo fiscal INSS/IRRF/FGTS real; sem eSocial/PDF	FUNCIONAL (amplitude parcial)	2
7.7	RH · Férias e Ausências	<code>/rh/ferias</code>	herda <code>rh.</code>	—	FUNCIONAL	1
7.8	RH · Desempenho	<code>/rh/desempenho</code>	herda <code>rh.</code>	—	FUNCIONAL	1
7.9	RH · Regras da Rede	<code>/rh/regras</code>	herda <code>rh.</code>	conteúdo estático	FUNCIONAL (estático)	0,25
8	Marketing	<code>/marketing</code>	<code>marketing.</code>	Materiais reais; campanhas WhatsApp CRUD sem disparo/métricas	PARCIAL	5
9	Comunicados	<code>/comunicados</code>	<code>operacoes.</code>	"WhatsApp/e-mail" e agendamento = flag decorativa	FUNCIONAL (ressalva)	2
10	Chamados	<code>/chamados</code>	<code>operacoes.</code>	in-app puro (≠ SAC; sem WhatsApp)	FUNCIONAL	1,5
11	Checklist de Indicadores	<code>/checklist</code>	<code>operacoes.</code>	SULTS mensal não-persistido/heurístico ; sem cron de coleta	PARCIAL	3

#	ITEM	ROTA	PERM (MENU)	INTEGRAÇÃO-CHAVE	ESTADO	DIAS P/ 100%
12	Universidade Corporativa	/universidade	treinamento.curso	YouTube externo + Supabase; certificado HTML client-side	FUNCIONAL (certificado/prazo fracos)	3
13	Disco Virtual	/disco	operacoes.	Storage disco-virtual real; Google Drive = STUB	PARCIAL (Drive fachada)	4
14	Notas Fiscais	/notas	financeiro. (EM BREVE)	Emissão fiscal = STUB (sem API); tabelas/CRUD reais	PARCIAL / emissão FALTA	12
	TOTAL					~68,75 dev-days

Contagem de telas/abas/modais (escopo deste módulo): - **Funcionalidades:** 14 (contando RH como grupo) / **24 rotas** (RH = 9 folhas). - **Telas (page.tsx):** 24. - **Abas internas:** Disparos 4 · Recrutamento 2 · Férias 2 · Desempenho 3 · Marketing 4 · Comunicados 4 · Chamados 2 · Checklist 3 · Universidade 4 = **28 abas** (+ toggle Presencial/Home-office no Ponto). - **Modais/painéis:** Automações 3 · Disparos ~3 · CRM 1 · Canais ~5 ações · Indiques 3 · Ponto 2 · Colaboradores 1 · Recrutamento 2 · Folha 1 · Férias 3 · Desempenho 2 · Marketing 2 · Comunicados 3 · Chamados 1+detalhe · Checklist 1 · Universidade 1(Quiz) · Notas 2 = **~38 modais/painéis**.

Achados-chave de homologação: 1. **Automações não executam:** /automacoes persiste toggles/custom/no-show mas **nenhum scheduler dispara** — KPIs de envio honestamente null. 2. **Disparos com métricas cegas:** o envio é real, porém disparo_campanhas.entregues/lidas/respostas e vip_grupos.membros **nunca são gravados por código** (só insert/select/delete) → sempre 0; isso deixa respondentesParaCRM inoperante (lê respostas=0). 3. **Marketing e Comunicados prometem envio via flags/copy sem backend** de disparo — o único envio de campanha real é /disparos (via expansao/disparos). 4. **Integrações realmente reais nesta seção:** Canais (Uazapi completo), Disparos-envio, Recrutamento-WhatsApp, Ponto-GPS, Disco-Storage, Leads-do-Site-ponte, Folha-cálculo-fiscal. 5. **Stubs confirmados:** Automações (execução), NFS-e (emissão fiscal), Google Drive (Disco), notificarGanhador (Indiques), checklist SULTS mensal (não-persistido/heurístico). 6. **Dualidade RBAC:** visibilidade por recursos/cargos ≠ escrita por papel — usuário pode ver a tela e ser barrado nas ações; as 8 folhas RH sem perm próprio não têm recorte fino por sub-tela.

Módulo 4 — Administração (Financeiro Franqueadora, SAC, Expansão, Jurídico, Auditoria)

Documento oficial de homologação. Fonte: leitura direta do código em `src/app/(app)/`, `src/components/`, `src/lib/` e `scripts/migrations/` (não do material de apoio). Hierarquia canônica em `src/lib/menu.ts`, seção `Administração`. Datas de referência das validações citadas nos comentários do código (02–05/07/2026).

Convenções e RBAC (válido para todo o módulo)

- **Estado FUNCIONAL** segue a convenção do projeto: rota presente em `ROTAS_FUNCIONAIS` (`menu.ts`) = tela "acesa" (funcional). Todas as 30 folhas deste módulo estão listadas em `ROTAS_FUNCIONAIS` — a verificação abaixo confirma, folha a folha, se há query/mutação Supabase real ou apenas UI/estado-vazio.
- **Dois níveis de RBAC:** (1) **visibilidade de menu** por `recurso` (`Sidebar.tsx`: `perm` com sufixo `.`, casa por prefixo `recursos.some(startsWith)`, senão match exato); (2) **gate de ação** nas server actions, quase sempre por **papel** via `temPapel(papel, ...aceitos)` (`src/lib/rbac.ts`), onde `admin_geral` (`PAPEL_ADMIN`) **sempre passa**.
- `recursos` derivam de `usuario_cargos` → `cargo_permissoes` → `permissoes` (`session.ts`). `admin_geral` tem `recursos=[]` mas passa por bypass `isAdmin`.
- **Papéis (enum) relevantes:** `admin_geral`, `gestor`, `financeiro`, `sac`. **Cargos SAC:** `atendente_sac`, `supervisor_sac`, `consulta_sac` (resolvem só para recursos `sac.*`).
- **Contagem de telas do módulo:** Financeiro 9 (1 componente `FinanceiroTabs`, 9 abas/rotas) · SAC 11 · Expansão 7 · Implantação 1 · Jurídico 1 · Auditoria 1 = **30 folhas**.

Submódulo A — Financeiro Franqueadora (coração do sistema)

Arquitetura (verificada em `src/lib/financeiro-ledger.ts` + `scripts/migrations/financeiro-razao.sql`). O financeiro é um **serviço central com razão único**. Existe uma **única porta de escrita** — `postLancamento()` / `repostLancamento()` — que grava na tabela-razão `fin_lancamento`. Todas as telas (DRE, Fluxo de Caixa, Contas a Receber/Pagar) **derivam** do razão via RPCs; não guardam valor próprio. Assim "o número bate igual em toda tela".

- **Roteamento:** as 9 rotas (`/financeiro`, `/financeiro/dre`, `/calc`, `/receber`, `/pagar`, `/conciliacao`, `/royalties`, `/cobranca`, `/config`) são **wrappers triviais** que renderizam o mesmo `FinanceiroPage` (`src/app/(app)/financeiro/page.tsx`) com `tab` diferente. A URL é preservada (o item do menu "acende"). O componente cliente é `src/components/financeiro/FinanceiroTabs.tsx` (~135 KB, 9 abas).
- **RBAC:** menu perm `financeiro.`; **gate de página** `temPapel(ctx.papel, 'financeiro', 'gestor')` + bypass `admin_geral`. Papéis com acesso: `admin_geral`, `financeiro`, `gestor`. Todas as server actions repetem o gate `temPapel(op.papel, 'financeiro', 'gestor')` (`PAPEIS_FIN`).
- **Escopo por unidade:** `activeUnitId` filtra; correção do bug 03/07 — `activeUnitName` nunca é null ("Todas as unidades"), então o nome só é usado como filtro (`fin_contas_pagar.escopo`, `fin_conciliacao.unidade_nome`) quando há `activeUnitId` real.

Tabelas Supabase (migration `financeiro-razao.sql`)

TABELA	PAPEL
<code>fin_lancamento</code>	Razão único (fonte da verdade). Colunas: <code>natureza</code> (receita/despesa/transferencia), <code>competencia</code> (1º dia do mês do fato), <code>data_prevista</code> , <code>data_caixa</code> , <code>valor</code> , <code>origem</code> (bemp/royalty/sac/folha/compra/taxa_cartao/manual/despesa_config), <code>origem_ref</code> , <code>idem_key</code> (única), <code>status</code> (previsto/realizado/conciliado/cancelado/suspensao), <code>centro_custo_id</code> , <code>plano_conta_id</code> . Índice único em <code>idem_key</code> (idempotência). RLS: leitura autenticado; escrita só via service role (<code>adminClient</code>).
<code>centro_custo</code>	Um por unidade + um <code>tipo='rede'</code> (Rede/Franqueadora).
<code>plano_conta</code>	Plano de contas (DRE). Seed curado: receitas 3.1.01–3.1.06, custos/despesas 4.1.01–4.1.05, 4.2.01–4.2.05, 4.2.99. <code>codigo</code> = conta do sistema (protegida); categorias criadas pelo usuário têm <code>codigo=null</code> .
<code>fin_recebiveis</code>	Sub-livro "A Receber" (Royalties, Fundo, etc.). Campo <code>lançamento_id</code> liga ao razão (a baixa concilia o caixa).
<code>fin_contas_pagar</code>	Sub-livro "A Pagar" (despesas manuais).
<code>fin_conciliacao</code>	Extrato importado × esperado × recebido.

TABELA	PAPEL
<code>fin_config</code>	Parâmetros (royalty %, fundo %, venc_dia, imposto/comissão/taxa, desconto automático, banco, adquirentes, categorias, régua). Upsert por <code>empresa_id</code> .
<code>unidades</code> (colunas add)	<code>royalty_pct_override</code> , <code>venc_dia_override</code> , <code>tipo_loja</code> ('propria' 'franquia'), <code>bemp_salon_id</code> .

RPCs (todas em `financeiro-razao.sql`)

- `fin_faturamento_por_salon(ini,fim)` — faturamento real do BEMP por salon = `sum(total - desconto)` de `bemp_billings` (base líquida, definição do CEO 02/07).
- `fin_faturamento_por_salon_entidade(ini,fim)` — idem, quebrado por `entity` (packages/services/products/subscriptions) → conta de receita.
- `fin_ultima_competencia()` — default do DRE/Fluxo. Retorna o último mês **com RECEITA** apurada `<= mês_atual` (fix 05/07: reembolso do SAC em mês sem faturamento abria DRE vazio); fallbacks encadeados.
- `fin_escopo_ok(escopo, cc_tipo, tipo_loja)` — filtro de escopo **composto por vírgula**. `'franqueadora,proprias'` = franqueadora + lojas próprias, **sem franquias** (a receita da franquia não é dinheiro da franqueadora; só o royalty é). Invariante: `franqueadora+proprias+franquias == consolidado` sem dupla contagem. Lançamento sem centro cai no balde `franquias`.
- `fin_dre(ini,fim,escopo,unidade)` — DRE por competência (regime de exercício: suspenso permanece, cancelado sai).
- `fin_dre_anual(ano,escopo,unidade)` — 12 meses em colunas.
- `fin_fluxo(ini,fim,escopo,unidade)` — série mensal por data efetiva (`coalesce(data_caixa, data_prevista, competencia)`); suspenso e cancelado NÃO andam o caixa.
- `fin_fluxo_resumo(escopo,unidade)` — KPIs: `a_receber` (receita previsto), `recebido` (realizado/conciliado), `vencido` (previsto com data<hoje), `a_pagar`, `pago`.
- `fin_fluxo_composicao(escopo,unidade)` — "a receber" por conta do plano.

Produtores (server actions que escrevem no razão — `financeiro/actions.ts`)

- `apurarFaturamentoBemp(ano,mes)` — apura a RECEITA real do BEMP por unidade e tipo de venda (`CONTA_POR_ENTIDADE`); semântica de **substituição** (`repostLancamento('bemp',...)` — apaga a competência e regrava, reapurável). Avisa unidades sem centro de custo.
- `gerarRoyaltiesDoFaturamento(ano,mes)` — o núcleo. Faturamento por salon (RPC) × % → cria 2 recebíveis (Royalties, Fundo) por franquia com faturamento, e 4 eventos no razão por unidade (receita da rede + despesa da unidade, para royalty e fundo). Regras: **só FRANQUIA paga** (`tipo_loja != 'propria'`); % **padrão 10** (`fin_config.royalty_pct`), override por unidade (`royalty_pct_override`); **fundo hoje = 0** (não cobrado, validação CEO 02/07); vencimento dia X (`venc_dia/override`) do mês seguinte; **desconto automático**: faturamento `< teto` (80k) E sem recebível atrasado → royalty cai por `desc_pct` (ex.: 10%→5%). Idempotente por (`unidade_id, categoria, competencia`); re-vincula recebíveis ao razão após repost.
- `apurarDespesasDaCompetencia(ano,mes)` — apura imposto/comissão/taxa de cartão sobre a receita real já no razão. **Comissão vem da Matriz de Comissões** (`matriz_comissoes`, média das taxas efetivas por cargo; fallback `fin_config.comissao_pct`). Base de comissão configurável (`COMISSAO_BASE_OPCOES`). Substituição (`repostLancamento('despesa_config',...)`).
- `novaDespesa`, `importarRecebiveis`, `importarDespesas` — lançam manual (conta pelo nome da categoria; fallback 4.2.99).

O `RoyaltiesTab` dispara os 3 produtores em sequência (1 clique "Apurar mês"): faturamento → royalties → despesas.

A.1 — Fluxo de Caixa · `/financeiro (tab fluxo)`

1. **Rota/perm:** `/financeiro`, menu `financeiro`. · gate `admin_geral/financeiro/gestor`.
2. **O que faz:** visão de caixa derivada do razão — série de 6 meses (entradas×saídas), KPIs por status e composição do "a receber", com seletor de escopo (franqueadora/próprias/franquias/unidade).
3. **Telas:** `FluxoTab` — cards KPI (a receber, recebido, vencido, a pagar, pago), gráfico de barras 6 meses, composição por conta, projeção de caixa (`ProjecaoCaixa` usa recebíveis+contas a pagar+saldo realizado). Seletor de escopo refaz via `fluxoDoRazao` (server action).
4. **Backend:** RPCs `fin_fluxo` + `fin_fluxo_resumo` + `fin_fluxo_composicao`; `normalizaFluxo/janelaFluxo` (lib pura). Default de escopo: `franqueadora,proprias`.
5. **Integrações:** alimentado indiretamente pelo BEMP (via apuração) e royalties.
6. **Estado real:** **FUNCIONAL**. Evidência: `page.tsx` chama as 3 RPCs no load; suspenso fora do caixa (RPC `status not in ('cancelado','suspenso')`).
7. **Req p/ 100%:** nada crítico (depende de as apurações terem sido rodadas na competência).
8. **Esforço:** 0.

A.2 — DRE · `/financeiro/dre`

1. **Rota/perm:** idem. 2. **O que faz:** DRE derivado do razão por competência, com escopo (consolidado/franqueadora/unidades/próprias/franquias, combináveis por vírgula) e **por loja**, mais **visão anual** (12 meses).
2. **Telas:** `DreTab` — seletor mês + seletor de escopo (`EscopoPicker` com checkboxes) + seletor de unidade + toggle anual; tabela por grupo/conta/natureza.
3. **Backend:** `fin_dre` (mês) e `fin_dre_anual` (ano) via `dreDaCompetencia / dreAnual`. Escopo validado por `escopoValido` (whitelist DRE_ESCOPOS). Default do mês = `fin_ultima_competencia`; default de escopo `franqueadora, proprias`.
4. **Integrações:** — 6. **Estado real: FUNCIONAL.** Evidência: `fin_dre` join `plano_conta + centro_custo + unidades`, filtro `fin_escopo_ok`, exclui `cancelado`; dados reais mar/abr apurados.
5. **Req p/ 100%:** nada crítico. 8. **Esforço:** 0.

A.3 — Cálculos · `/financeiro/calc`

1. **Rota/perm:** idem. 2. **O que faz:** atualização de débitos em atraso — correção monetária por índice oficial + multa + juros de mora.
2. **Telas:** `CalcTab` — parâmetros (índice, multa %, juros % a.m., data, modo nominal/acréscimos); tabela dos recebíveis atrasados com Original/Correção/Multa/Juros/Atualizado + totais.
3. **Backend:** importa automaticamente recebíveis `status='atrasado'`. Correção = `valor × (acum12m/100) × dias/365`. **É uma calculadora (read-only)** — não persiste o resultado no razão.
4. **Integrações:** Banco Central (API SGS) REAL — `src/lib/indices-bcb.ts`, séries 189 IGP-M, 433 IPCA, 188 INPC, 432 SELIC, 4389 CDI, acumulado 12m, cache 6h.
5. **Estado real: FUNCIONAL** (com degradação honesta: se o BCB estiver indisponível, desativa a correção e mantém multa+juros).
6. **Req p/ 100%:** opcional — botão para lançar os acréscimos calculados de volta no recebível/razão (hoje só exibe). 8. **Esforço:** 0,5 dia (persistir acréscimos, se desejado).

A.4 — Contas a Receber (Franqueadora) · `/financeiro/receber`

1. **Rota/perm:** idem. 2. **O que faz:** sub-livro de recebíveis da franqueadora (royalties, taxas, locações) com ações por linha e "Nova conta a receber".
2. **Telas:** `ReceberTab` — tabela com filtros (`FiltroFinModal` : período/pessoa/descrição/valor), modal `NovaReceitaModal`, importação de planilha (`.xlsx / .csv` via SheetJS + modelo), ações por linha.
3. **Backend/ações:** `fin_recebiveis` (escopado por unidade). Ações: `gerarBoleto`, `darBaixaReceivel` (exige boleto; concilia o razão via `conciliarLancamento`), `escalarJuridico` (grava `jur_id`), `notificarCobranca`, `suspenderLancamento('receber')` (espelha no razão como `suspenso`, fora do fluxo). **Atraso derivado em read-time** (fix QA 05/07): `aberto` + vencido = `atrasado` (auto-corrigue sem cron). `importarRecebiveis` lança no razão (receita prevista, centro rede).
4. **Integrações:** boleto/banco (ver estado). 6. **Estado real: FUNCIONAL** para o ciclo interno (lançar/baixar/suspender/escalar); **boleto é simulado** (ver A.7).
5. **Req p/ 100%:** baixa por retorno bancário real. 8. **Esforço:** coberto por A.7.

A.5 — Contas a Pagar (Franqueadora) · `/financeiro/pagar`

1. **Rota/perm:** idem. 2. **O que faz:** sub-livro de despesas da franqueadora com prioridade, pagamento e "Nova despesa".
2. **Telas:** `PagarTab` — tabela + filtros, `NovaDespesaModal` (categoria vinda do plano de contas), importação de planilha, ações por linha.
3. **Backend/ações:** `fin_contas_pagar` (escopado por `escopo` = nome da unidade). Ações: `pagarDespesa` (concilia despesa manual no razão), `definirPrioridade` (alta/média/baixa), `novaDespesa` (lança no razão pela categoria — conta de mesmo nome no plano, fallback 4.2.99), `suspenderLancamento('pagar')`. `importarDespesas` idem.
4. **Integrações:** reembolsos do SAC também aparecem aqui (via `lancamentos_financeiros` — financeiro por UNIDADE — e razão da franqueadora conta 4.2.05). 6. **Estado real: FUNCIONAL.** 7. **Req p/ 100%:** nada crítico. 8. **Esforço:** 0.

A.6 — Conciliação Bancária · `/financeiro/conciliacao`

1. **Rota/perm:** idem. 2. **O que faz:** importa extrato (qualquer banco) e cruza esperado × recebido, marcando divergências.
2. **Telas:** `ConciliacaoTab` — tabela, botão "Rodar conciliação", `ImportExtratoModal` (usuário vincula colunas da planilha aos campos).
3. **Backend/ações:** `importarExtrato` (grava `fin_conciliacao` status `pendente`, via `adminClient`), `rodarConciliacao` (cruzamento linha a linha: `esperado` informado ou `venda×(1-taxa%)`; tolerância R\$0,05; marca `ok / divergencia`).

4. **Integrações:** importação **MANUAL** de planilha — não há feed OFX/Open Finance automático.
5. **Estado real:** **PARCIAL**. Evidência: matemática de conciliação é real, mas a origem do extrato é upload manual; sem integração bancária automática.
6. **Req p/ 100%:** conector bancário (OFX/CNAB/Open Finance) para puxar o extrato automaticamente. 8. **Esforço:** 3–5 dias (depende do banco/credenciais — decisão + integração).

A.7 — Automação de Royalties · `/financeiro/royalties`

1. **Rota/perm:** idem. 2. **O que faz:** pipeline de cobrança de royalties (sempre % do faturamento bruto, venc. dia X do mês seguinte): apura do BEMP, gera boleto, lança crédito, envia ao franqueado, baixa no retorno, escala atraso ao Jurídico.
2. **Telas:** `RoyaltiesTab` — cards (banco, competência, total royalties), pipeline visual de 6 passos, **botão "Apurar mês (faturamento + royalties)"** (dispara `apurarFaturamentoBemp` → `gerarRoyaltiesDoFaturamento` → `apurarDespesasDaCompetencia`), "Gerar cobrança" (`gerarCobrancaRoyalties`), "Processar retorno bancário" (`processarRetornoBancario`), "Rodar régua de atraso" (`rodarReguaAtraso`), console de log.
3. **Backend:** BEMP `fin_faturamento_por_salon`; royalties/fundo/despesas no razão (ver Produtores).
4. **Integrações:** **BEMP (real)** para faturamento; **boleto e retorno bancário são SIMULADOS**.
5. **Estado real:** **PARCIAL** — **apuração FUNCIONAL, cobrança bancária STUB**. Evidências: (a) `finBoletoNum()` em `src/lib/financeiro.ts` gera um **número de boleto formatado sinteticamente** (determinístico por seq), sem registro no banco; (b) banner amarelo no próprio `RoyaltiesTab`: "Este módulo **simula** o ciclo para validação do fluxo" (integração real por API/Open Finance/CNAB no servidor); (c) `processarRetornoBancario` marca **pago** boletos em aberto sem atraso, sem retorno bancário real; (d) `gerarCobrancaRoyalties` gera boletos sintéticos e loga "Enviado por e-mail e WhatsApp" **sem envio real**.
6. **Req p/ 100%:** integração bancária de registro/baixa de boleto (API/CNAB) + envio real e-mail/WhatsApp da cobrança. 8. **Esforço:** 4–6 dias (boleto/banco) + 1 dia (envio) — inclui decisão de banco/credenciais.

A.8 — Cobrança & Jurídico · `/financeiro/cobranca`

1. **Rota/perm:** idem. 2. **O que faz:** régua de cobrança por atraso; a partir de D+10 encaminha ao Jurídico.
2. **Telas:** `CobrancaTab` — card de inadimplência (unidades/valor), tabela de atrasados (contato via `finFranqEmail` por slug, próxima ação da régua, status), ações "Notificar"/"Jurídico", tabela da régua (configurável).
3. **Backend/ações:** recebíveis `status='atrasado'`; `proximoPassoRegua` (lib) sobre `config.regua`; `notificarCobranca` (placeholder — comentário no código: "Integração real de e-mail/WhatsApp acontece no servidor (placeholder honesto)"); `escalarJuridico` / `rodarReguaAtraso` (só gravam `jur_id`).
4. **Integrações:** **régua/notificação são STUB** (nenhum envio real; escalonar = flag `jur_id`). Cruza com o módulo Jurídico (que tem `juridico_notificacoes` reais — ver Submódulo D; hoje não há gatilho automático ligando a régua àquela tabela).
5. **Estado real:** **PARCIAL**. Evidência: a régua é configurável e a UI mostra o próximo passo real; mas notificação não envia e o "acionar Jurídico" só marca `jur_id` (não cria `juridico_notificacoes`).
6. **Req p/ 100%:** envio real (e-mail/WhatsApp) + gatilho automático régua → `juridico_notificacoes` + job/cron que rode a régua diariamente. 8. **Esforço:** 2–3 dias.

A.9 — Configurações · `/financeiro/config`

1. **Rota/perm:** idem. 2. **O que faz:** todos os parâmetros do financeiro da franqueadora + plano de contas + royalty por unidade.
2. **Telas:** `ConfigTab` — seções: **Royalties & cobrança** (royalty %, fundo %, dia venc., **desconto automático** teto+%), **Banco de cobrança**, **Regras de despesa** (imposto %+regime, comissão %+base, taxa cartão %), **Adquirentes** (taxas deb/cred/parc/pix/ prazo, add/remove), **Categorias de recebíveis** (Royalties protegida), **Régua de cobrança** (passos editáveis add/remove), **Plano de contas** (criar/ativar/excluir categorias), **Royalty por unidade** (override % + venc + tipo_loja própria/franquia).
3. **Backend/ações:** `salvarConfig` (upsert `fin_config`, validações: venc 1–28, % ≥0 ≤100); `criarContaPlano` / `setContaPlanoAtivo` / `removerContaPlano` (categoria do sistema com código não exclui; categoria com lançamentos no razão não exclui — só desativa); `salvarRoyaltyUnidade` (override por unidade, vale na próxima apuração).
4. **Integrações:** —. 6. **Estado real:** **FUNCIONAL**. Evidência: upsert real + CRUD do plano de contas + override por unidade. 7. **Req p/ 100%:** máscara/cofre real das credenciais do banco (hoje mascaradas na UI; integração real seria server-side). 8. **Esforço:** 0 (fora a integração bancária de A.7).

Submódulo B — SAC (11 folhas + camada de integração) — EM PRODUÇÃO REAL

Escopo centralizado (verificado): `session.ts` — se `papel === 'sac'`, `activeUnitId` é forçado a `null` e `activeUnitName='Franqueadora'`. Leituras usam RLS do usuário; webhook/ingest/buscar-cliente usam `adminClient`.

RBAC SAC (verificado): menu grupo `perm:'sac.'` (prefixo); leaf **Canais** sobrescreve `perm:'sac.canal'` (match exato → concedido a todos os cargos SAC). Papel `'sac'` sozinho NÃO dá acesso — precisa do vínculo de **cargo** (`atendente_sac / supervisor_sac / consulta_sac`). `consulta_sac` vê mas fica fora de distribuição/ranking. **Gate de ações é por papel** (`temPapel(papel, 'sac', 'gestor')`, admin sempre) — divergência com a nota "gate por recurso": no código as *actions* gateiam por papel; só a *visibilidade de menu* é por recurso. **Só `/sac/relatorios` tem gate de página** (`temPapel('sac', 'gestor', 'financeiro')`); as outras 10 páginas renderizam para qualquer logado que chegue à rota (proteção real = menu escondido + RLS + gate nas mutations).

B.1 Dashboard — `/sac`

- **Perm:** `sac.` (atendente/supervisor/consulta/gestor/admin). Sem gate de página.
- **Faz:** painel (6 KPIs + gráficos por canal/motivo/fase + reembolsos + chamados recentes) com filtro de período e multi-atendente.
- **Telas:** `SacDashFiltros`, `RelKpis`, 4x `BarChart`, tabela "Chamados recentes".
- **Backend:** `sac_tickets` (varredura paginada 1000/pág, tabulada em JS — evita ~33 counts), `sac_motivos`, `listAtendentesSac`. Tempo médio de resolução = média real (`concluido_em - criado_em`) (retorna vazio se nenhum). $SLA\% = (total - sla_violado) / total$.
- **Estado:** **FUNCIONAL**. Ressalva: `sla_violado` é lido mas nunca escrito por nenhuma action → "Em atraso"/"Taxa SLA" dependem de um job de SLA inexistente (provável sempre 0/100%).
- **Req/Esforço:** job que marque `sla_violado` conforme `slaHoras` da config — **0,5 dia**.

B.2 Chamados — `/sac/chamados`

- **Perm:** `sac.`; mutations `temPapel('sac', 'gestor')`.
- **Faz:** lista/busca/pagina (30/pág) e CRUD de chamados; ponto de entrada "leads do site → chamado".
- **Telas:** `SacFiltros`, modal `NovoChamado`, `ChamadosTabela`, paginação.
- **Backend:** `sac_tickets` (busca `or` em nome/protocolo/cpf/telefone/motivo/canal/unidade). Actions `criarChamado / atualizarChamado` (coerência fase↔status; 'Concluído'→resolvido+ `concluido_em`). **Dívida de schema:** tipo (Franquia/Própria) e data-reclamação não têm coluna → gravados no prefixo de `observacoes` (`montarObs / lerObsMeta`).
- **Integração:** `ingestSacBestEffort()` no load (throttle 1x/min) — dispara ingest site→chamado sem depender do cron.
- **Estado:** **FUNCIONAL**. **Req/Esforço:** colunas próprias tipo/data + persistir `sla_violado` — **1 dia**.

B.3 Kanban — `/sac/kanban`

- **Perm:** `sac.`; mutations `temPapel('sac', 'gestor')`.
- **Faz:** board por fase (7 colunas), mover, e gerar pedido de cancelamento/reembolso.
- **Telas:** `SacKanban` (7 colunas, 120 cards/coluna, totais reais).
- **Backend:** `sac_tickets` por fase (limit 120) + varredura paginada de `fase` p/ totais. Actions `moverTicketFase` (confere unidade via `scopeUnidade` antes de escrever — não confia só na RLS); `gerarPedidoCancelamento` (motivo=Reembolso, fase 'Em pagamento', grava `valor_devolucao / multa_aplicada`; **não** lança no Financeiro — isso é `solicitarReembolso`).
- **Estado:** **FUNCIONAL**. **Esforço:** 0.

B.4 Conversa (Triagem) — `/sac/triagem` ⚠ **EM PRODUÇÃO (IA + humano)**

- **Perm:** `sac.`; toda action passa por `guardTriagem = temPapel('sac', 'gestor')`.
- **Faz:** inbox WhatsApp do SAC — recebe (webhook), responde, transfere, assume, abre chamado, notas, respostas rápidas, inicia conversa. **Coração do módulo em produção.**
- **Telas/modais:** `TriagemWhatsapp` — lista (Todas/Minhas/Fila com counts), fio de mensagens, notas internas, painel do cliente (auto-import por CPF/telefone), "Nova conversa", respostas rápidas (barra `/`), status.
- **Backend:** `sac_whatsapp_chats`, `sac_whatsapp_mensagens`, `sac_whatsapp_notas`, `sac_respostas_rapidas`, `sac_tickets`, `sac_motivos`. Escopo por unidade **defensivo** (fallback se coluna não existir). Msgs buscadas DESC+reverse (corte ~1000 do PostgREST). Actions: `responderConversa / enviarMidia` (envia + grava + assume + `bot_ativo=false`), `assumirConversa`, `devolverConversa`, `transferirConversa`, `marcarLido`, `reativarIA`, `adicionarNota`, `alterarStatusConversa`, `descartarConversa`, `abrirChamadoDaConversa` (valida CPF 11díg/email, vincula `ticket_id`), `iniciarConversa`, `buscarClientePorContato` (**adminClient**, casa por CPF/telefone; agrega agendamentos + `bemp_billings` total gasto), CRUD respostas rápidas.
- **Integrações:** `@/lib/uazapi` (`sendText / sendMedia / normTel`), `@/lib/sac-midia` (bucket `sac-midia`). Roteamento de canal de envio prioriza canal de origem → canal da unidade → qualquer "laser" conectado (evita responder unidade B pelo número A). Avisa restrição 463 (iniciar conversa com número novo).

- **Estado:** FUNCIONAL / EM PRODUÇÃO. **Req/Esforço:** confirmar coluna `unidade_id` no schema de `sac_whatsapp_chats` (hoje defensivo) + realtime/poll (hoje `revalidatePath`) — **1–1,5 dia**.

B.5 Canais — `/sac/canais`

- **Perm:** override `sac.canal` (todos os cargos SAC). Admin vê todas as instâncias UAZAPI; não-admin só as vinculadas.
- **Faz:** gerencia canais WhatsApp **centrais** (escopo 'geral'/franqueadora).
- **Telas:** `CanaisManager` (modo central), cards com status de conexão + restrição de envio (463).
- **Backend:** `canais_whatsapp` (`escopo='geral'`; colunas `instancia_nome/escopo/unidade_id/rotulo/delay_min/delay_max/atendente_id`), `listAtendentesSac`. Só instâncias com nome `/laser/i`.
- **Integrações:** UAZAPI `listInstances`, `limitesEnvio` (restrição 463 `WHATSAPP_REACHOUT_TIMELOCK`), `uazapiConfigurado()` (exige envs). Vínculo `atendente_id` sustenta o "número próprio da atendente" do webhook.
- **Estado:** FUNCIONAL (depende de envs UAZAPI de produção). **Esforço:** 0,5 dia (validar envs/mutations do `CanaisManager`).

B.6 Relatórios — `/sac/relatorios`

- **Perm:** `sac.` + **gate de página real** `temPapel('sac','gestor','financeiro')` (única leaf com bloqueio no load).
- **Faz:** relatórios agregados (KPIs, breakdown canal/fase/prioridade/motivo/unidade/atendente, ranking por SLA, reembolsos) com export CSV.
- **Backend:** `sac_tickets` (varredura paginada tabulada, evita 60+ counts), `sac_motivos`, `listAtendentesSac`. SLA cumprido = `(total-violados)/total`.
- **Estado:** FUNCIONAL (depende de `sla_violado` real). **Esforço:** 0 (herda job SLA).

B.7 Pagamentos — `/sac/pagamentos`

- **Perm:** `sac.`; `podeBaixar=admin|| financeiro`; `podeValidar=admin|| gestor || financeiro`.
- **Faz:** espelho SAC↔Financeiro — reembolsos (Contas a Pagar) e acordos parcelados, com validação do gestor e baixa que fecha o chamado.
- **Telas:** `AcordosSac` (+modal `NovoAcordo`), `PagamentosSac`.
- **Backend:** `lancamentos_financeiros` (`ilike 'Reembolso SAC%'`), `sac_acordos` + embed `sac_parcelas`, `sac_tickets`. Actions (`sac/actions.ts`): `solicitarReembolso` (despesa em `lancamentos_financeiros`; guard anti-duplicidade por `origem_ref_id`; move ticket 'Em pagamento'; **também posta no RAZÃO** `fin_lancamento` conta 4.2.05, `idempKey sac:<ticket>:reembolso`, status 'previsto'); `criarAcordo` / `criarAcordoAvulso` (parcelas 1–24, regra dia-15 `primeiroPagamentoValido`, status 'aguardando_ok'); `validarAcordo` (gera N parcelas como lançamentos pendentes). `financeiro/actions-sac.ts`: `darBaixaLancamento` (paga; espelha de volta — parcela paga → acordo 'pago' → ticket 'Concluído' + `conciliarReembolsoRazao` marca `fin_lancamento` 'conciliado'), `receberLancamento`.
- **Integração:** ponte SAC→Financeiro por unidade (`lancamentos_financeiros`) + razão franqueadora (`fin_lancamento`).
- **Estado:** FUNCIONAL (fluxo criar→validar→baixar→conciliar→fechar, com idempotência). **Req/Esforço:** categoria dedicada "Reembolso SAC" no plano de contas (hoje fallback null gracioso) — **0,5 dia**.

B.8 Atendentes — `/sac/atendentes`

- **Perm:** `sac.`; `podeCriar=admin only`; `podeDistribuir=admin|| sac || gestor`.
- **Faz:** cria login de atendente, liga/desliga presença, troca cargo, ativa/desativa, distribui e reequilibra a fila.
- **Telas:** `AtendentesManager` (grid com carga/KPIs/prêmio/presença/cargo, filas não atribuídas, distribuir/reequilibrar).
- **Backend:** `listAtendentesSac`, `perfis_usuario` (`sac_online`), `usuario_cargos` + `cargos`, `sac_premiacao_config.pesos`, contagens reais em `sac_tickets` / `sac_whatsapp_chats`. Actions (todas com `audit_log`): `criarAcessoAtendente` (`adminClient auth.admin.createUser` papel 'sac' + upsert perfil + vincula cargo via `usuario_cargos` → `cargos`; sem cargo → `recursos=[]` sem acesso), `definirPresencaSac` / `definirPresencaAtendente`, `definirCargoAtendente` (menu só reflete após novo login — cache RBAC), `setAtendenteAtivo` (não pode auto-desativar), `distribuirFila` (só ONLINE, menos carregado, escopo unidade), `reequilibrarBacklog`.
- **Integração:** Supabase Auth Admin (cria login); base da auto-distribuição do webhook (`sac_online`).
- **Estado:** FUNCIONAL ("Novo atendente cria papel/cargo já cabeado" confirmado). **Req/Esforço:** invalidar cache de sessão ao trocar cargo (hoje exige re-login) — **0,5 dia**.

B.9 Ranking — `/sac/ranking`

- **Perm:** `sac.`. Sem gate de página.
- **Faz:** ranking de premiação monetária por atendente (período/unidade) + "Destaque do mês".

- **Backend:** `sac_premiacao_config.pesos`, `listAtendentesSac` (exclui consulta), varredura `sac_tickets` por `atribuido_para`, `premioValor / PREM_DEFAULT`.
- **Estado:** **PARCIAL**. Evidência: **vendas=0, pacotes=0, CSAT=0 hardcoded** (sem fonte real ligada ao atendente); prêmio calcula só o mensurável (atend./finaliz./reversão/SLA); rótulo é honesto.
- **Req/Esforço:** ligar vendas/pacotes/CSAT ao `atribuido_para` — **2 dias**.

B.10 Importar Leads — `/sac/importar`

- **Perm:** `sac.;` `importarTickets` `temPapel('sac','gestor')`.
- **Faz:** importa reclamações de planilha (Reclame Aqui/Procon/Sults/CSV) como chamados em lote (até 5000).
- **Backend:** `sac_tickets` insert em lotes de 500, `unidades / empresas`, `montarObs`; canal normalizado contra CHECK (fora→'Manual'); valida unidade permitida.
- **Estado:** **FUNCIONAL**. **Esforço:** 0.

B.11 Configurações — `/sac/config`

- **Perm:** `sac.;` `podeEditar=admin||sac||gestor`; actions `temPapel('sac','gestor')`.
- **Faz:** catálogos (motivos, tags), SLA em horas, pesos de premiação.
- **Backend:** `sac_motivos`, `sac_tags`, `sac_premiacao_config.pesos` (9 pesos + `slaHoras`, default 48h). Actions: CRUD motivo/tag, `salvarPremiacaoConfig`, `salvarSlaHoras` (1–1000h).
- **Estado:** **FUNCIONAL**. Ressalva: `slaHoras` gravado/lido mas nenhum código escreve `sac_tickets.sla_violado` → SLA configurado ainda não é aplicado (falta o job). **Esforço:** incluído em B.1.

B.INFRA — Camada de integração SAC

- **UAZAPI** (`src/lib/uazapi.ts`, `uazapiGO v2`): envs `UAZAPI_BASE_URL`, `UAZAPI_ADMIN_TOKEN`, `UAZAPI_TOKEN`. `listInstances / sendText / sendMedia / downloadMessage / limitesEnvio` (restrição 463). **LIVE**. Nº `5519997565531` restrito p/ iniciar conversas até 04/07 (463) — tratado no código.
- **Webhook de entrada** (`src/app/api/webhooks/uazapi/route.ts`): **EM PRODUÇÃO**. Auth por `UAZAPI_WEBHOOK_SECRET / UAZAPI_TOKEN` (exigido em prod). Grava chats+mensagens (dedup por `wa_id`), resolve canal→unidade/atendente, re-hospeda mídia, atualiza status de conexão. Insert defensivo.
- **IA de atendimento** (`src/lib/ia.ts`): **LIVE se OPENROUTER_API_KEY / AGENTE_IA_API_KEY**. Provider **OpenRouter** (compat. OpenAI), default `openai/gpt-4o-mini`. `gerarRespostaSAC` → JSON `{resposta,transferir,motivo,nomeCliente,cpf}` (roteiro oficial v1.0/1.1). No webhook: IA faz o 1º atendimento se `bot_ativo && sem atendente && iaConfigurada`; `transferir=true` → desliga bot, escolhe atendente online e **abre o chamado automaticamente** (pedido Julio 02/07). Sem IA → fila humana.
- **Auto-distribuição** (`src/lib/sac-distribuicao.ts`): `candidatosOnline` (papel 'sac' + `sac_online` + ativo + cargo operacional); embed desambiguado `usuario_cargos!usuario_cargos_perfil_id_fkey` (senão PGRST201). `escolherAtendenteOnline` = menos carregado. **LIVE**.
- **Auto-offline de atendentes:** **NÃO EXISTE** cron de auto-offline — `sac_online` é só toggle manual. **GAP** se a bíblia promete auto-offline.
- **Cron Vercel** (`vercel.json`): **1 cron** — `/api/cron/ingest-sac` diário `0 6 * * *` (`CRON_SECRET`), chama `ingestSacLeadsDoSite`.
- **Site form → chamado** (`src/lib/sac-ingest.ts`): lê `lasercompany_leads` (via `siteClient()`), filtra `tipo ∈ {sac,reclamacao,suporte,pos_venda...}`, cria `sac_tickets` na franqueadora (`empresa_id='0...0001'`, `unidade_id=null`, canal `formulario`), atribui a online, marca `roteado` (idempotente). Aciona-se por 3 vias: cron, `ingestSacBestEffort` no load de `/sac/chamados`, e a IA no webhook. **Ressalva:** o site **ainda não publica** formulário `tipo='sac'` → o ingest existe e funciona mas hoje **não materializa nada** (FUNCIONAL mas **ocioso**, aguardando o cliente publicar o form).

Submódulo C — Expansão (7 folhas) + Implantação (1)

Expansão é o CRM de **franquias** (pipeline separado do CRM comercial). Menu grupo `perm:'crm.lead'`. Base: `crm_leads / crm_etapas` com `pipeline='franquia'` (migration 050). Todas as páginas são **defensivas à migration ausente** (banner + estado vazio). RBAC de escrita: `criarLeadFranquia / moverEtapa / simularLeadFranquia` exigem operador; `admin_geral` sempre, demais precisam de unidade ativa (RLS confirma).

C.1 Dashboard — `/expansao`

- **Faz:** painel do funil de franquias (KPIs + funil por etapa + leads recentes), escopado por unidade.

- **Telas:** `ExpansaoTabs` (componente cliente).
- **Backend:** `crm_etapas` (`pipeline='franquia'`, ativo), `crm_leads` (lista capada 500 + **varredura paginada** de `etapa_id` para totais reais por etapa — evita fan-out de counts). Actions `criarLeadFranquia`, `moverEtapa`, `simularLeadFranquia`.
- **Estado:** **FUNCIONAL** (queries reais + feature-detect migration 050). **Esforço:** 0.

C.2 Captação de Leads (Geo + Site) — `/expansao/captacao`

- **Faz:** relatório **read-only** dos leads que entram automaticamente por `origem in ('geolocalizado','site')` no pipeline de franquia (KPIs 7 dias, por canal, entrada recente).
- **Backend:** `crm_leads` (`pipeline='franquia'`, origens de captação), escopo por unidade. Robustez: query falha → estado vazio.
- **Integração:** a "entrada por site" depende do webhook/simulação (`simularLeadFranquia` cria origem 'site'/geolocalizado' para validar). **Sem webhook de site publicado ainda** → alimentado hoje por simulação/manual.
- **Estado:** **FUNCIONAL (read-only), mas dependente de fonte** — o canal automático real (formulário do site de franqueado) ainda não emite. **Req/Esforço:** publicar o webhook/formulário de captação do site — **1–2 dias** (parte é decisão/cliente).

C.3 Funil — `/expansao/funil`

- **Faz:** funil de vendas de franquia (KPIs, barras por origem/temperatura, conversão por etapa) com filtro de período e export CSV.
- **Telas:** `RelFiltros`, `RelKpis`, `BarChart`, `ExportCsvButton`.
- **Backend:** `crm_leads` (`pipeline='franquia'`) com paginação (PULL_CAP 8000), `crm_etapas`. Rótulos de origem/temperatura do CHECK da migration 050.
- **Estado:** **FUNCIONAL** (dado real, paginado). **Esforço:** 0.

C.4 Leads (Kanban/Lista) — `/expansao/leads`

- **Faz:** lista filtrável de leads de franquia (busca, origem, temperatura, etapa, período) com export CSV.
- **Telas:** `LeadsFiltros`, `RelFiltros`, `RelKpis`, `ExportCsvButton`.
- **Backend:** `crm_leads` paginado + `crm_etapas`; ações de mover etapa via `moverEtapa`.
- **Estado:** **FUNCIONAL**. **Esforço:** 0.

C.5 Disparos WhatsApp — `/expansao/disparos`

- **Faz:** composer de disparo em massa (templates + listas) pelos canais WhatsApp conectados, com histórico.
- **Telas:** `DisparoComposer`, `DisparosResumo`.
- **Backend:** `canais_whatsapp` + `listInstances` / `uazapiConfigurado` (UAZAPI); `listarTemplates`, `dadosDisparos` (listas + histórico) de `expansao/disparos/actions.ts`.
- **Integração:** **UAZAPI (real)** — só canais `/laser/i` e `connected`. Sujeito à restrição 463 para números novos.
- **Estado:** **FUNCIONAL (depende de envs UAZAPI e canal conectado)**. É a única sub-folha de Expansão originalmente em `ROTAS_FUNCIONAIS` desde o início. **Req/Esforço:** validar throttle/anti-ban e templates de produção — **0,5 dia**.

C.6 WhatsApp CRM — `/expansao/whatsapp`

- **Faz:** relatório **read-only** das conversas de WhatsApp que alimentam o CRM (KPIs abertas/não-lidas/em atendimento/no bot + conversas recentes).
- **Backend:** reusa `sac_whatsapp_chats` / `sac_whatsapp_mensagens` (as únicas tabelas de conversa que existem no código — o comentário do arquivo confirma que `whatsapp_conversas` / `whatsapp_mensagens` NÃO existem). Sem filtro por unidade (coluna não confirmada). Robustez: query falha → "Relatório em preparação".
- **Estado:** **FUNCIONAL (read-only), com ressalva de escopo:** as conversas exibidas são as do **SAC** (não há inbox de WhatsApp separado para Expansão). **Req/Esforço:** se Expansão precisar de inbox próprio, é feature nova — **decisão + 2–3 dias**; como relatório está OK (0).

C.7 Tipo de Lead — `/expansao/tipos`

- **Faz:** relatório por linha de oferta (Ultracell/Quanta/Franquia/Ultracell Pro/Quanta Light) — leads, valor estimado, distribuição por etapa/temperatura.
- **Telas:** `RelFiltros`, `RelKpis`, `BarChart`, `ExportCsvButton`.
- **Backend:** `crm_leads` (`pipeline='franquia'`, agregação por `tipo_lead`) + `crm_etapas`. Cores espelham `EXP_TIPOS` do legado.
- **Estado:** **FUNCIONAL**. **Esforço:** 0.

C.8 Implantação de Unidade — /implantacao

- **Perm:** menu **sem perm** (visível a todos os logados); **edição** `ehAdmin || gestor` (`PAPEIS_EDITA`).
- **Faz:** cronograma de implantação da unidade (projeto → etapas → tarefas), estilo project plan, com CRUD.
- **Telas:** `ImplantacaoView` (projeto da unidade ativa ou o mais recente; etapas com tarefas).
- **Backend:** `implantacao_projetos`, `implantacao_etapas`, `implantacao_tarefas`. Actions (com `audit_log`): `salvarProjeto`, `definirSituacao`, `editarTarefa`, `adicionarTarefa/excluirTarefa`, `editarEtapa`, `adicionarEtapa/excluirEtapa`. Defensivo à tabela ausente (banner "migration").
- **Estado:** **FUNCIONAL** (CRUD real). Ressalva: sem projeto/tabela → estado vazio honesto. **Req/Esforço:** seed de template padrão de implantação (se desejado) — **0,5 dia**; núcleo 0.

Submódulo D — Jurídico — /juridico

- **Perm:** menu `admin:true` → **gate de página** `isAdmin` — **exclusivo** `admin_geral` ("O Jurídico é de acesso exclusivo da franqueadora").
- **Faz:** notificações extrajudiciais de cobrança, modelos de notificação, documentos contratuais por unidade e documentos para assinatura eletrônica.
- **Telas (abas):** `JuridicoTabs` — (1) **Cobranças/Notificações** (`CobrancasTab`), (2) **Modelos** (`ModelosTab`), (3) **Unidades & documentos** (`UnidadesTab`), (4) **Documentos para assinatura** (`JuridicoManager`).
- **Backend:**
 - `juridico_notificacoes` (id, unidade_id, fin_id, franqueado, cnpj, categoria, valor, vencimento, dias_atraso, assunto, corpo, status pendente/enviada, enviada_em). KPIs reais via `count:exact` (pendentes, enviadas, valor pendente, unidades em atraso).
 - `juridico_templates` (modelos: nome/assunto/corpo/ordem).
 - `juridico_documentos` (documentos contratuais por unidade: contrato/pré/COF) — cruzado com `unidades` (cnpj/ativa).
 - `documentosassinatura` + embed `signatarios_documento(status)` — fluxo de assinatura eletrônica com status (rascunho/em andamento/concluído/expirado), prazo, ordem sequencial, paginação 30/pág, filtros e KPIs por status. Detecção de migration ausente (`tabelaAusente`).
- **Integrações:** **cruza com A.8 (Cobrança & Jurídico)** — o "acionar Jurídico" do Financeiro hoje só grava `jur_id` no recebível; **não** cria `juridico_notificacoes` automaticamente (gatilho ausente). Assinatura eletrônica: há tabelas/fluxo, mas **não há evidência de integração com provedor externo (DocuSign/Clicksign)** no `page.tsx` — é gestão interna do status (verificar `actions.ts` do módulo para envio real).
- **Estado real:** **FUNCIONAL** para leitura/gestão (notificações, modelos, documentos, assinatura com dados reais e KPIs reais). **PARCIAL** nas integrações: (a) ponte automática régua→notificação ausente; (b) provedor de assinatura eletrônica externo não confirmado no código lido.
- **Req p/ 100%:** gatilho régua(Financeiro)→ `juridico_notificacoes` ; envio real da notificação (e-mail/WhatsApp/carta); integração de assinatura eletrônica com provedor (se exigido). **Esforço:** 3–5 dias (2 ponte+envio; 2–3 assinatura externa, se necessária).

Submódulo E — Auditoria — /auditoria

- **Perm:** menu `perm:'sistema.audit'` ; **gate de página** `ctx.isAdmin` → **exclusivo** `admin_geral` .
- **Faz:** visualizador **read-only** da trilha de auditoria (`audit_log`) — log imutável, com política de soft-delete ("nada é apagado; registros viram Ativo/Inativo").
- **Telas:** 4 KPIs (Eventos registrados, Hoje, Usuários, Política=Soft-delete), `AuditoriaFiltros` (busca, ação, usuário, resultado sucesso/erro, período), `AuditoriaTabela` paginada (25/pág).
- **Backend:** lê `audit_log` via `adminClient` (**service role, somente leitura** — o log fica fora da RLS de negócio; **nenhuma escrita** ocorre na página). Colunas: `usuario_id`, `acao`, `recurso_id`, `recurso_label`, `resultado`, `origem`, `ip`, `mensagem_erro`, `dados_depois`, `criado_em`. Resolve nomes via `perfis_usuario.nome_completo` . Opções de filtro (ações distintas + usuários) e KPIs por `count:exact` .
- **Integrações:** consome os `audit_log.insert(...)` espalhados pelas actions do sistema (ex.: Atendentes, Implantação). É o leitor central.
- **Estado real:** **FUNCIONAL** (leitura real, filtros, paginação, KPIs). Cobertura de auditoria depende de cada action gravar em `audit_log` (algumas gravam — Atendentes/Implantação; nem todas as actions de Financeiro/SAC gravam, então a trilha é **parcial em cobertura**, não em funcionalidade da tela).
- **Req p/ 100%:** padronizar `audit_log.insert` em todas as mutations sensíveis (Financeiro, SAC). **Esforço:** 1–2 dias (instrumentação transversal).

Tabela-resumo (item | estado | dias p/ 100%)

#	ITEM	ESTADO	DIAS
A.1	Financeiro · Fluxo de Caixa	FUNCIONAL	0
A.2	Financeiro · DRE (mês/anual/escopo/loja)	FUNCIONAL	0
A.3	Financeiro · Cálculos (BCB real)	FUNCIONAL	0,5
A.4	Financeiro · Contas a Receber	FUNCIONAL	0
A.5	Financeiro · Contas a Pagar	FUNCIONAL	0
A.6	Financeiro · Conciliação Bancária	PARCIAL (import manual; sem feed banco)	3–5
A.7	Financeiro · Automação de Royalties	PARCIAL (apuração BEMP OK; boleto/banco STUB)	4–6
A.8	Financeiro · Cobrança & Jurídico	PARCIAL (notificação/régua STUB)	2–3
A.9	Financeiro · Configurações	FUNCIONAL	0
B.1	SAC · Dashboard	FUNCIONAL (falta job SLA)	0,5
B.2	SAC · Chamados	FUNCIONAL (schema tipo/data)	1
B.3	SAC · Kanban	FUNCIONAL	0
B.4	SAC · Conversa/Triagem (produção)	FUNCIONAL	1–1,5
B.5	SAC · Canais	FUNCIONAL (envs UAZAPI)	0,5
B.6	SAC · Relatórios	FUNCIONAL	0
B.7	SAC · Pagamentos	FUNCIONAL	0,5
B.8	SAC · Atendentes	FUNCIONAL	0,5
B.9	SAC · Ranking	PARCIAL (vendas/CSAT=0 hardcoded)	2
B.10	SAC · Importar Leads	FUNCIONAL	0
B.11	SAC · Configurações	FUNCIONAL (falta job SLA)	0
B.INFRA	UAZAPI + webhook + IA + auto-distribuição	LIVE	—
B.INFRA	Auto-offline de atendentes	NÃO IMPLEMENTADO	0,5
B.INFRA	Cron ingest site→chamado	LIVE mas ocioso (site não emite <code>tipo='sac'</code>)	dep. cliente
C.1	Expansão · Dashboard	FUNCIONAL	0
C.2	Expansão · Captação (Geo+Site)	FUNCIONAL (read-only; fonte site pendente)	1–2
C.3	Expansão · Funil	FUNCIONAL	0
C.4	Expansão · Leads	FUNCIONAL	0
C.5	Expansão · Disparos WhatsApp	FUNCIONAL (UAZAPI)	0,5
C.6	Expansão · WhatsApp CRM	FUNCIONAL (read-only; reusa chats do SAC)	0
C.7	Expansão · Tipo de Lead	FUNCIONAL	0
C.8	Implantação de Unidade	FUNCIONAL	0,5
D	Jurídico	FUNCIONAL (integrações PARCIAIS)	3–5
E	Auditoria	FUNCIONAL (cobertura parcial)	1–2

Total de telas do módulo: 30 folhas (Financeiro 9 · SAC 11 · Expansão 7 · Implantação 1 · Jurídico 1 · Auditoria 1).

Esforço agregado p/ 100% (aprox.): Financeiro ~10–15 dias (dominado por integração bancária real: boleto/CNAB/Open Finance A.6+A.7, e ponte de cobrança A.8) · SAC ~5–6 dias (job SLA, schema, ranking, realtime) · Expansão/Implantação ~2–3 dias (fonte de captação do site + template) · Jurídico ~3–5 dias · Auditoria ~1–2 dias. **Dependências não-dev (cliente/decisão):** credenciais/ escolha de banco (boletos e conciliação), publicação do formulário `tipo='sac'` no site, e envs de produção UAZAPI/OpenRouter.

Pontos de atenção para homologação (confirmados no código): (1) boleto de royalties usa `finBoletoNum()` — número **sintético**, com banner explícito "simula o ciclo"; (2) régua/notificação de cobrança são **placeholders** (não enviam); (3) conciliação bancária depende de **upload manual** de extrato; (4) SAC `sla_violado` nunca é escrito (SLA não aplicado sem job); (5) **não há cron de auto-offline** de atendentes SAC; (6) o ingest site→chamado está pronto mas **ocioso** (site não publica `tipo='sac'`); (7) Expansão "WhatsApp CRM" reusa as conversas do SAC (não há inbox próprio).

Módulo 5 — Rede & Conta + Integrações Transversais + RBAC

Documento oficial de homologação. Estado auditado direto no código (não no discurso) em 2026-07-06. Backend real = Supabase `lkiihnznphxqekrgsgi` (lkii). Fonte de leads do site = Supabase separado `riutcbwilllvqjrpaefkb` (riut). Convenção de estado: **REAL** (funciona com dado/rede de verdade) · **PARCIAL** (funciona em parte, tem buraco declarado) · **STUB** (casca/alias/placeholder, sem lógica real).

PARTE A — Seção de menu "Rede & Conta"

Origem no menu: `src/lib/menu.ts` (seção **Rede & Conta**, linhas 196-205). Cinco itens, um deles com `perm`, os demais visíveis a qualquer logado. Todas as 5 rotas estão em `ROTAS_FUNCIONAIS` (menu.ts L243-244) → "acendem" no sidebar.

A.1 Minha Unidade — `/minha-unidade`

- **RBAC:** sem `perm` no menu → visível a **todo** usuário logado. Escrita gated na página: `podeEditar = ehAdmin(papel) || ['gestor','proprietario','operacoes'].includes(papel)`.
- **Arquivos:** `src/app/(app)/minha-unidade/page.tsx` · `src/components/unidades/MinhaUnidadePanel.tsx`.
- **O que faz:** mostra os dados da **unidade ativa** (`ctx.activeUnitId`, que é `perfis_usuario.unidade_id`). Query real: `unidades.select(id, nome, cnpj, endereco, cidade, estado, cep, ativa, bemp_salon_id).eq('id', activeUnitId)`. Se o usuário não tem `unidade_id` (ex.: `admin_geral/SAC` → `activeUnitId=null`), cai em `semUnidade` (estado vazio "selecione/associe uma unidade").
- **Telas/abas (5):** **Dados básicos** (editável, formulário real), **Horários**, **Bloqueios**, **Fotos**, **NFS-e**.
- **Estado real: PARCIAL.** Aba **Dados básicos** = REAL (lê e edita `unidades`). As outras 4 abas são **estado-vazio honesto** via componente `NeedsTable` — apontam para tabelas que **não existem** no lkii: `unidade_horarios`, `unidade_bloqueios`, `unidade_fotos`, `unidade_nfse_config` (evidência: `MinhaUnidadePanel.tsx` L65-68, cada aba renderiza `NeedsTable` com ícone `ti-database-off`).
- **Para 100%:** criar as 4 tabelas + CRUD (horários por dia da semana, bloqueios recorrentes/pontuais, galeria em Storage, config NFS-e). **Esforço: 4–5 dias.**

A.2 Todas unidades — `/unidades`

- **RBAC:** `perm: 'sistema.unidade'`. Gestão (ativar/inativar) gated: `podeGerir = ehAdmin(papel) || papel==='proprietario'`.
- **Arquivos:** `src/app/(app)/unidades/page.tsx` · `src/components/unidades/UnidadesManager.tsx`.
- **O que faz:** lista **real** das unidades da rede (≈82) com KPIs `count(exact, head)` de total/ativas/inativas, busca por `nome|cidade|cnpj` (`.or(ilike)`), filtro por UF (distinct em memória), status, paginação server-side (`.range`, `PAGE_SIZE=20`). Colunas incluem `bemp_salon_id` (chave da origem BEMP).
- **Estado real: FUNCIONAL/REAL** (queries reais + paginação + KPIs). RLS filtra o universo visível; `admin_geral` vê todas.
- **Para 100%:** confirmar CRUD completo de criação/edição de unidade dentro do `UnidadesManager` (a página cobre listagem/gestão; criação de nova unidade pode delegar à Implantação). **Esforço: 1 dia** (fechamento de CRUD).

A.3 Minha conta — `/minha-conta`

- **RBAC:** sem `perm` → todo logado. (No menu achatado do SAC/Financeiro é explicitamente preservada — Sidebar `canSee` mantém `/minha-conta`.)
- **Arquivos:** `src/app/(app)/minha-conta/page.tsx` · `actions.ts` (`salvarMinhaConta`) · `src/components/unidades/MinhaContaPanel.tsx`.
- **O que faz:** lê `perfis_usuario.select(id, nome_completo, email, telefone, papel, status).eq('id', user.id)`. Editar **nome e telefone** via action `salvarMinhaConta`. E-mail e papel são **read-only** (geridos em RH/auth).
- **Estado real: FUNCIONAL/REAL** para nome+telefone.
- **Falta p/ 100%:** troca de senha (não há chamada `auth.updateUser`), 2FA, foto/avatar, preferências de notificação. **Esforço: 1–2 dias.**

A.4 App do Cliente — `/app-cliente`

- **RBAC:** sem `perm` → todo logado.
- **Arquivos:** `src/app/(app)/app-cliente/page.tsx` · `src/lib/app-cliente.ts` · `src/components/app-cliente/AppClienteMockup.tsx`.

- **O que faz: prévia navegável** (mockup de celular) do app do consumidor, **alimentada com dados reais do tenant**: `servicos` (catálogo, top 12), `unidades` ativas, `colaboradores` ativos, e **um cliente real** (o de maior `saldo_pontos`, com `saldo_creditos`, próximo agendamento e histórico). Regras de fidelidade em `REGRAS_PONTOS` / `nivelDePontos` (Bronze/Prata/Ouro).
- **Estado real: PARCIAL**. Os *dados* exibidos são reais (queries a `servicos/unidades/colaboradores/clientes`), mas é **demonstrativo**: as ações dentro do telefone **não persistem** (docstring do `page.tsx`: "As ações dentro do telefone seguem sendo demonstrativas (sem persistência)"). Constantes mockadas remanescentes em `app-cliente.ts`: `APP_DATAS` (["Qua 11"...]), `APP_HORARIOS`, `APP_REDEEM`.
- **Para 100%**: definir se vira app real (agendar/resgatar de verdade) — é produto próprio (PWA/app nativo), não só uma tela. Como **prévia comercial** já cumpre. Como **app funcional: 15–25 dias** (fora de escopo do painel).

A.5 Exportações — `/exportacoes`

- **RBAC**: sem `perm` → todo logado (escopado por unidade ativa).
- **Arquivos**: `src/app/(app)/exportacoes/page.tsx` · `actions.ts` · `src/lib/exportacoes.ts` (`EXPORT_LIMIT=5000`) · `src/components/exportacoes/ExportacoesHub.tsx`.
- **O que faz**: hub com **contagens reais** (`count exact, head`) por dataset, escopadas pela unidade ativa na coluna própria de cada tabela: `clientes(unidade_origem_id)`, `lancamentos_financeiros(unidade_id)`, `agendamentos(unidade_id)`, `colaboradores(unidade_id)`, `sac_tickets(unidade_id)`. **6 datasets exportáveis** (`DatasetKey`): clientes, contas, leads, agendamentos, colaboradores, chamados. Cada `exportX()` gera linhas reais (limite 5000, flag `truncado`).
- **Leads do site**: exportação lê `lasercompany_leads` via `siteClient()` se `siteConfigurado()`, senão fallback `lkii.site_leads`.
- **Estado real: FUNCIONAL/REAL** (contagens + geração de linhas reais). Verificar apenas o *download* final (CSV/XLSX) no `ExportacoesHub` — as actions montam os `rows`; o encoding/entrega do arquivo é o último elo.
- **Para 100%**: confirmar geração/entrega do arquivo (CSV escaping) e formatos. **Esforço: 1 dia**.

Tabela-resumo — Parte A

ITEM	ROTA	PERM	ESTADO	DIAS P/ 100%
Minha Unidade	<code>/minha-unidade</code>	(logado)	PARCIAL (1 de 5 abas real)	4–5
Todas unidades	<code>/unidades</code>	<code>sistema.unidade</code>	REAL	1
Minha conta	<code>/minha-conta</code>	(logado)	REAL (falta senha/2FA)	1–2
App do Cliente	<code>/app-cliente</code>	(logado)	PARCIAL (prévia c/ dados reais, sem persistência)	15–25 (app real)
Exportações	<code>/exportacoes</code>	(logado)	REAL (validar entrega do arquivo)	1

PARTE B — Camada de Integrações Transversais

Fato estruturante: o único agendador que roda de verdade é o **Vercel Cron** (`vercel.json`). **Não há `pg_cron` nem `pg_net / net.http_post` em nenhuma migration** (grep em `scripts/` e `supabase/` = zero). E **não há biblioteca de e-mail** no `src/` (nenhum `resend/nodemailer/smtp/sendgrid`). **Evolution não aparece no código** — só UAZAPI. Isso baliza tudo abaixo.

B.1 Site → Sistema (leads + SAC automático) — REAL/PARCIAL

Dois caminhos, ambos cruzando o Supabase do site (`riut`, tabela `lasercompany_leads`) para o backend (`lkii`):

(a) Ingestão automática de SAC (cron): `src/app/api/cron/ingest-sac/route.ts` → `ingestSacLeadsDoSite()` em `src/lib/sac-ingest.ts`. - Agendado no `vercel.json`: { "path": "/api/cron/ingest-sac", "schedule": "0 6 * * *" } (diário 06:00, região `gru1`). Protegido por `CRON_SECRET` (Bearer). Também acionável por GET manual. - Lê `lasercompany_leads` (via `siteClient()`), cria `sac_tickets` na **franqueadora** (`empresa_id = 00000000-....-0001`, `unidade_id = null`), classifica `motivo_label` (`resolverMotivoSac`) e **auto-atribui** ao atendente com menos tickets abertos (`atribuirChamado`). Marca `dados`, `roteado` no lead de origem (idempotente). - **Estado: REAL**.

(b) Roteamento manual dos demais leads: `src/app/(app)/leads-site/page.tsx` (lista, fonte real `lasercompany_leads`) + `actions.ts:rotearSiteLead(siteLeadId, unidadeId)`. - Roteia por `tipo: curriculo` → RH (`vagas` "Banco de Talentos (Site)" + `candidatos`); `sac` → `sac_tickets` (franqueadora); demais tipos comerciais (`oferta/avaliacao/agendamento/franquia/indicacao`) → `crm_leads` (etapa inicial `pipeline='cliente'`, `responsavel_id`, `origem` mapeada). Marca `_roteado / _routed_to` na origem. - **Estado: REAL** (insert real em 3 destinos), com **triagem manual** (o operador escolhe a unidade de destino). Fallback: se a service-key do site não estiver configurada, usa `lkii.site_leads`.

Ponte / infra: `src/lib/supabase/site.ts` (`siteClient()` usa `SITE_SUPABASE_URL` + `SITE_SUPABASE_SERVICE_KEY`; a anon key do site só INSERE, a RLS bloqueia SELECT → a leitura exige service-key). `siteConfigurado()` gate. - **PARCIAL:** o roteamento comercial é **manual** (não há auto-roteamento por `unidade/unidade_email` → `unidades.id` como previa a "Opção A" do doc de ecossistema). O casamento texto-da-unidade → `unidade_id` não é automático fora do SAC. - **Para 100%:** auto-roteamento comercial por unidade + dedupe robusto + Realtime (ou cron) para os tipos não-SAC. **Esforço: 3–4 dias.**

B.2 WhatsApp — REAL (UAZAPI) / Evolution = fora do código

- **Provedor no código: UAZAPI** (uazapiGO v2), `src/lib/uazapi.ts`. Env: `UAZAPI_BASE_URL`, `UAZAPI_ADMIN_TOKEN`, `UAZAPI_TOKEN`. Funções reais: `listInstances`, `createInstance`, `connectInstance` (QR/paircode), `getStatus`, `disconnect/deleteInstance`, `sendText`, `sendMedia`, `downloadMessage`, `normTel`, `limitesEnvio`, `traduzErroEnvio`. `uazapiConfigurado()` gate.
- **Recebimento:** `src/app/api/webhooks/uazapi/route.ts` — grava em `sac_whatsapp_chats` + `sac_whatsapp_mensagens`, resolve o canal de origem contra `canais_whatsapp` (propaga `unidade_id`), re-hospeda mídia (`sac-midia.ts`), escolhe atendente online (`sac-distribuicao.ts`) e pode gerar resposta automática (`ia.ts:gerarRespostaSAC` se `iaConfigurada()`). Auth por `?secret=/header/body.token`. Cobre os dois envelopes UAZAPI.
- **Envio:** `sendText` / `sendMedia` por instância (token do canal) — disparado no clique (SAC, disparos). Rota a resposta **pelo mesmo número** que recebeu.
- **Evolution:** não existe no `src/` (grep = 0). É referência **operacional externa** (memória `reference-evolution-demandas: zap.cyberalpha.net`), não integração do sistema.
- **Restrição do número (erro 463 / `RESTRICT_ALL_COMPANIONS`):** tratada como *limite de envio* (`limitesEnvio` / `traduzErroEnvio`) — o número `5519997565531` esteve restrito para **iniciar** conversas via API (memória `project-laserco-whatsapp-463`). É restrição do WhatsApp, não bug do código.
- **Estado: REAL** (conectar, receber, enviar, mídia, IA opcional). **Para 100%:** confirmar produção com número liberado + reconexão automática de instância caída. **Esforço: 2 dias.**

B.3 API / Webhooks — REAL (2 rotas)

Só existem dois route handlers (`find src/app/api -name route.ts`): 1. `POST src/app/api/webhooks/uazapi/route.ts` — entrada de mensagens WhatsApp (ver B.2). Excluído do middleware de auth (matcher ignora `api/webhooks`) para aceitar POST sem cookie. 2. `GET src/app/api/cron/ingest-sac/route.ts` — cron de ingestão SAC (ver B.1), Bearer `CRON_SECRET`. - **Estado: REAL.** Não há endpoint público `/api/webhooks/leads-site` (a "Opção B" do doc nunca foi construída — o site grava direto no `riut`, não POSTa aqui). **Para 100%:** opcional adicionar webhook direto do site + rate-limit/log. **Esforço: 2 dias** (se decidirem sair do modelo pull).

B.4 Disparos programados / Automações — PARCIAL/STUB (sem motor)

- **Único agendamento real:** o Vercel Cron do SAC (B.1). **Nada mais é agendado.**
- `/automacoes` (`src/lib/automacoes.ts`): é um **catálogo de regras** (oferta pós-8-meses, confirmação de agendamento, reativação 60d, expiração de pacote, METAS gerente/consultora a cada 3 dias) com `ativoDefault` — mas **não há engine** que dispare por gatilho (nenhum `cron.schedule`, `setTimeout`, fila/queue). São definições + toggles de UI.
- `/disparos` (Disparos WhatsApp API) e `/expansao/disparos`: envio **na hora, por clique**, via `uazapi.sendText` (usam `scopeUnidade`). Não há fila/scheduler de campanha agendada.
- **Estado: STUB (automação por gatilho) / REAL (disparo manual imediato).**
- **Para 100%:** motor de automação — Vercel Cron adicional (ou pg_cron+pg_net) que avalie os gatilhos (agendamento criado, 60d inativo, pacote a 30d, métricas a cada 3 dias) e chame `sendText`/push; fila com retry. **Esforço: 6–9 dias.**

B.5 E-mail — STUB

- **Não há** biblioteca de envio de e-mail no `src/` (grep `resend/nodemailer/smtp/sendgrid/mailgun/@react-email` = 0). O único e-mail que sai é o **transacional do Supabase Auth** (login/reset), fora do app.
- **Estado: STUB. Para 100%:** provedor (Resend) + templates (confirmação, cobrança, comunicados, NF). **Esforço: 3–4 dias.**

B.6 Banco / Conciliação / Boleto (CNAB/API) — STUB

- As **8 sub-rotas** de "Financeiro Franqueadora" (`dre`, `calc`, `receber`, `pagar`, `conciliacao`, `royalties`, `cobranca`, `config`) são **aliases**: cada `page.tsx` é literalmente `import FinanceiroPage from '../page'` (re-renderiza o mesmo componente de abas). Ou seja, `/financeiro/conciliacao` **não é uma tela própria de conciliação.**
- **Fluxo de Caixa + DRE:** REAIS e razão-cêntricos (razão `lancamentos_financeiros` / `centro_custo` / `plano_conta`, dados reais mar/abr — ver `financeiro-ledger.ts` e memória `financeiro-razao`).
- **Conciliação Bancária (CNAB/OFX/API bancária):** não há parser CNAB, leitura OFX, nem chamada a API de banco. **STUB.**

- **Boleto:** não há geração/registro de boleto (nenhuma API bancária). **STUB.**
- **Automação de Royalties:** não há cálculo/geração automática de cobrança de royalty (sem cron; a rota é alias). Existe a *regra de negócio* (INATIVA paga royalty — commit 4787b84) e a base de razão, mas o disparo é manual/inexistente. **STUB→PARCIAL.**
- **Cobrança & Jurídico:** alias, sem régua de cobrança automatizada. **STUB.**
- **Para 100%:** import CNAB 240/400 + matcher de conciliação; integração de boleto (API Cobrança — Itaú/Sicoob/Asaas); job de royalties (cron mensal → gera `contas_receber` por unidade); régua de cobrança. **Esforço: 12–18 dias** (bloco financeiro pesado).

B.7 BEMP (import + robô de documentos) — REAL (dados) / BLOQUEADO (docs)

Scripts em `scripts/` (Node, rodam com service-key + `.env.local`): - `import-bemp-clientes.mjs` → **upsert** em `clientes` por `bemp_id` (saldos, etc.). **REAL / rodou** (memória: "dados reais do BEMP em tudo"). - `import-bemp-os.mjs` → staging + **insert into os** (empresa/unidade/cliente/status/preço/..., `bemp_id`, `on conflict do nothing`). **REAL.** - `import-bemp-colaboradores.mjs` → **upsert** de colaboradores. **REAL.** - `sync-bemp-operacional.mjs` / `backfill-whatsapp-historico.mjs` → sync operacional + histórico de WhatsApp. - **Importado:** clientes, OS, colaboradores (+ catálogo/agenda operacional via sync). Agenda: `agendamentos` existe e é grande (memória cita ~136k) — vinda do sync/BEMP. - **Robô de documentos** — `baixar-docs-bemp.mjs`: **infra pronta** (bucket privado `clientes-docs` path `bemp/<customer_id>/<tipo>/...` + tabela `clientes_documentos` + fila de **8.363 clientes com pacote**), mas **execução BLOQUEADA**: (1) precisa de `BEMP_WEB_EMAIL` / `BEMP_WEB_SENHA` **definitivos** (aguarda o Lucas concluir o Perfil no BEMP — mudar a senha antes invalida a credencial); (2) `fetchDocsDoCliente(_sessao, _customerId)` é **stub** — falta mapear no DevTools as rotas de auth/documentos do BEMP e preencher a função. **Estado: PARCIAL/BLOQUEADO (aguardando cliente).** - **Para 100%:** credencial definitiva + mapear endpoints BEMP + rodar a fila. **Esforço: 2–3 dias** após destravar credencial.

B.8 Storage (Disco Virtual / Google Drive) — REAL (Storage) / STUB (Drive API)

- `/disco` (`src/app/(app)/disco/page.tsx`): usa **Supabase Storage** — tabelas `disco_pastas` (árvore) e `disco_arquivos` (`arquivo_path`, `tipo`, `bytes`, `por`, `drive`). Upload/organização reais.
- **Google Drive:** `disco_config(drive_linked, drive_url)` — é apenas um **link salvo** para um Drive externo, **não** integração de API (sem OAuth/sync de arquivos do Drive). O flag `drive` por arquivo/pasta apenas marca origem.
- **Estado:** **REAL** para arquivos no Supabase Storage; **STUB** para Drive (só URL).
- **Para 100%:** OAuth Google Drive + sync bidirecional (se exigido). **Esforço: 4–6 dias** (só se quiserem Drive real; hoje o link já atende).

Tabela-resumo — Parte B

INTEGRAÇÃO	ARQUIVOS-CHAVE	ESTADO	DIAS P/ 100%
Site → SAC (cron)	<code>api/cron/ingest-sac</code> , <code>lib/sac-ingest.ts</code> , <code>vercel.json</code>	REAL	—
Site → CRM/RH (manual)	<code>leads-site/actions.ts</code> , <code>lib/supabase/site.ts</code>	PARCIAL (sem auto-roteamento)	3–4
WhatsApp (UAZAPI)	<code>lib/uazapi.ts</code> , <code>api/webhooks/uazapi</code>	REAL	2
Evolution	—	NÃO integrado (externo)	n/a
API/Webhooks	2 route handlers	REAL	2 (opcional)
Automações por gatilho	<code>lib/automacoes.ts</code> , <code>/disparos</code>	STUB (sem motor)	6–9
E-mail	—	STUB	3–4
Conciliação bancária (CNAB/ OFX)	<code>/financeiro/conciliacao</code> (alias)	STUB	6–8
Boleto (API banco)	—	STUB	4–6
Automação de Royalties	<code>/financeiro/royalties</code> (alias)	STUB→PARCIAL	4–6
Cobrança & Jurídico	<code>/financeiro/cobranca</code> (alias)	STUB	3–4
DRE / Fluxo de Caixa	<code>lib/financeiro-ledger.ts</code>	REAL	—
BEMP — clientes/OS/colab	<code>scripts/import-bemp-*.mjs</code>	REAL (importado)	—
BEMP — robô de docs	<code>scripts/baixar-docs-bemp.mjs</code>	PARCIAL/BLOQUEADO (aguarda credencial)	2–3
Storage — Disco	<code>/disco</code> , Supabase Storage	REAL	—

INTEGRAÇÃO	ARQUIVOS-CHAVE	ESTADO	DIAS P/ 100%
Storage — Google Drive	<code>disco_config.drive_url</code>	STUB (só link)	4–6

PARTE C — RBAC e Perfis (franqueado × franqueador)

C.1 O modelo (como o código realmente gate)

Existem **DUAS** camadas de RBAC em paralelo, e é preciso conhecer as duas para homologar:

Camada 1 — papel (enum legado, em `perfis_usuario.papel`). Usada por `src/lib/rbac.ts` (`PAPEL_ADMIN='admin_geral'`, `ehAdmin`, `temPapel`, `exigirPapel`) e por `src/lib/session.ts` (`isAdmin = papel==='admin_geral'`). É a checagem **grossa** usada em **Server Actions** e em alguns **page-guards** (ex.: `financeiro/page.tsx` gate `temPapel(papel,'financeiro','gestor')`; `os/page.tsx` write por `PAPEIS_ESCRITA`). Papéis observados: `admin_geral`, `sac`, `gestor`, `proprietario`, `operacoes`, `financeiro`, `colaborador` (default).

Camada 2 — recursos (RBAC granular por cargo). `session.ts:resolveRecursos` resolve `usuario_cargos` → `cargo_permissoes` → `permissoes.recurso_id` (via **service-role**, ignorando RLS). Cada permissão = `recurso` (`modulo.entidade`, ex.: `financeiro.caixa`, `sac.ticket`) × `acao` (`ler/criar/editar/deletar/aprovar/exportar/admin`) × escopo. `admin_geral` **bypassa** (recebe `recursos=[]` e vê tudo). É essa camada que **filtra o menu** (`Sidebar.canSee/hasPerm`): `perm` terminando em `.` = prefixo de módulo; sem `.` = recurso exato.

Estrutura de dados (backend Ikii, migrations 009/010 + `scripts/migrations/perfis-acesso.sql`): `empresas` → `unidades` (franquia com `empresa_id`); `cargos` (sistema `is_sistema` + custom por empresa) `n:m` `cargo_permissoes` → `permissoes` → `recursos` × `acoes`; `usuario_cargos` liga `perfis_usuario` ↔ `cargos`. Extras: `cargos.bate_ponto` (migration `rbac.sql`), `cargos.slug` (deriva nível SAC). **Atenção PGRST201:** o embed `perfis_usuario→usuario_cargos` é ambíguo (2 FKs) — usar `usuario_cargos!usuario_cargos_perfil_id_fkey`.

Onde o RBAC é (e não é) aplicado — verdade de homologação: - **Menu:** filtrado de verdade por `recursos` (Sidebar). ✓ - **Middleware** (`src/middleware.ts`): **só checa autenticação** (usuário logado), **não** autorização por rota. Um usuário que digite a URL direta de uma tela fora do seu menu **não é barrado pelo middleware**. - **Page/Action-level:** enforcement **inconsistente** — algumas páginas gate por `papel` (`financeiro`, `os`), a maioria **não** hard-gate e confia na **RLS do Supabase** como backstop. `grep redirect('/')|notFound()` nas pages = 0 guards de permissão. - **RLS** é a 2ª (e às vezes única) linha de defesa real para acesso a dado.

C.2 Perfis existentes e o que cada um enxerga no menu

Seed em `scripts/migrations/perfis-acesso.sql` cria **17 cargos-de-sistema** (`empresa_id` null) com permissões por módulo. Mapeando `recursos` → seções visíveis via `Sidebar.canSee`:

PERFIL (CARGO)	RECURSOS (MÓDULO × AÇÕES)	O QUE ENXERGA NO MENU
Super Administrador (<code>perfil_super_admin</code>)	<code>* / *</code>	Tudo, inclusive <code>sistema.*</code> (Perfis, Auditoria, Unidades).
Administrador (<code>perfil_administrador</code>)	todos os módulos exceto <code>sistema</code>	Tudo de negócio; some Auditoria/Perfis/Unidades.
Diretor	<code>* / ler,exportar,aprovar</code>	Vê tudo em leitura.
Financeiro (<code>perfil_financeiro</code>)	<code>financeiro / *</code> (módulo único)	Menu achatado só-dinheiro (<code>ehSoModulo='financeiro'</code>): Financeiro Franqueadora + Contas + categorias/formas/comissões + relatórios/dashboards financeiros + Minha conta.
Marketing	<code>marketing/*</code> + <code>crm/ler</code>	Marketing, Automações, Campanhas, Canais; CRM leitura.
RH	<code>rh/*</code> + <code>treinamento/*</code>	Recursos Humanos (grupo), Ponto, Universidade.
Expansão	<code>crm/*</code> + <code>comercial/ler</code> + <code>marketing/ler</code>	Expansão (grupo), CRM, Leads do Site.
SAC (papel <code>sac</code>)	<code>sac/*</code>	Menu SAC-only (grupo SAC), centralizado na franqueadora. Recorte por nível de cargo (ver abaixo).
Jurídico	<code>financeiro/ler,exportar</code> + <code>sac/ler</code>	Financeiro (leitura) + SAC (leitura).
TI	<code>sistema/*</code>	Config/Perfis/Auditoria/Unidades.
Auditor	<code>* / ler,exportar</code>	Tudo em leitura/exportação.

PERFIL (CARGO)	RECURSOS (MÓDULO × AÇÕES)	O QUE ENXERGA NO MENU
Franqueado (<code>perfil_franqueado</code>)	<code>comercial/* + operacoes/* + financeiro/ler + rh/ler + treinamento/ler</code>	Cadastros, Clientes, Agenda, OS, Relatórios comerciais; e vê o menu Financeiro (financeiro.) por causa do financeiro/ler .
Gerente de Unidade	<code>comercial/* + operacoes/* + rh/ler + financeiro/ler</code>	Igual franqueado, sem treinamento.
Supervisor	<code>comercial/ criar,editar,ler,exportar + operacoes/ler + sac/ler</code>	Comercial (escrita) + operações/SAC leitura.
Comercial/Recepção	<code>comercial/ criar,editar,ler,exportar</code>	Cadastros/Clientes/Agenda/OS.
Profissional Técnico	<code>comercial/ler + operacoes/ler</code>	Só leitura de agenda/comercial/operações.

Recorte fino do SAC (por cargo, `session.nivelSac` via `cargos.slug`): `supervisor_sac` vê tudo; `atendente_sac` só `{/sac, /sac/chamados, /sac/kanban, /sac/triagem, /sac/canaais}`; `consulta_sac` acrescenta `relatorios` e `ranking` (Sidebar `SAC_ATENDENTE / SAC_CONSULTA`).

Menu de módulo único (`ehSoModulo`): quando **todos** os recursos do usuário começam com `sac` ou `financeiro`, o Sidebar **achata** o menu para só aquele módulo + Minha conta (pedido do cliente: "menu personalizado como o SAC"). Só `sac` e `financeiro` têm esse tratamento.

C.3 Franqueador × Franqueado — como o sistema separa hoje e o que FALTA

Como o escopo funciona hoje (concreto): - O escopo por unidade é derivado de `perfis_usuario.unidade_id` → `session.activeUnitId`. O seletor de unidade do header foi REMOVIDO (03/07) e o cookie `lc_unit` deixou de ser honrado (`session.ts` L130-135). Ou seja: **um usuário está preso à sua única `unidade_id`**. - A separação de dado por unidade é **manual e não-universal**: `src/lib/sb.ts:scopeUnidade(q, activeUnitId)` faz `.eq('unidade_id', activeUnitId)` **só quando é chamado** (≈89 pontos no código). Onde não é chamado, **não filtra** (depende da RLS). - **Franqueador (`admin_geral`):**

`activeUnitId=null` → `scopeUnidade` vira **no-op** → vê **todas** as unidades. `activeUnitName='Todas as unidades'`. Bypassa recursos. - **Franqueado (cargo `perfil_franqueado`, `unidade_id` preenchido):** `activeUnitId=<sua unidade>` → onde houver `scopeUnidade/.eq('unidade_id')`, vê só a sua loja. Vê o menu de comercial/operações + Minha Unidade (essa sim escopada).

O que um franqueado veria de diferente hoje: menu de Cadastros/Clientes/Agenda/OS/Relatórios comerciais + "Minha Unidade" com os dados da própria loja; contas/relatórios escopados **quando** a query aplica `unidade_id`.

O que AINDA FALTA para uma visão de franqueado sólida (gaps concretos): 1. **Vazamento de escopo no Financeiro.** O menu "Financeiro Franqueadora" é gated por `perm:'financeiro.'`, e `perfil_franqueado` tem `financeiro/ler` → **o franqueado VÊ o menu Financeiro da franqueadora**. Pior: as telas de Financeiro (Fluxo/DRE) são **nível-rede/empresa** (razão por `centro_custo/empresa_id`), **não escopadas à unidade do franqueado**. **Não existe um "DRE/Fluxo da MINHA unidade"**. → Precisa: (a) separar recurso `financeiro.unidade` (loja) de `financeiro.franqueadora` (rede); (b) uma tela financeira escopada por `activeUnitId`. 2. **Sem multi-unidade por franqueado.** `activeUnitId` é uma unidade e o seletor foi removido. Um franqueado dono de 2+ lojas (ou "RH de várias franquias", caso de uso citado no ecossistema) **só enxerga uma**. → Precisa: reintroduzir seletor de unidade **restrito às unidades do usuário** + tabela de vínculo usuário ↔ múltiplas unidades (o `escopo_permissao='empresa'` do modelo 009 **não é aplicado** no app). 3. **Enum de escopo não implementado.** O modelo tem `escopo_permissao` (`global/empresa/unidade/proprio`), mas o app **só usa papel + recursos + uma unidade_id**. Não há enforcement de `empresa` (dono franqueado que vê todas as unidades da sua empresa) nem de `proprio` (só os próprios dados). → Precisa: honrar o escopo por recurso na resolução (hoje `resolveRecursos` traz só `recurso_id`, descarta o escopo). 4.

Enforcement por rota ausente. Como o middleware só autentica, um franqueado pode **abrir URLs fora do menu** (ex.: `/auditoria`, `/financeiro/dre`) e só a RLS o barra (se houver policy). → Precisa: guard central por `recursos/escopo` no `layout/` middleware (deny-by-default por rota). 5. **Sem white-label / visão própria do franqueado.** Não há tema/cor por franquia, subdomínio, nem dashboard "meu negócio" próprio — o franqueado usa o **mesmo layout roxo** com menu filtrado. `disco_config` guarda um `drive_url`, mas não há branding por unidade. → Precisa (se exigido comercialmente): tema por `unidade/empresa`, home do franqueado com KPIs só da sua loja. 6. **`admin_franqueado` ≈ `admin_geral` no menu.** O `papel` só distingue `admin_geral`; qualquer outro cai em `recursos`. Um "franqueado admin" com recursos amplos veria quase tudo **em nível de rede** porque as telas de gestão/financeiro não são escopadas. A distinção "franqueadora vê tudo × franqueado vê a sua" **existe de fato só para as telas que chamam `scopeUnidade`** — não é garantia sistêmica.

Resumo da separação atual: franqueador = `admin_geral` (`unidade_id` null, no-op de escopo, vê tudo). Franqueado = cargo com recursos + 1 `unidade_id`, escopado **onde a query lembra de escopar**, sem escopo `empresa`, sem multi-unidade, sem financeiro próprio, sem branding próprio, sem deny-by-default por rota.

Tabela-resumo — Parte C

ITEM	ESTADO	DIAS P/ 100%
Camada <code>papel</code> (rbac.ts) — guards grossos	REAL (mas inconsistente entre páginas)	—
Camada <code>recursos</code> (cargos) — filtro de menu	REAL	—
Seed 17 perfis (<code>perfis-acesso.sql</code>)	REAL (idempotente)	—
Recorte fino SAC por cargo	REAL	—
Enforcement por ROTA (middleware/layout)	STUB (só autentica)	3–4
Escopo <code>empresa</code> / <code>proprio</code> (enum 009) no app	STUB (não aplicado)	5–7
Multi-unidade por usuário (seletor restrito)	FALTA (seletor removido)	3–4
Financeiro escopado à unidade do franqueado (DRE da loja)	FALTA (só nível rede)	6–8
Corrigir vazamento: franqueado vendo Financeiro Franqueadora	FALTA (separar recurso)	2
White-label / visão própria do franqueado	FALTA	6–10

Apêndice — Evidências-chave (arquivos)

- Menu/estado funcional: `src/lib/menu.ts` (seção Rede & Conta L196-205; `ROTAS_FUNCIONAIS` L215-256).
- Sessão/escopo: `src/lib/session.ts` (`activeUnitId` L130-144; `resolveRecursos` L63-82; `nivelSac` L52-59).
- RBAC menu: `src/components/layout/Sidebar.tsx` (`canSee` / `hasPerm` / `ehSoModulo` L23-58; sets SAC L12-13).
- RBAC guards: `src/lib/rbac.ts`; middleware `src/middleware.ts` (só auth).
- Seed perfis: `scripts/migrations/perfis-acesso.sql`; `scripts/migrations/rbac.sql`.
- Escopo query: `src/lib/sb.ts` (`scopeUnidade` L28-30).
- Site→sistema: `src/lib/sac-ingest.ts`, `src/app/(app)/leads-site/actions.ts`, `src/lib/supabase/site.ts`, `vercel.json`, `src/app/api/cron/ingest-sac/route.ts`.
- WhatsApp: `src/lib/uazapi.ts`, `src/app/api/webhooks/uazapi/route.ts`, `src/lib/sac-distribuicao.ts`, `src/lib/sac-midia.ts`.
- Financeiro (alias): `src/app/(app)/financeiro/{dre,calc,receber,pagar,conciliacao,royalties,cobranca,config}/page.tsx` = `import FinanceiroPage from '../page'`; razão `src/lib/financeiro-ledger.ts`.
- BEMP: `scripts/import-bemp-{clientes,os,colaboradores}.mjs`, `scripts/baixar-docs-bemp.mjs`, `scripts/sync-bemp-operacional.mjs`.
- Storage: `src/app/(app)/disco/page.tsx` (`disco_pastas` / `disco_arquivos` / `disco_config`).